

OpenTelemetry ve Jaeger

Kapsamlı Teknik Referans Rehberi

*Junior İçin Öğretici, Senior İçin Referans,
SRE İçin Operasyonel Rehber, Mimar İçin Karar Dokümanı*

Distributed Tracing, Metrics, Logs, Context Propagation,
Sampling, Collector Pipeline, Production Operations

Argela Technologies — Fault Management Stajı Bağlamı

Mayıs 2026

İçindekiler

Aşağıda 37 ana bölümün listesi ve sayfa numaraları yer alır. Alt başlıklar için Word'ün sol navigasyon panelini (View → Navigation Pane) veya CTRL+F arama özelliğini kullanın.

Önsöz 4

Bölüm 1 — Observability'nin Tarihi ve Mühendislik Felsefesi	5
Bölüm 2 — Metrics, Logs, Traces: Derin Karşılaştırma	9
Bölüm 3 — OpenTracing, OpenCensus ve OpenTelemetry'nin Doğuşu	11
Bölüm 4 — OpenTelemetry Mimarisi: Üst Düzey Görünüm	13
Bölüm 5 — OpenTelemetry SDK Internals	17
Bölüm 6 — Context Propagation Internalleri	21
Bölüm 7 — Span Anatomisi: Attribute, Event, Link, Status	25
Bölüm 8 — Semantic Conventions ve Resource Attributes	28
Bölüm 9 — Instrumentation: Auto, Manual, Library, Zero-Code	31
Bölüm 10 — Java Agent Internalleri ve Bytecode Manipülasyonu	35
Bölüm 11 — Spring Boot Entegrasyonu ve Micrometer İlişkisi	38
Bölüm 12 — Async, Reactive ve Virtual Thread'lerde Tracing	41
Bölüm 13 — OpenTelemetry Collector Derin İnceleme	44
Bölüm 14 — OTLP Protokolü ve gRPC vs HTTP	49
Bölüm 15 — Collector Pipeline Tasarım Pattern'ları	52
Bölüm 16 — Exporter'lar ve Backend Entegrasyonu	55
Bölüm 17 — Jaeger Mimarisi ve Bileşenleri	58
Bölüm 18 — Jaeger Storage Backend'leri: Elasticsearch, OpenSearch, Cassandra	61
Bölüm 19 — Tempo, Zipkin ve Diğer Tracing Backend'leri	64
Bölüm 20 — Prometheus, Grafana, Loki ile Tam Stack	66
Bölüm 21 — Sampling: Head, Tail, Adaptive ve Bütünleşik Stratejiler	70
Bölüm 22 — JVM Üzerinde OpenTelemetry'nin Performans Maliyeti	74
Bölüm 23 — Cardinality, Metric Explosion ve Cost Bombs	77
Bölüm 24 — Capacity Planning ve Maliyet Optimizasyonu	79
Bölüm 25 — Docker Compose ile Tam Observability Stack	81
Bölüm 26 — Kubernetes Üzerinde Open Telemetry	86
Bölüm 27 — Helm Chart'lar, Kustomize ve GitOps Akışı	89
Bölüm 28 — High Availability, Disaster Recovery ve Scaling	91
Bölüm 29 — TLS, mTLS ve Authentication	94
Bölüm 30 — PII, GDPR ve Veri Maskeleme	96
Bölüm 31 — Multi-Tenant Mimari ve RBAC	99
Bölüm 32 — Observability Governance, SLO/SLI ve Incident Response	101

Bölüm 33 — Troubleshooting Handbook: 100+ Gerçek Vaka	104
Bölüm 33B — Troubleshooting Handbook (devam): Vakalar	61-130 120
Bölüm 34 — Production Checklist	137
Bölüm 35 — Sık Sorulan Sorular (FAQ)	139
Bölüm 36 — Argela Fault Management Case Study	141
Bölüm 37 — Observability'nin Geleceği: eBPF, Profiles, AI	147

Önsöz

Bu döküman, OpenTelemetry ve Jaeger ekosistemini Türkçe olarak en derin biçimde belgelemek üzere tasarlanmıştır. Yüzeysel anlatım değil; production deneyimine dayanan mühendislik bakış açısı sunar. 37 bölüm, 130+ troubleshooting vakası ve 100'den fazla kod örneği içerir.

Döküman dört seviyede de hizmet verir: junior bir Java developer için adım adım öğretici, senior bir engineer için referans, SRE için operasyonel rehber, mimar için karar dökümanı.

Önce Bölüm 1-4'ü okuyup observability'nin tarihsel ve mimari arka planını oturtmanız, sonra ilgilendiğiniz alana göre dalış yapmanız önerilir. Troubleshooting Handbook (Bölüm 33) bir başvuru kataloğu; baştan sona okunması gerekmez. Production checklist (Bölüm 34) deploy öncesi son kontrol için.

Döküman, Argela Technologies'deki Fault Management staj projesini referans alarak yazılmıştır. Spring Boot 3, Java 17/21, OpenTelemetry Java Agent 2.x, Jaeger v2, PostgreSQL, Docker Compose ve Kubernetes ekseninde örnekler verir. Ancak döküman büyük ölçüde proje-agnostiktir; herhangi bir kurumda OpenTelemetry referansı olarak kullanılabilir.

Konuların derinlik sırası şöyledir: temeller (1-4) → SDK ve context (5-8) → instrumentation (9-12) → collector ve OTLP (13-16) → Jaeger ve backend'ler (17-20) → sampling ve performans (21-24) → deployment (25-28) → security ve governance (29-32) → troubleshooting handbook (33) → production checklist + FAQ + Argela case study + gelecek (34-37).

Bölüm 1 — Observability'nin Tarihi ve Mühendislik Felsefesi

Observability terimi mühendislik literatürüne ilk olarak 1960'larda kontrol teorisi alanında girdi. Rudolf E. Kálmán, doğrusal dinamik sistemlerin durumunu yalnızca dış çıktılarına bakarak tam olarak çıkarabilme kabiliyetini matematiksel olarak tanımladı. Bu tanım, yazılım sistemleri için doğrudan uygulanabilir değildi; ancak temel sezgi aynıydı: bir sistemin iç durumunu, sadece onun yaydığı dış sinyallere bakarak ne kadar iyi anlayabiliyorsanız, o sistem o kadar gözlemlenebilirdir.

Dağıtık yazılım sistemleri için bu kavramı modern anlamıyla popülerleştiren ekip, 2010'ların ortalarında Twitter'ın altyapı ekibiydi. Aynı dönemde Google, Borg ve Spanner gibi mega ölçek sistemlerinin işletilmesinden öğrendikleri dersleri Dapper makalesi (2010) ile yayınladı. Dapper, dağıtık sistemlerde tek bir kullanıcı isteğinin onlarca servisi nasıl gezdiğini görselleştirmek için 'trace' kavramını formalize etti ve bugünkü distributed tracing endüstrisinin temelini attı.

1.1 Monitoring'den Observability'ye Geçiş

Monitoring ve observability sıklıkla birbirinin yerine kullanılır; ama bu iki kavram aynı şey değildir. Monitoring, bilinen sorunları bilinen metriklerle takip etme pratiğidir. CPU %80'i geçtiğinde alarm çalar; disk doluluk oranı %90'a vardığında PagerDuty arar. Monitoring soru-cevap odaklıdır: "Cevabını önceden bildiğim soruları operasyonel olarak izliyorum."

Observability ise daha farklı bir vaattir: sistemden gelen sinyaller o kadar zengin olmalıdır ki, daha önce hiç sormadığınız soruların cevabını da bu sinyallerden çıkarabilesiniz. Charity Majors'ın sıkça tekrar ettiği gibi, "observability bilinmeyen-bilinmezleri (unknown unknowns) anlamak içindir." Yeni bir hata sınıfı ortaya çıktığında, dashboard'larınızı yeniden yapılandırmak zorunda kalmadan elinizdeki telemetri verisini sorgulayıp kök nedeni bulabiliyorsanız, sistem gözlemlenebilirdir.

1.1.1 Monolit Çağında İzleme

2000'lerin başında tipik bir kurumsal sistem tek bir uygulama sunucusunda çalışan büyük bir monolitten ibaretti. Bir hata oluştuğunda mühendisin yapacağı şey basitti: SSH ile sunucuya bağlan, /var/log altındaki dosyaları tail et, stack trace'i oku, sorunu çöz. Nagios, Zabbix gibi araçlar sistem seviyesinde metrikleri (CPU, RAM, disk, ağ) topluyor; uygulama logları ise text dosyaları halinde merkezi olmadan dağınık duruyordu.

Bu modelin sınırları çok netti. Birden fazla sunucu olduğunda 'tail -f' yetmiyordu; rsyslog ve daha sonra ELK (Elasticsearch, Logstash, Kibana) stack'i bu boşluğu doldurmaya başladı. Ancak ana paradigma değişmemişti: hata olduğunda gidip log'a bakılırdı.

1.1.2 Mikroservis Patlaması ve Eski Yöntemlerin Tıkanması

2014-2018 arasında konteyner devrimi (Docker, ardından Kubernetes), continuous deployment kültürü ve organizasyonel olarak iki-pizza ekipleri, monolitleri parçalayan büyük bir göçü tetikledi. Netflix, Uber, Airbnb gibi şirketler yüzlerce, sonrasında binlerce mikroservise sahip oldular. Tek bir kullanıcı isteği artık 30-50 servis üzerinden geçiyordu. Bir API çağrısının yavaşladığı raporlandığında, mühendisin "hangi servis yavaşladı?" sorusuna cevap vermesi günlerce sürebiliyordu.

Bu noktada eski metrik+log paradigması çöktü. Metrikler, hangi servisin yavaşladığını agrega düzeyde gösterse de, belirli bir isteğin neden yavaşladığını anlatamıyordu. Loglar ise her servisin kendine ait olduğundan, tek bir isteğin tüm yolunu loglardan yeniden inşa etmek ("log correlation") inanılmaz zahmetliydi. Distributed tracing kavramı, tam olarak bu eksikliği doldurmak için hayata geçti.

1.1.3 Google Dapper Makalesi

Google'ın 2010'da yayımladığı "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure" makalesi, modern distributed tracing'in tohumudur. Makale üç temel ilke öne sürdü:

- **Düşük genel maliyet (low overhead):** Trace toplama, prod sistemde fark edilebilir bir yavaşlama yaratmamalı. Dapper bunu sampling ile çözdü; bütün isteklerin değil, sadece bir kısmının izi tutuldu.
- **Uygulama-şeffaf (application-level transparency):** Geliştirici, her servise tek tek tracing kodu enjekte etmek zorunda kalmamalı. RPC ve thread kütüphanelerine instrumentation gömerek otomatize edildi.
- **Ölçeklenebilir:** Sistem milyonlarca servis çağrısı/saniyeyi kaldırabilmeli. Dapper'da bu, asenkron toplama, batch'leme ve Bigtable üzerine kalıcılaştırma ile sağlandı.

Dapper, span (tek bir iş birimi) ve trace (birden çok span'ın oluşturduğu ağaç) kavramlarını ilk kez net olarak ortaya koydu. Bugün OpenTelemetry'de gördüğümüz API yüzeyinin büyük çoğunluğu, doğrudan veya dolaylı olarak Dapper'dan miras kalmıştır.

1.1.4 Açık Kaynak Tracing: Zipkin, Jaeger, OpenTracing

Dapper makalesinden hemen sonra Twitter, kendi iç implementasyonunu Zipkin (2012) olarak açtı. Uber ise 2015'te Jaeger projesini başlattı ve 2017'de CNCF'e bağışladı. İki proje de benzer modeli benimsedi: dilden bağımsız tracer kütüphaneleri, merkezi bir toplayıcı, kalıcı bir depolama (Cassandra veya Elasticsearch) ve trace görselleştirme için web arayüzü.

Ancak her tracing kütüphanesi kendi API'si ile gelince, ekosistem parçalandı. Bir Java servisinde Zipkin Brave, başka bir Python servisinde OpenTracing Jaeger client kullanılınca, span'lar arası bağlantı kopuyor; mühendis hangi kütüphaneyi öğreneceğine karar veremiyordu. Bu sorunu çözmek için 2016'da OpenTracing spesifikasyonu hayata geçti: vendor-neutral, dile gömülü bir tracing API. Yaklaşık aynı dönemde Google, OpenCensus projesini yayımladı; bu proje metric ve trace API'lerini birleştirmek istiyordu.

İki rakip standardın olması doğal olarak kafa karışıklığı yarattı. 2019'da iki projenin yöneticileri bir araya gelip iki projeyi birleştirmeye karar verdi. Bu birleşmenin adı OpenTelemetry oldu. OpenTelemetry, OpenTracing ve OpenCensus'un iyi yanlarını alıp yeni bir API+SDK seti çıkardı. 2021'de CNCF Incubating, 2024 itibarıyla Tracing ve Metrics sinyalleri Graduated statüsüne ulaştı.

1.2 Üç Sinyalin Yükselişi: Metrics, Logs, Traces

Endüstri uzun süre "three pillars of observability" söylemiyle metrikler, loglar ve trace'leri eşit ağırlıkta üç ayrı sinyal olarak konumlandırı. Bu sınıflandırma operasyonel olarak hâlâ değerli; ama mühendislik açısından bakıldığında her birinin neyi modellediği farklıdır.

Sinyal	Veri Yapısı	Tipik Kullanım	Tipik Maliyet
Metric	Sayısal zaman serisi (timestamp + value + labels)	Sistem sağlığı, SLO ölçümü, alerting	Düşük (saniyede milyonlarca data point)
Log	Zaman damgalı serbest metin veya yapılandırılmış JSON	Hata kök nedeni, audit, ayrıntılı bağlam	Yüksek (compress edilmiş bile büyük)
Trace	Span ağacı (parent-child ilişkili olaylar)	İstek bazlı performans, dağıtık akış görselleştirme	Orta (sampling ile yönetilir)

1.2.1 Metrikler — Toplu Verinin Gücü ve Sınırı

Metrikler, belirli bir boyutta agrega edilmiş sayısal değerlerdir. "Son 1 dakikada login endpoint'ine gelen istek sayısı" bir metriktir. Metriklerin asıl gücü, çok yüksek hacimde veriyi sabit bir bellek alanında temsil edebilmeleridir: 1 saniyede 1 milyon istek geliyorsa bile, sayaç tek bir long integer olarak ilerler.

Bu güç aynı zamanda sınırdır. Tek bir metric satırı (counter, histogram, gauge) hiçbir zaman tek bir isteğe ait detayı içermez. "Login endpoint'i bugün ortalama 50ms'de cevap veriyor" derken, p99'da 5sn yiyen bir kullanıcının olduğunu görmek için histogram'lara ihtiyacınız vardır; o kullanıcının kim olduğunu, hangi DB sorgusunun yavaşlattığını metrik tek başına söyleyemez. Bu bağlamda metrikler "sistemin nabızı", logs ve trace'ler ise "röntgen filmi" gibidir.

1.2.2 Loglar — Detaylı Bağlam, Yüksek Maliyet

Loglar, sistemin belirli anlarındaki olayları metinsel olarak kaydeder. INFO seviyesinde 'User 42 logged in', ERROR seviyesinde stack trace içeren bir mesaj — bunların hepsi log'tur. Modern observability'de loglar artık structured logging (JSON) olarak üretilir; çünkü `timestamp=... level=ERROR user\_id=42 reason=invalid\_token` formatı, sonraki sorgu motorlarında çok daha verimli aranır.

Logların asıl problemi maliyettir: 1 saniyede 100k log satırı üreten bir sistem, günde ~8.6 milyar satır demektir. Elasticsearch'e index'lemek diske, CPU'ya ve özellikle SSD ömrüne ciddi yük getirir. Bu yüzden olgun ekipler log seviyelerini dikkatle ayarlar, debug logs'u prod'da kapatır ve örnekleme (sampling) kurallarını log seviyesine kadar indirir.

1.2.3 Trace'ler — İstek Bazlı Doğru Resim

Bir trace, tek bir isteğin sistemin içinde geçtiği bütün yolu temsil eder. Frontend kullanıcı butona basar, request gateway'e gider, oradan auth servisine, oradan kullanıcı servisine, ardından da veritabanına. Her duraktaki süre, hata, attribute'lar tek bir trace ID altında toplanır.

Trace'ler bir nevi metric'i ve log'u birleştirir: her span hem süre bilgisi içerir (metric benzeri), hem de attribute ve event'ler aracılığıyla bağlam (log benzeri). Tracing'in temel zorluğu, context propagation'dır: span A, span B'nin parent'ı olduğunu nereden bilecek? Cevap: HTTP/gRPC çağrılarında W3C Trace Context başlıklarının taşınması (bkz. Bölüm 6).

1.3 Üç Sinyalin Ötesi: Profiles ve Continuous Profiling

Son 2-3 yılda endüstri üç sinyalin yetmediği görüşüne varmaya başladı. Özellikle CPU profiling, memory profiling gibi sürekli profillemeye verisi de bir telemetri sinyali olarak konumlanmaya başlıyor. Grafana Pyroscope, Polar Signals Parca, eBPF tabanlı Pixie, Datadog Continuous Profiler gibi araçlar bu yeni sinyalin temsilcileri. OpenTelemetry topluluğu da 2024 itibarıyla 'profiles' adlı dördüncü sinyali resmî olarak spesifikasyona ekleme çalışmalarını sürdürüyor.

1.4 Observability Olgunluk Modeli

Bir ekibin observability uygulamasındaki yetkinliği genellikle 5 seviyede modellenir. Bu model, organizasyonun nereye yatırım yapması gerektiğine karar vermek için kullanışlıdır.

Seviye	Tipik Durum	Eksiklik
L0 Reaktif	Müşteri arıyor, prod kapalı; SSH ile log bakılıyor	Hiç merkezi telemetri yok
L1 Temel	ELK + CPU/RAM grafikleri var	Trace yok; correlation manuel
L2 Yapılandırılmış	JSON loglar, Prometheus metrikleri, basit alerting	Trace zayıf; SLO yok
L3 Distributed	OpenTelemetry + Jaeger/Tempo, dashboard'lar prod-grade	Maliyet kontrolü, sampling stratejisi gelişmemiş
L4 Adaptive	Tail sampling, RUM, profiling, error budget, otomatik anomali tespiti	Yedi 9 saplantısı yoksa burası iyi noktadır

Argela'daki Fault Management projeniz, bu modelde L2'den L3'e geçişe denk gelen iyi bir staj başlangıcıdır: structured log + Prometheus + Jaeger + OpenTelemetry hepsi bir araya geliyor.

1.5 Bölüm 1 — Kritik Notlar ve Sık Yapılan Hatalar

PROD TAVSİYESİ

Observability bir araç seti değil, mühendislik kültürüdür. Yeni servis prod'a çıkmadan önce 'instrumentation review' adımı code review'un parçası olmalı.

Üç sinyali tek başına değerlendirmeyin: metric size 'bir şey yanlış' der, trace 'nerede yanlış' der, log 'tam olarak ne yanlış' der.

Telemetri sinyallerinin tamamı bir cost center'dır. Maliyet/değer dengesini en başından kurun (bkz. Bölüm 24).

ANTI-PATTERN

'Önce instrumentation yapalım, sampling sonra' demek: prod'a çıktığınız gün backend'in çökmesi demektir.

Sadece log yazıp 'observability'miz var' demek: kuruyemiş yiyip 'akşam yemeği yedim' demektir.

Üç sinyalda da aynı bilgiyi dublike etmek (örn. her metric'i ayrıca log olarak yazmak): maliyet artar, sinyal kirlenir.

Bölüm 2 — Metrics, Logs, Traces: Derin Karşılaştırma

Bu bölüm üç ana sinyali yalnızca tanım düzeyinde değil, mühendislik tradeoff'ları açısından karşılaştırır. Bir feature'ı izlerken hangi sinyalin kullanılacağı kararının nasıl verileceğini, hangi sinyalin neyi yapamayacağını açıkça koymak amacıyla yazılmıştır.

2.1 Bilgi Yoğunluğu (Information Density)

Metric, log ve trace farklı bilgi yoğunluklarına sahiptir. Bir metric satırı tipik olarak 50-200 byte iken, bir trace 5-50 KB, ayrıntılı bir log satırı ise 500 byte ile 5 KB arasındadır. Bu rakamlar prod ölçeğinde devasa fark yaratır: günde 1 milyar isteklik bir sistem için sadece full trace toplamak 5-50 TB demektir. Sampling olmadan bu sürdürülebilir değildir.

Sinyal	Boyut/satır	Cardinality limiti	Tipik retention
Metric (Prometheus)	~50-200 byte (TSDB sıkıştırılmış)	Yüksek (etiket başına 10k+ kötü)	15-90 gün
Log (ES)	~1-3 KB	Düşük (sınırsız metin)	7-30 gün
Trace (Jaeger ES)	~5-50 KB	Yok (sampling şart)	3-14 gün

2.2 Sorgulama Modeli Farkları

Üç sinyalin sorgulama dili farklıdır. PromQL toplam (aggregation) odaklıdır:

`rate(http_requests_total[5m])`. Lucene/KQL serbest metin aramaya optimize edilmiştir. Trace sorgu motorları (Jaeger, Tempo) ise span attribute eşleştirmesine ve trace ağacı topolojisine bakar.

Bunun pratik sonucu: 'Bana en yavaş 10 trace'i göster' diye Prometheus'tan sorgu yazamazsınız. Aynı şekilde 'Login endpoint'inin son 5 dakikadaki istek sayısı kaç' sorusunu Jaeger ile cevaplamak teknik olarak mümkün ama çok yavaş ve pahalıdır. Her sinyal kendi sorgu motoruyla anlamlıdır.

2.3 Sinyaller Arası Korelasyon

Olgun bir observability platformunda asıl güç, sinyaller arasında köprü kurabilmektir. Bu köprü'nün anahtarı genellikle `trace_id`'dir. Bir log satırı `trace_id` taşıyorsa, o log'tan tek tıkla aynı isteğin trace'ine geçilebilir. Tersine, trace'i incelerken o spans için yazılmış log'lar listelenebilir. Aynı şekilde 'Bu metric spike'ı hangi trace'lerden geliyor?' sorusunu exemplars (Prometheus 2.43+) ile cevaplanabilir.

```
# Spring Boot loglarında trace_id örneği (Logback MDC + OTel bridge)
2026-05-23 09:14:23.142 [http-nio-8080-exec-3] INFO
com.argela.collector.AlarmService -
Alarm received [alarmId=ALM-7782, sourceIp=10.0.42.15]
trace_id=4bf92f3577b34da6a3ce929d0e0e4736
span_id=00f067aa0ba902b7
```

2.4 Sinyal Seçimi Karar Tablosu

Soru / İhtiyaç	Önerilen Sinyal	Neden
Sistemim genel olarak sağlıklı mı?	Metric	Toplu sağlık ölçümü ucuzdur
Hangi servisin SLO'su tehlikede?	Metric + SLO calc	Burn rate analizi metric'le yapılır
Bu kullanıcının request'i neden yavaşladı?	Trace	Sadece trace istek-bazlı detay verir
Stack trace'in tam metni nedir?	Log	Trace span event'i veya structured log
DB hangi sorguyu çalıştırdı?	Trace (otomatik instrumentation)	Span attribute olarak db.statement
Bir feature flag'i kimleri etkiledi?	Log (audit) veya metric (counter)	Audit gereksinimi varsa log şart
Process memory'de ne tutuluyor?	Profile	Heap profile metric/log/trace ile çözülmez

2.5 Bölüm 2 — Anti-Pattern'ler ve Tavsiyeler

ANTI-PATTERN

Trace verisini metric olarak kullanmak: 'her span'dan saniyede bir metric türetelim' = cardinality patlaması.

Loglara 30 farklı sayısal alan koyup metric olarak kullanmak: log toplama sistemleri toplamaya değil aramaya optimize edilmiştir; pahalı ve yavaş.

Aynı veriyi üç sinyalde aynı format/etiketle göndermek: storage, ağ, lisans maliyeti birden 3x.

PROD TAVSİYESİ

Her servis için minimum sinyal seti tanımlayın: 4 golden signal metric (rate, errors, duration, saturation) + structured logs + OTEL trace.

Sinyaller arası köprü ZORUNLU: trace_id log'ta olmalı, exemplar metric'e bağlı olmalı.

Çekirdek dashboard'lar SLO odaklı olmalı, vanity dashboard (CPU yüzdesi) operasyonel olmaz.

Bölüm 3 — OpenTracing, OpenCensus ve OpenTelemetry'nin Doğuşu

Bugün OpenTelemetry'yi tek standart olarak görmek, mevcut sistemlerin 2016-2020 arasında ne kadar parçalı olduğunu unutturmamalı. Bu bölüm, geriye dönüp neden iki standardın birleştiğini ve OpenTelemetry'nin neden bugünkü mimarisine sahip olduğunu açıklar. Bu, salt tarihçe değil; çünkü ileride göreceğiniz birçok API tercihi (ör. span'ın neden interface olduğu, tracer provider'ın neden global olduğu) doğrudan bu evrimin izlerini taşır.

3.1 OpenTracing — Vendor-Neutral API Denemesi

OpenTracing (2016, CNCF), tracing kütüphanelerini standartlaştırma çabasıydı. Felsefe basitti: bir 'Tracer' interface'i tanımlanır; uygulama kodu yalnızca bu interface'e bağımlıdır. Hangi vendor'un (Jaeger, Zipkin, Datadog, Lightstep) implementasyonunu seçeceğinizi runtime'da belirlenir. Bu, dependency injection mantığının tracing'e uygulanmış halidir.

OpenTracing'in temel API'sinde Tracer, Span, SpanContext ve Carrier (HTTP header taşıyıcı) abstractions vardı. Bu zarif bir tasarımdı ama eksiklikleri de hızla görüldü:

- Yalnızca trace'i kapsıyordu; metric ve log ayrı tutuluyordu.
- Semantic conventions (ör. `http.method`, `db.statement`) spec'in dışındaydı; her vendor kendi attribute isimlerini koyuyordu.
- Auto-instrumentation tarafı zayıftı; her dilde kütüphane yazarları manuel olarak tracing eklemek zorunda kalıyordu.

3.2 OpenCensus — Tek API, Tracing + Metrics

Google merkezli OpenCensus (2017), aynı sorunu farklı bir açıdan ele aldı. Trace ve metric'i tek bir API içinde birleştirmek istedi. Stats (metric), Trace, Tags (attribute) ve Exporter abstractions vardı. OpenCensus, OpenTracing'e göre daha 'opinionated' bir kütüphaneydi; SDK ve API birbirinden çok ayrılmamıştı.

OpenCensus'un avantajı, metric + trace'i aynı zincirden geçirebilmesi ve Stackdriver gibi Google Cloud Operations entegrasyonunun çok iyi olmasıydı. Dezavantajı, vendor-neutrality OpenTracing'e göre daha az hissettirilmesi idi. Sonuç: pazar ikiye bölündü.

3.3 Birleşme: OpenTelemetry

2019 Mayıs'ında iki proje birleşeceğini duyurdu. Birleşmenin temel ilkeleri:

1. API ve SDK net bir şekilde ayrılacak. API'yi uygulama yazılımı kullanır; SDK ise yapılandırma, batching, exporter detaylarını sağlar.
2. Üç sinyal (traces, metrics, logs) tek bir projede ele alınacak.
3. Semantic conventions ilk-sınıf vatandaş olacak.
4. Tüm sinyalleri taşıyan tek bir protokol (OTLP) tanımlanacak.
5. Vendor-neutral collector hayata geçirilecek.

Bugün bildiğimiz OpenTelemetry, bu beş ilkenin somutlaşmış halidir. 2021'de CNCF Incubating, 2024 itibarıyla Tracing ve Metrics 'Stable' statüsünde, Logs Beta'da.

3.4 OpenTracing'den OpenTelemetry'ye Migrasyon

Hâlâ OpenTracing tabanlı eski sistemleri olan ekipler mevcut. Migrasyon için OpenTelemetry bir 'shim' (köprü kütüphanesi) sunar:

6. OpenTelemetry SDK + Exporter kurulumu yap, ama eski OpenTracing kodlarına dokunma.
7. OpenTracing GlobalTracer'ı shim üzerinden OTel tracer'a bağla.
8. Eski span üretim kodlarını yavaş yavaş OTel API'sine taşı (servis bazında).
9. Tüm servisler OTel'e geçince shim'i kaldır.

3.5 Bölüm 3 — Kritik Notlar

NOT

OpenTelemetry API ve SDK ayırımına dikkat: uygulama kodu sadece API'ye bağımlı olmalı; SDK'yı sadece konfigürasyon katmanı görmeli. Bu sayede ileride exporter değiştirmek tek satırlık iştir. Vendor-neutrality demek 'kimseyi bağlamıyoruz' değil; 'aynı API ile farklı backend'lere yazabiliyoruz' demektir. Yine de attribute isimlerinizi semantic conventions'a uydurmazsanız, backend değiştirdiğinizde dashboard'larınız kırılır.

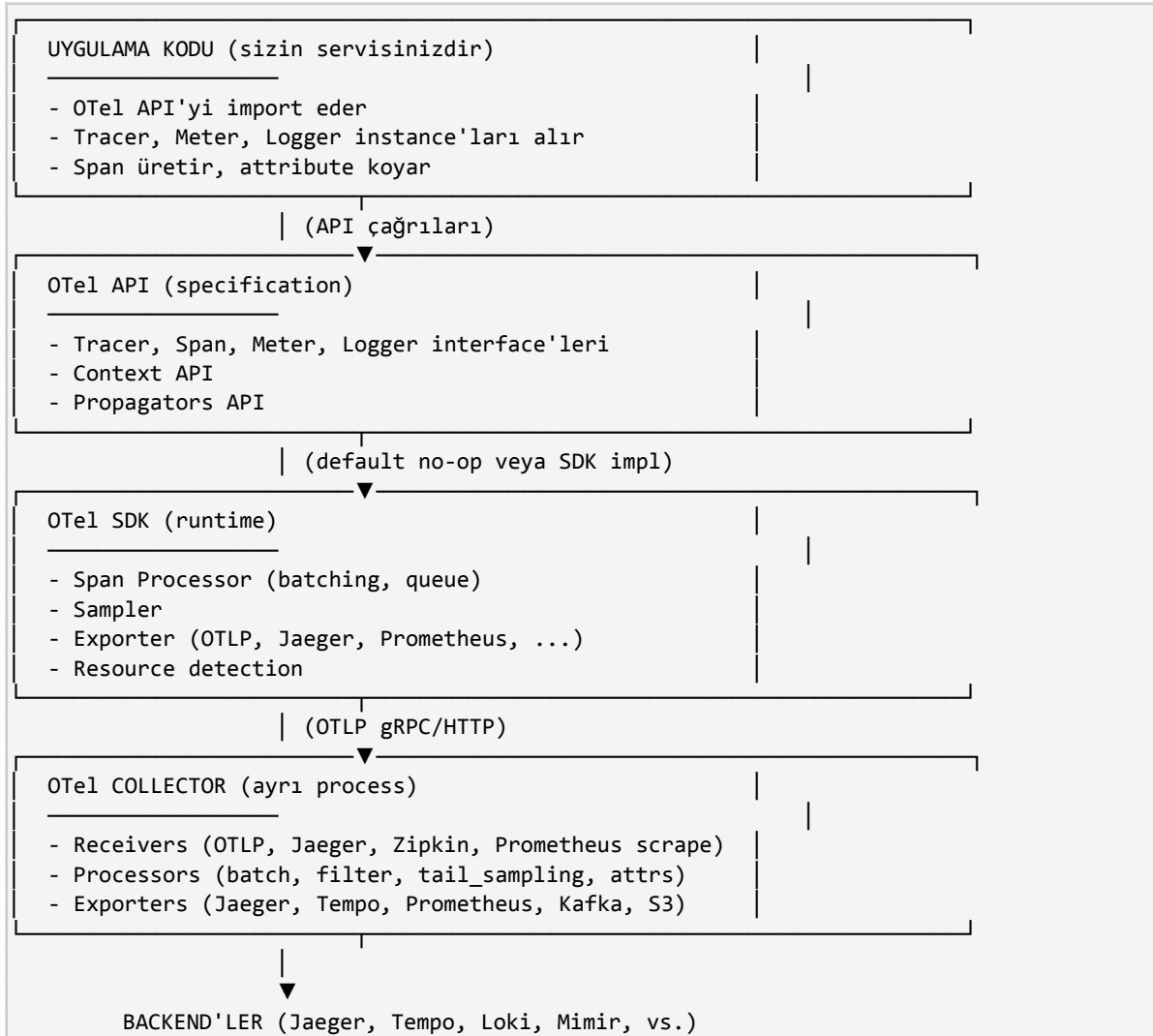
OpenTracing artık deprecated. Yeni proje başlatıyorsanız OpenTelemetry; eski proje ise migrasyon planı.

Bölüm 4 — OpenTelemetry Mimarisi: Üst Düzey Görünüm

OpenTelemetry'yi yalnızca bir kütüphane olarak görmek eksik bir bakıştır. OpenTelemetry, dört ana parçanın bir araya geldiği bir ekosistemdir: API spesifikasyonları, SDK'lar, OTLP protokolü ve Collector. Bu bölüm dört parçanın birbirine nasıl bağlandığını ve sorumluluk sınırlarını açıklar.

4.1 Dört Katmanlı Mimari

OpenTelemetry'nin mimari katmanlarını yukarıdan aşağıya doğru şöyle sıralayabiliriz:



4.2 API ve SDK Ayırımı — Neden Bu Kadar Önemli

OpenTelemetry'nin en kritik tasarım kararı, API ile SDK'yı net olarak ayırmasıdır. Java tarafında bu, iki ayrı Maven artifact'ı olarak görünür:

- `io.opentelemetry:opentelemetry-api` — Uygulama kodu bu artifact'a bağımlıdır. Yalnızca interface'leri içerir; default implementasyon no-op'tur.
- `io.opentelemetry:opentelemetry-sdk` — Runtime'da kurulur, span processor + exporter + sampler içerir. Kütüphane geliştiricileri buna asla bağımlı olmamalı.

Bu ayırım, kütüphane yazarları için kritiktir. Bir HTTP client kütüphanesi geliştiriyorsanız ve içine OTel tracing eklemek istiyorsanız, sadece API'ye bağımlı olmalısınız. Aksi takdirde kütüphanenizi kullanan her uygulama, sizin seçtiğiniz SDK versiyonuna mahkum olur. Aynı yaklaşım Java'nın SLF4J / Logback ayırımıyla bire bir aynıdır: API'yi import edersin, SDK'yı runtime'da değiştirebilirsin.

4.3 Default No-Op Davranışı

OpenTelemetry API, herhangi bir SDK kurulu değilse 'no-op' (hiçbir şey yapmıyor) modunda çalışır. Span üretirsiniz, attribute koyarsınız, ama hiçbir veri dışarı çıkmaz, hiçbir performans cezası ödenmez. Bu, kütüphane yazarları için son derece önemli bir özelliktir; çünkü kütüphaneniz tracing destekli olabilir ama kullanıcı tracing istemiyorsa hiçbir cost yüklenmez.

4.4 Resource Modeli

OpenTelemetry'de 'Resource', telemetri verisinin hangi servisten/host'tan/region'dan geldiğini açıklayan immutable attribute kümesidir.

Resource her span/metric/log'a otomatik olarak yapışır; bu sayede Jaeger'da servis dropdown'unda doğru isimler çıkar, Prometheus'ta target'lar doğru etiketle indekslenir.

```
# Collector tarafındaki resource processor örneği
processors:
  resource:
    attributes:
      - key: service.namespace
        value: argela-fm
        action: insert
      - key: deployment.environment
        value: production
        action: upsert
```

4.5 Tracer Provider, Meter Provider, Logger Provider

Üç sinyal için ayrı ayrı provider abstraction'ları vardır. Bir provider, ilgili sinyali üretecek tracer/meter/logger instance'larının fabrikasıdır. Tipik olarak SDK kurulumunda tek bir global provider yaratırsınız ve bunu Java'da

```
// Java - SDK manuel kurulum (auto-config kullanılmıyorsa)
OpenTelemetry openTelemetry = OpenTelemetrySdk.builder()
    .setTracerProvider(SdkTracerProvider.builder()
        .addSpanProcessor(BatchSpanProcessor.builder(
            OtlpGrpcSpanExporter.builder()
                .setEndpoint("http://otel-collector:4317")
                .build())
            .setMaxQueueSize(2048)
            .setMaxExportBatchSize(512)
            .setScheduleDelay(Duration.ofSeconds(5))
            .build())
        .setResource(Resource.getDefault().merge(
            Resource.create(Attributes.of(
                AttributeKey.stringKey("service.name"), "fault-collector",
                AttributeKey.stringKey("service.version"), "1.2.0"
            )))
        .setSampler(Sampler.parentBased(Sampler.traceIdRatioBased(0.1)))
        .build())
```

```
.setPropagators(ContextPropagators.create(
    W3CTraceContextPropagator.getInstance()))
.buildAndRegisterGlobal();
```

4.6 Collector — Telemetri'nin Veri Düzlemi

Collector, OpenTelemetry'nin altyapı tarafının kalbidir. Uygulamalardan gelen OTLP verisini alır, dönüştürür ve nihai backend'lere iletir. Collector'ı uygulamayla aynı süreçte çalıştırmak teknik olarak mümkün olsa da prod'da neredeyse her zaman ayrı bir process/pod olarak deploy edilir. Bunun nedenleri:

- Uygulama crash olduğunda kuyruğunuzdaki span'lar kaybolmasın.
- Vendor değişimi sadece collector konfigürasyonu değiştirilerek yapılabilir.
- Tail-based sampling, attribute redaction gibi merkezi işlemler tek noktada uygulansın.
- Backend'in yavaşlığı veya kesintisi uygulamayı etkilemesin (collector backpressure absorbe eder).

4.7 Sidecar vs Gateway Collector Topolojisi

Collector iki temel topolojide deploy edilir:

4.7.1 Agent Mode (Sidecar/DaemonSet)

Her host'ta veya her pod'da bir collector instance'ı çalışır. Uygulamalar localhost'a (4317/4318) veri gönderir. Avantaj: minimum ağ gecikmesi, network bölünmelerinde tolerans. Dezavantaj: konfig değişimlerini her node'a uygulamak gerekir; tail sampling gibi global kararlar burada alınmaz.

4.7.2 Gateway Mode

Birden çok collector replica'sı bir LB arkasında 'gateway' olarak çalışır. Tüm uygulamalar bu gateway'e gönderir. Avantaj: tail sampling, cross-trace agregasyon gibi kararlar burada alınır; konfig tek yerde yönetilir. Dezavantaj: tek başına gateway, ekstra network hop demektir; HA için en az 2 replica şart.

4.7.3 İki Katmanlı Mimari (Hybrid)

Endüstri standardı haline gelen yaklaşım: her node'da agent collector + merkezi gateway collector. Agent collector temel batching ve resource enrichment yapar; gateway collector tail sampling, attribute filter, multi-backend fan-out gibi ağır işleri yürütür.

```
# İki katmanlı topoloji
[App] —> [Agent Collector (DaemonSet)] —> [Gateway Collector (Deployment)] —>
[Jaeger / Tempo / Prometheus]
```

4.8 Bölüm 4 — Özet ve Tavsiyeler

PROD TAVSİYESİ

Yeni servis prod'a alırken: API'yi import et, SDK'yı autoconfigure ile yükle, Collector'a OTLP/gRPC ile gönder.

Collector'ı uygulamayla aynı pod'da sidecar olarak başlatmak küçük ekipler için iyi başlangıç; trafik arttıkça gateway katmanı eklemek doğru.

Resource attribute'larını standartlaştırın: ekipler kendi başına yarattıkça service.name çakışmaları yaşanır.

ANTI-PATTERN

Uygulama kodundan doğrudan Jaeger/Zipkin client kullanmak: vendor lock-in, migrasyon imkansız.

Collector'ı atlayıp uygulamadan doğrudan Jaeger backend'ine push etmek: backend kesintisi uygulama kesintisine dönüşür.

Her node'a tek collector ama gateway yok ve tail sampling yapmaya çalışmak: cross-trace görünürlük yoktur, kararlar yanlış alınır.

Bölüm 5 — OpenTelemetry SDK Internals

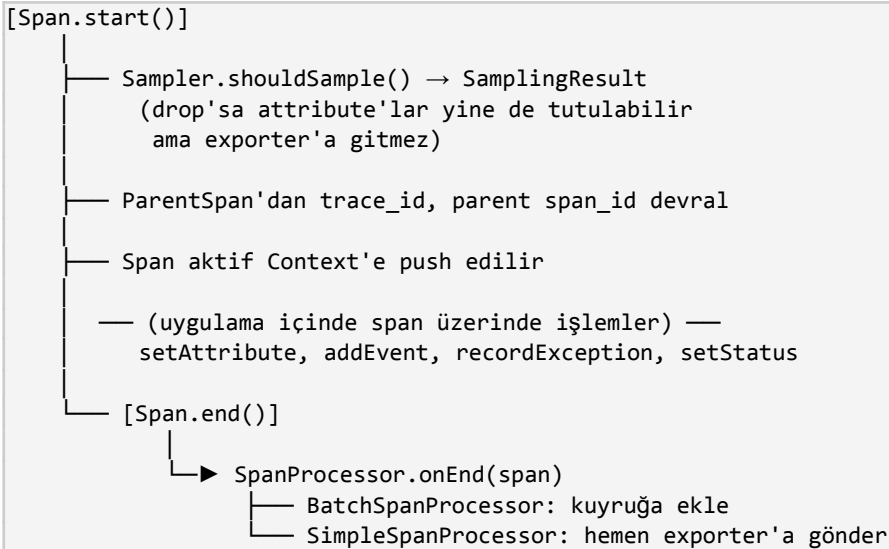
SDK katmanı, OpenTelemetry'nin 'çalışan' tarafıdır. API tarafında 'Span başlat, attribute koy' demek tek satırlık iştir; ama bu satırın arkasında SDK'nın yaptığı işler bütün performans karakteristiğini belirler. Yanlış SDK ayarı yapan bir ekip, prod'a çıktığı gün heap'i şişiren, GC pause yaratan ve bazen tüm trace'leri sessizce kaybeden bir sistemle karşılaşabilir. Bu bölüm SDK'nın iç yapısını ve ayar noktalarını ayrıntılı işler.

5.1 SDK'nın Ana Bileşenleri

Bileşen	Sorumluluk	Tipik Ayar Noktası
TracerProvider	Tracer fabrikası, resource ve processor'lara host'luk yapar	Builder pattern, AutoConfigure
SpanProcessor	Tamamlanan span'ları topluyor (sync veya batch)	BatchSpanProcessor.maxQueueSize
Sampler	Yeni span üretilirken örneklenip örneklenmeyeceğine karar verir	TraceIdRatio, ParentBased
Exporter	Toplu span'ları dış sisteme aktarır	OTLP gRPC/HTTP, Jaeger, Zipkin
Resource	Servisin kim olduğunu söyleyen attribute kümesi	ENV: OTEL_RESOURCE_ATTRIBUTES
ContextPropagator	Servisler arası context geçişini yönetir	W3C TraceContext, Baggage
IdGenerator	Trace/Span ID üretimi	Random (default), W3C uyumlu
Clock	Span timestamp kaynağı	System nanoTime (default)

5.2 Span'in Yaşam Döngüsü

Bir span'ın doğumundan ölümüne kadar geçen aşamalar SDK içinde net bir state machine olarak modellenir. Bu modeli anlamak, ileride göreceğiniz 'span bitmiyor', 'attribute eklenmiyor' gibi sorunları teşhis etmek için kritiktir.



```
└─▶ Exporter.export(batch) → OTLP push
```

Span yaşam döngüsü ile ilgili birkaç sık karşılaşılan yanlış anlama: Span

5.3 Span Processor — Sync vs Batch

5.3.1 SimpleSpanProcessor

Her span end olduğunda anında exporter'a gönderir. Test ortamlarında, unit testlerde, küçük projelerde uygundur. Prod'da kullanmak tehlikelidir çünkü her span için bir IO işlemi tetiklenir; latency-critical bir endpoint'te 50ms'lik gRPC çağrısı request thread'ini bloklar. Ayrıca exporter yavaşladığında doğrudan request hattını yavaşlatır.

5.3.2 BatchSpanProcessor (BSP)

Default prod processor. İç yapısı kabaca şöyle çalışır:

- `maxQueueSize`: Span'ların biriktirildiği bounded queue (default 2048). Queue dolarsa yeni span'lar drop edilir.
- `maxExportBatchSize`: Tek bir export çağrısında gönderilecek max span (default 512).
- `scheduleDelay`: Queue dolu olmasa bile bu süre geçince export tetiklenir (default 5s).
- `exportTimeout`: Tek bir export çağrısının max süresi (default 30s).

BSP, ayrı bir worker thread çalıştırır. Request thread'i

PERFORMANS ETKİSİ

BSP queue capacity'sini hesaplarken: ortalama RPS × tipik span/istek × tahmini 2-3x burst payı.
100 RPS, 5 span/istek, 3x burst = 1500 span/sn → queue 2048 yetmez, 4096-8192 doğru.
Queue overflow'u izleyin: BSP, OTEL SDK metric'i olarak `otel.bsp.dropped_spans` yayar. Bunu Prometheus'tan scrape edin.

5.4 Exporter Çeşitleri

Exporter	Protokol	Tipik Kullanım
OtlpGrpcSpanExporter	OTLP/gRPC (4317)	Prod-default, en performanslı
OtlpHttpSpanExporter	OTLP/HTTP (4318)	Firewall/Proxy gRPC desteklemiyorsa
JaegerThriftSpanExporter (deprecated)	Jaeger Thrift	Eski Jaeger 1.x; yeni Jaeger OTLP destekliyor
ZipkinSpanExporter	Zipkin JSON	Zipkin backend kullanıyorsanız
LoggingSpanExporter	stdout	Debug, prod'da kapalı olmalı
InMemorySpanExporter	Java collection	Sadece testler için

5.5 SDK Konfigürasyonu — Üç Yöntem

5.5.1 Programmatic (Manuel Builder)

Yukarıdaki örnekte gösterildiği gibi,

5.5.2 Auto-Configure (Java agent + env)

OpenTelemetry Java auto-configuration modülü, environment variable'lardan SDK'yı otomatik yapılandırır:

```
OTEL_SERVICE_NAME=fault-collector
OTEL_RESOURCE_ATTRIBUTES=service.version=1.2.0,deployment.environment=prod
OTEL_TRACES_EXPORTER=otlp
OTEL_METRICS_EXPORTER=otlp
OTEL_LOGS_EXPORTER=otlp
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
OTEL_EXPORTER_OTLP_PROTOCOL=grpc
OTEL_TRACES_SAMPLER=parentbased_traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1
OTEL_BSP_MAX_QUEUE_SIZE=4096
OTEL_BSP_MAX_EXPORT_BATCH_SIZE=512
OTEL_BSP_SCHEDULE_DELAY=2000
```

Avantaj: kod değişikliği yok, deploy ortamı bazında ayar. Dezavantaj: env değişkenlerinin kalabalıklaşması; bazı edge case'leri programmatic kadar esnek değildir.

5.5.3 Java Agent (zero-code)

Java agent yaklaşımı sıfır kod değişikliği ile auto-instrumentation sağlar. Detayını Bölüm 10'da işleyeceğiz; SDK perspektifinden bilinmesi gereken nokta: agent kendi içinde OTel SDK'yı içerir ve env değişkenleri ile yapılandırılır.

5.6 Resource Detector'lar

SDK başlangıçta hangi ortamda çalıştığını öğrenmek için 'resource detector' adı verilen modülleri sırayla çalıştırır:

- ProcessResourceDetector — process.pid, process.executable.path
- OsResourceDetector — os.type, os.description
- HostResourceDetector — host.name, host.id
- ContainerResourceDetector — container.id (cgroup'tan)
- ProcessRuntimeResourceDetector — process.runtime.name=Java, version
- EnvironmentResourceDetector — OTEL_RESOURCE_ATTRIBUTES env'ini parse eder
- Kubernetes detector (cloud-specific) — k8s.namespace, k8s.pod.name, k8s.node.name (downward API ile)

Detector çıktıları birleştirilerek tek bir Resource objesi oluşturulur. Çakışan attribute'larda 'son detector kazanır' kuralı vardır; bu yüzden EnvironmentResourceDetector en sonda çalışacak şekilde konfig edilir.

5.7 IdGenerator ve Trace/Span ID Format

Trace ID 16 byte (32 hex char), Span ID 8 byte (16 hex char) olarak tanımlıdır. Default IdGenerator kriptografik olmayan

- AWS X-Ray uyumluluğu: ilk 4 byte timestamp olmalı (AwsXrayIdGenerator)
- Distributed deterministic ID için trace ID'yi mevcut request ID'den türetmek isteyenler

- Test ortamı: deterministic ID generator ile reproducible test

5.8 Bölüm 5 — Tavsiyeler ve Sık Hatalar

ANTI-PATTERN

Prod'da SimpleSpanProcessor kullanmak: tek bir yavaş exporter çağrısı tüm request hattını boğar.

BSP queue size'ı 256 gibi düşük değerlerde bırakmak: high-throughput sistemde span drop oranı %50+ olabilir.

Aynı uygulamada hem manuel builder hem auto-configure aktif: çift initialization, beklenmedik davranış.

PROD TAVSİYESİ

BSP metric'lerini (queue size, dropped count) Prometheus'tan toplayın; alerting kurun.

Resource detector'ların production'da çakışmadığından emin olun; özellikle service.name'in env'den geldiğinden emin olun.

Exporter timeout'unu collector'ın retry policy'sine göre ayarlayın; aşırı uzun timeout backpressure'i maskeler.

Bölüm 6 — Context Propagation Internalleri

Distributed tracing'in en sık kırılan parçası context propagation'dır. 'Trace görünmüyor', 'parent span bozuk', 'iki ayrı trace oluşmuş' şikayetlerinin %80'inin altında yatan sebep context propagation hatasıdır. Bu bölüm Context'in ne olduğu, nasıl taşındığı, nasıl bozulabileceği konularını derinlemesine işler.

6.1 Context Nedir?

OpenTelemetry'de Context, mevcut işlem hakkında 'çağrı bazlı' bilgiyi taşıyan immutable bir key-value map'tir. İçinde başlıca iki bilgi vardır:

- **SpanContext:** Aktif span'ın trace_id, span_id, trace_flags, trace_state bilgisi.
- **Baggage:** Uygulamanın istek bazında taşımak istediği her türlü ek key-value (örn. tenant_id, feature_flag).
- **Özel key'ler:** Kütüphaneler kendi key'lerini de Context'e koyabilir.

Java'da Context, thread-local olarak taşınır (default). Yani 'aktif span' diye bir kavram aslında 'aktif thread'in context'indeki span'dır. Bu, async/reactive programlamada hassas bir hale gelir; ileriki bölümlerde detaylandırılacak.

6.2 W3C Trace Context Standardı

HTTP üzerinden context taşımak için 2020'de W3C 'Trace Context' Recommendation'ını yayımladı. Bu standart iki HTTP başlığı tanımlar:

6.2.1 traceparent başlığı

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

(01=sampled)

trace id (16 byte)

version (00)

parent span id (8 byte)

flags

Bu başlık her downstream HTTP isteğine eklenir. Alıcı servis,

6.2.2 tracestate başlığı

```
tracestate: rojo=00f067aa0ba902b7,congo=t61rcWkgMzE
```

Vendor-specific bilgilerin taşındığı opsiyonel bir başlıktır. Multi-vendor sistemlerde her vendor kendi key'ini ekleyebilir. Çoğu uygulama tracestate'i hiç dokunmaz.

6.3 B3 Propagation (Zipkin Mirası)

W3C'den önce Zipkin'in B3 propagation formatı yaygındı. Hâlâ bazı legacy sistemlerde görülür. B3 birkaç farklı varyantta gelir:

```
# Multi-header B3
X-B3-TraceId: 4bf92f3577b34da6a3ce929d0e0e4736
X-B3-SpanId: 00f067aa0ba902b7
```

```
X-B3-ParentSpanId: e457b5a2e4d86bd1
X-B3-Sampled: 1

# Single-header B3 (compact)
b3: 4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-1-e457b5a2e4d86bd1
```

OpenTelemetry SDK her iki propagation formatını destekler. Çoklu format desteği için

6.4 Baggage — Cross-Cutting Bağlam

Baggage, span attribute'undan farklı olarak istek boyunca tüm downstream servisler taşıyan key-value'lardır. Örnek kullanım: 'tenant_id'yi gateway'de set et, tüm downstream'lerde her metric'e tenant_id label'ı ekle'.

```
# baggage başlığı
baggage: userId=alice,serverNode=DF%2028,isProduction=false
```

Baggage'ın asıl tehlikesi büyüklük: her header her HTTP isteğine eklendiği için 8KB+ baggage 'a sahip bir istek, HTTP server limit'lerine takılabilir, gateway'lerde drop edilebilir.

6.5 Async Bağlamda Context Propagation

Context, thread-local olduğu için thread sınırını aşan işlemlerde otomatik taşınmaz. Bu, modern Java'da büyük bir sorun:

6.5.1 Standart ExecutorService

```
// YANLIŞ: context kaybolur
ExecutorService es = Executors.newFixedThreadPool(8);
es.submit(() -> {
    Span span = tracer.spanBuilder("worker").startSpan();
    // Bu span'ın parent'ı yok — context kaybedildi
    span.end();
});

// DOĞRU: Context.taskWrapping ile sarmala
ExecutorService traced = Context.taskWrapping(es);
traced.submit(() -> { /* context aktif */ });

// Alternatif: agent zaten executor'ı instrument ediyor
// (otel.instrumentation.executors.enabled=true default)
```

6.5.2 CompletableFuture

CompletableFuture chain'leri varsayılan ForkJoinPool.commonPool'unda çalışır. Agent kullanılıyorsa instrumented; manuel yazıyorsanız

6.5.3 Spring @Async

Spring'in

```
@Configuration
public class AsyncConfig {
    @Bean
    public TaskExecutor taskExecutor() {
        ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
        exec.setCorePoolSize(8);
        exec.setTaskDecorator(new ContextPropagatingTaskDecorator());
    }
}
```

```

    exec.initialize();
    return exec;
  }
}

```

6.5.4 Reactor (WebFlux)

Reactor 3.5+

6.5.5 Virtual Threads (Java 21+)

Java 21'de gelen virtual thread'ler context propagation açısından bir özel durum yaratır: ScopedValue kullanmadığınız sürece, virtual thread'ler ThreadLocal'ı kopyalamaz. OpenTelemetry Java agent 1.32+ virtual thread instrumentation desteği ekledi; ama kendi yazdığınız

```

// Virtual thread + Context
Context current = Context.current();
Thread.startVirtualThread(() -> {
    try (Scope s = current.makeCurrent()) {
        Span span = tracer.spanBuilder("vt-task").startSpan();
        try { /* iş */ } finally { span.end(); }
    }
});

```

6.6 Mesajlaşma Sistemlerinde Propagation (Kafka, RabbitMQ, JMS)

HTTP context propagation görece kolaydır çünkü her HTTP isteğine başlık eklenebilir. Mesajlaşma sistemlerinde ise context, mesaj header'ları (Kafka headers, AMQP properties, JMS properties) üzerinden taşınır.

Sistem	Taşıyıcı	Tipik Sorun
Kafka	ProducerRecord.headers()	Header eklemeyen producer'lar context kaybeder
RabbitMQ	BasicProperties.headers	Native exchange'lerde header korunmaz
ActiveMQ/JMS	JMS Message Properties	Bazı broker'lar property limit'i koyar
AWS SQS	MessageAttributes	Max 10 attribute limiti, baggage'a dikkat
Google Pub/Sub	PubsubMessage.attributes	Attribute size limit 1024 byte

Producer tarafında: gönderdiğiniz mesaja

6.7 Context'in Beslenmeyeceği Yerler — Yaygın Kayıp Noktaları

DEBUGGING

Yeni Thread.start() ile başlatılan thread — agent yoksa context geçmez.

JNI çağrıları — native koda geçince context biter (eBPF ile kapatılabilir).

Reflection ile çağrılan async metodlar — bazı agent versiyonlarında yakalanmaz.

Custom thread pool decorator olmayan executor'lar.

Quartz Scheduler, CronTrigger — manuel context capture gerekir.

Webhook callback'leri — outbound HTTP yapılırken trace context taşınır ama callback inbound olarak gelirse yeni trace başlar.

Kafka partition rebalance sırasında tüketim — context drop olabilir.

6.8 Bölüm 6 — Sık Yapılan Hatalar ve Çözümleri

ANTI-PATTERN

OTel SDK'sı kuruldu ama propagator set edilmedi: span üretilir ama traceparent header gönderilmez, downstream yeni trace başlatır.

B3 ve W3C aynı anda set edilmedi: legacy servisler B3 gönderiyor, yeni servis sadece W3C dinliyor; trace zinciri kopuyor.

Baggage'a PII koymak: GDPR ihlali, log aggregation'da kişisel veri sızar.

Thread pool decorator'ı unutmak: 'Async metodda neden parent span yok?' şikayetlerinin %90'ı bu.

Bölüm 7 — Span Anatomisi: Attribute, Event, Link, Status

Span yalnızca 'isim + başlangıç/bitiş zamanı' değildir. Modern bir span; attribute'lar, event'ler, link'ler, status ve exception ile zenginleştirilebilir. Bu bölüm bu altı öğeyi tek tek inceler ve hangi durumlarda hangisini kullanmanız gerektiğini netleştirir.

7.1 Span Kind

Bir span'ın 'kind'i, ne tür bir iş birimini temsil ettiğini söyler. Beş kind vardır:

Kind	Anlam	Tipik Örnek
INTERNAL	Servis içi iş	İş kuralı, hesaplama
SERVER	Inbound RPC handle eden span	HTTP endpoint handler
CLIENT	Outbound RPC yapan span	WebClient çağırısı, JDBC sorgusu
PRODUCER	Mesaj üreten span	Kafka producer send
CONSUMER	Mesaj tüketen span	Kafka consumer poll

Span kind, sampling kararları ve UI gösteriminde önemlidir. Jaeger ve Tempo, SERVER ve CLIENT kind'lerini farklı ikonlarla gösterir; sampling decision'ı SERVER span'da alınır.

7.2 Span Attribute'ları

Attribute'lar, span'ın metadata'sıdır. Key-value pair'ler olarak eklenir. Best practice: attribute isimlerini semantic conventions'a uygun seçin.

```
Span span = tracer.spanBuilder("process alarm")
    .setSpanKind(SpanKind.INTERNAL)
    .setAttribute("alarm.id", alarmId)
    .setAttribute("alarm.type", "LINK_DOWN")
    .setAttribute("alarm.source_ip", sourceIp)
    .setAttribute("alarm.severity", "HIGH")
    .setAttribute(SemanticAttributes.NET_PEER_NAME, "fault-processor")
    .startSpan();
```

7.2.1 Attribute Limitleri

Sınırsız attribute eklenemez. SDK default limitleri:

- **Max attribute count per span:** 128 (default)
- **Max attribute value length:** sınırsız (default), ama `OTEL_ATTRIBUTE_VALUE_LENGTH_LIMIT` ile sınırlandırılabilir
- **Attribute key uzunluk:** sınırsız; ama pratik olarak <100 char

7.2.2 High Cardinality Tuzakları

Trace attribute'larında cardinality, metric'tekiyle aynı problemi yaratmaz (her span unique zaten). Ancak attribute'ları SONRADAN metric'e çevirirseniz (örn.

UYARI

user_id, session_id, request_id gibi unique değerleri ASLA metric label'a çevirmeyin.
 URL'leri pattern'e indirgeyin: /users/12345 → /users/:id
 Span attribute olarak tutmak OK, metric'e çevirmek değil.

7.3 Span Events

Event, span içinde belirli bir anda olan log-benzeri olayı temsil eder. Bir event'in timestamp'i ve attribute'ları vardır. 'Cache'ten okudum', 'Validation başladı', 'Retry yapıldı' gibi durumlar için idealdir.

```
Span span = ...;
span.addEvent("cache.hit",
    Attributes.of(
        AttributeKey.stringKey("cache.key"), "user:42",
        AttributeKey.longKey("cache.ttl_remaining_ms"), 4500L
    ));
```

Event vs Attribute farkı: attribute span boyunca geçerli olan bir özelliktir; event ise belirli bir zamanda olan bir olaydır. 'Retry sayısı = 3' attribute olur, 'Retry tetiklendi' event olur.

7.4 Exception Recording

OpenTelemetry'de exception'lar span event'i olarak modellenir.

```
Span span = tracer.spanBuilder("process").startSpan();
try {
    doWork();
} catch (Exception e) {
    span.setStatus(StatusCode.ERROR, e.getMessage());
    span.recordException(e);
    throw e;
} finally {
    span.end();
}
```

7.5 Span Status

StatusCode üç değer alır:

7.6 Span Links

Link, bir span'ı kendi parent'ı dışında bir başka span'a bağlamayı sağlar. Asıl kullanım: bir span'ın causal olarak birden fazla 'önceki' span'ı varsa. En tipik örnek: batch processing. Tek bir consumer span'ı, batch'teki her mesajın producer span'larına link'lenir.

```
// Kafka consumer örneği
List<SpanContext> producerContexts = ...; // her mesajın trace contexti
SpanBuilder builder = tracer.spanBuilder("process batch")
    .setSpanKind(SpanKind.CONSUMER);
producerContexts.forEach(builder::addLink);
Span consumerSpan = builder.startSpan();
```

7.7 Span Hiyerarşisi ve Nesting

Span'lar genellikle hiyerarşik (parent-child) ilişki kurar. Bir HTTP endpoint span'ı altında DB span'ı, onun altında redis span'ı gibi. Doğru hiyerarşi için 'aktif span' kavramı kritiktir:

```
try (Scope outer = tracer.spanBuilder("parent").startSpan().makeCurrent()) {
    Span parent = Span.current(); // aktif span

    try (Scope inner = tracer.spanBuilder("child").startSpan().makeCurrent()) {
        // bu span'ın parent'ı 'parent' span (otomatik)
    }
}
```

Manuel olarak

7.8 Bölüm 7 — Production Tavsiyeleri

PROD TAVSİYESİ

Attribute key isimlerini ekip içinde standartlaştırın; tercihen OTEL semantic conventions'tan başlayın.

PII attribute'larını collector tarafında redact edin (attribute processor).

Exception'ı hem record exception hem setStatus(ERROR) yapın; sadece biri yetmez.

Span name'leri 'low cardinality' olsun: 'GET /users/123' yerine 'GET /users/:id' (semantic conv. http.route).

Bölüm 8 — Semantic Conventions ve Resource Attributes

Semantic conventions, OpenTelemetry'nin görünmez kahramanıdır. 'HTTP method'a attribute koyarken anahtarı ne yapacaksın?' sorusuna evrensel bir cevap verir. Bu cevap olmasa, her ekip kendi anahtarını seçer ('method', 'http_method', 'request.method') ve dashboard'lar, alerting'ler, sorgular karmaşılaşır. Bu bölüm semantic conventions'ın yapısını ve gündelik kullanımını işler.

8.1 Semantic Conventions Nedir?

Semantic conventions, OpenTelemetry spec'inin parçası olan

Conventions, conceptual area'lara göre gruplanır:

- HTTP — sunucu/istemci HTTP attribute'ları
- Database — SQL, NoSQL, cache sorgu attribute'ları
- Messaging — Kafka, RabbitMQ, vs.
- RPC — gRPC, JSON-RPC
- FaaS — Lambda, Cloud Functions
- Cloud — AWS, GCP, Azure platform metadata
- System — host, container, K8s
- Network — net.peer.ip, net.transport

8.2 En Sık Kullanılan HTTP Conventions

Attribute	Tip	Örnek	Notlar
http.request.method	string	GET, POST	Stable (1.27)
http.response.status_code	int	200, 404	Stable
http.route	string	/users/:id	Low-card route template
url.path	string	/users/12345	Yüksek cardinality
url.scheme	string	https	Stable
url.full	string	https://api.../path	Sensitive; PII riski
server.address	string	api.example.com	Stable
server.port	int	443	Stable
user_agent.original	string	curl/7.85	Stable

UYARI

Eski Java agent versiyonları (2.x öncesi) http.method, http.status_code gibi 'eski' attribute isimlerini kullanıyordu.

OTel SemConv 1.21+ sürümü ile isimler 'http.request.method' formatına geçti.

Dashboard/alert'leri yazarken her iki ismi de eşleştirmeyi (alias) düşünün veya tek versiyona kilitleyin.

8.3 Database Conventions

Attribute	Örnek
db.system	postgresql, mysql, redis, mongodb
db.name	argela_fm (DB adı)
db.statement	SELECT * FROM alarms WHERE ...
db.operation	SELECT, INSERT, UPDATE
db.sql.table	alarms
network.peer.address	db-host.local
network.peer.port	5432

Java agent, JDBC çağrılarında bu attribute'ları otomatik doldurur. Manuel olarak yazmanız nadiren gerekir. Önemli not:

8.4 Messaging Conventions

Attribute	Anlam
messaging.system	kafka, rabbitmq, jms
messaging.destination.name	alarms.events (topic/queue)
messaging.operation	publish, receive, process
messaging.message.id	Mesaj ID (broker'ın ürettiği)
messaging.kafka.partition	Partition no
messaging.kafka.consumer.group	Consumer group adı

8.5 Resource Attributes

Resource attribute, telemetri sinyalinin kaynağını tanımlar. Span attribute istek-bazlı bilgi taşırken, resource attribute servis-bazlı bilgi taşır.

Attribute	Tipik Değer
service.name	fault-collector
service.version	1.2.0
service.namespace	argela-fm
service.instance.id	fault-collector-pod-7af3
deployment.environment	production
host.name	node-01.k8s.local
host.arch	amd64
os.type	linux
k8s.namespace.name	argela-fm-prod
k8s.pod.name	fault-collector-7d6cf-x9z2
k8s.node.name	node-01
k8s.deployment.name	fault-collector

Attribute	Tipik Değer
telemetry.sdk.name	opentelemetry
telemetry.sdk.language	java
telemetry.sdk.version	1.34.1

Resource attribute'ları çoğunlukla SDK initialize edildiğinde tek seferlik set edilir; runtime'da değişmez. Bu yüzden bunları span/metric/log'a yapıştırmak ekstra cost yaratmaz; SDK her batch'e tek kopya ekler.

8.6 Custom Attribute Naming Stratejisi

Semantic conventions ihtiyacınızı karşılamadığında custom attribute eklemeniz olağandır. Önerilen format:

```
<şirket>.<domain>.<concept>
örn:
  argela.alarm.type
  argela.alarm.severity
  argela.tenant.id
  argela.feature.flag.canary_enabled
```

YANLIŞ:

```
type          ← çok genel
AlarmType     ← camelCase OTel için yanlış
argela_alarm_type ← snake_case OTel'de değil
```

8.7 Bölüm 8 — Tavsiyeler

PROD TAVSİYESİ

Yeni attribute koymadan önce semantic-conventions repo'sunda ara; %80 ihtimalle hazır var.
Custom attribute prefix'inizi belirleyin (org adı veya domain). Tüm ekipler aynı prefix'i kullansın.
Versiyon kararı: stable attribute'lar (1.27+) ile başlayın; experimental olanlara prod'da bağımlı olmayın.
Resource attribute'larını CI/CD'den otomatik enjekte edin (service.version, service.instance.id).

ANTI-PATTERN

Her ekibin kendi naming convention'ını icat etmesi: dashboard'lar fragmanlanır, korelasyon kırılır.
PII'yi attribute'a koymak: trace storage'da, log'larda, backend'lerde sızabilir.
Aynı şeyi hem resource hem span attribute olarak koymak: bandwidth/storage israfı.

Bölüm 9 — Instrumentation: Auto, Manual, Library, Zero-Code

Instrumentation, telemetri verisini kodunuza nasıl 'gömdüğünüz' meselesidir. Aynı problemi çözmek için OpenTelemetry üç-dört farklı yol sunar. Her yolun yeri, avantajı, gizli maliyetleri vardır. Bu bölüm hangi senaryoda hangi yaklaşımın doğru olduğunu mühendislik tradeoff'ları ile anlatır.

9.1 Dört Yaklaşım

Yaklaşım	Kod değişikliği?	Esneklik	Tipik kullanım
Zero-Code (Java agent)	Yok	Düşük (config ile)	Mevcut servise hızla tracing eklemek
Library instrumentation	Bağımlılık eklemek	Orta	Spring, Hibernate, etc. için resmi pkg
Manual instrumentation	Kod yazma	Yüksek	İş kuralı, custom span, attribute
Native instrumentation	Kütüphane yazarları	En yüksek	Kütüphane bizzat span üretir

9.2 Zero-Code: OpenTelemetry Java Agent

Java agent, JVM'in

```
# Java uygulamasını agent ile başlatma
java -javaagent:./opentelemetry-javaagent.jar \
  -Dotel.service.name=fault-collector \
  -Dotel.exporter.otlp.endpoint=http://otel-collector:4317 \
  -Dotel.traces.sampler=parentbased_traceidratio \
  -Dotel.traces.sampler.arg=0.1 \
  -jar fault-collector.jar
```

Agent'in yaptığı: classloader hook'larıyla belirli sınıfların yüklenmesini izler. 'spring-web' sınıflarını gördüğünde,

9.2.1 Agent'in Desteklediği Kütüphaneler

OpenTelemetry Java instrumentation reposu, 100'den fazla kütüphane için support sunar. Argela projenizde otomatik olarak çalışanlar:

- Tomcat / Netty / Undertow (HTTP server)
- Spring WebMVC, WebFlux (controller seviyesi span)
- Spring WebClient (outbound HTTP, traceparent eklenir)
- Apache HttpClient, OkHttp, JDK HttpClient
- JDBC (her statement için span)
- Hibernate / JPA
- Spring Data Repository
- Kafka, RabbitMQ, JMS (producer/consumer)
- Redis Lettuce/Jedis, MongoDB, Elasticsearch

- Logback / Log4j (MDC injection için)
- gRPC (server/client)
- Micrometer (metric bridge)
- Reactor, RxJava (context propagation)
- AWS SDK v1/v2 (S3, DynamoDB, SQS, ...)

9.2.2 Agent vs Manuel — Birlikte Kullanım

Yaygın yanılgı: 'agent kullanıyorsam manuel span yazamam.' Aksine, ikisi mükemmel uyumlu çalışır. Agent altyapı katmanını otomatik instrument eder; iş kuralı seviyesinde custom span'ları siz manuel yazarsınız.

```
// Agent zaten yüklü; sen sadece tracer'ı al
@Service
public class AlarmService {
    private final Tracer tracer = GlobalOpenTelemetry.getTracer("argela.collector");

    public void validate(AlarmRequest req) {
        Span span = tracer.spanBuilder("alarm.validate")
            .setAttribute("alarm.id", req.getAlarmId())
            .startSpan();
        try (Scope s = span.makeCurrent()) {
            // iş kuralı
        } finally {
            span.end();
        }
    }
}
```

9.2.3 Agent Yapılandırma Bayrakları

Agent davranışı 200+

Property	Default	Anlam
otel.javaagent.enabled	true	Agent'ı tamamen devre dışı bırak
otel.instrumentation.<name>.enabled	değişken	Belirli kütüphaneyi off
otel.instrumentation.common.default-enabled	true	Tüm instrumentation'ları on/off
otel.javaagent.debug	false	Debug log
otel.javaagent.extensions	-	Custom extension jar yolu
otel.bsp.max.queue.size	2048	BSP queue
otel.exporter.otlp.endpoint	http://localhost:4317	OTLP endpoint

9.3 Library Instrumentation Paketleri

Agent kullanmayı tercih etmeyenler için, OpenTelemetry her popüler kütüphane için ayrı bir 'instrumentation library' paketi de sunar. Bunlar Maven coordinate olarak

Spring Boot örneği — agent kullanmadan, sadece starter ile:


```
<dependency>
  <groupId>io.opentelemetry.instrumentation</groupId>
  <artifactId>opentelemetry-spring-boot-starter</artifactId>
  <version>2.10.0</version>
</dependency>
```

Bu yaklaşımın avantajı: agent'a kıyasla daha az

9.4 Manuel Instrumentation: Custom Span'lar

İş mantığını tracing'e dahil etmek için manuel span yazmak şart. Argela projenizde 'severity hesaplama', 'alarm doğrulama' gibi adımlar tipik manuel span adaylarıdır.

9.4.1 Temel Pattern

```
public SeverityLevel calculate(AlarmRequest req) {
    Span span = tracer.spanBuilder("severity.calculate")
        .setSpanKind(SpanKind.INTERNAL)
        .setAttribute("alarm.type", req.getAlarmType().name())
        .startSpan();
    try (Scope scope = span.makeCurrent()) {
        SeverityLevel result = strategy.evaluate(req);
        span.setAttribute("alarm.severity", result.name());
        return result;
    } catch (Exception e) {
        span.setStatus(StatusCode.ERROR, e.getMessage());
        span.recordException(e);
        throw e;
    } finally {
        span.end();
    }
}
```

9.4.2 @WithSpan Annotation

Tekrarlı boilerplate'i azaltmak için

```
@WithSpan("severity.calculate")
public SeverityLevel calculate(
    @SpanAttribute("alarm.type") String alarmType,
    @SpanAttribute("alarm.source_ip") String sourceIp,
    AlarmRequest req) {
    // ... method body
}
```

UYARI

@WithSpan AOP/proxy ile çalışır. Aynı sınıf içinden çağrılan metotlarda (this.method) işe yaramaz.

Spring @Transactional gibi davranır: external çağrı şart.

Hot path metotlara koymak performans cezası: AOP overhead + span yaratma maliyeti.

9.5 Native Instrumentation

Bazı kütüphaneler artık OpenTelemetry API'yi doğrudan kendileri kullanıyor. Örneğin Quarkus, Micronaut, Helidon framework'leri 'native' OpenTelemetry desteğiyle gelir. Aynı şekilde Apache Kafka 3.7+ client, AWS SDK v2 yeni sürümleri OTEL integration'ını içinde taşır. Avantaj: agent yok, ekstra dependency yok, en az overhead.

9.6 Bölüm 9 — Tavsiyeler

PROD TAVSİYESİ

Mevcut Spring Boot servisinde başlangıç: Java agent. Tek bir env değişkeniyle 80% kapsanır.

İş kuralı için manuel span ekle: agent'ın görmediği yerleri sen ekle.

@WithSpan'ı sade tut: framework'ün zaten span açtığı yerlere koyma (controller, repository).

Production rollout'ta önce sampling 1% ile başla, exporter sağlığını ölç, sonra arttır.

ANTI-PATTERN

Her method'a @WithSpan koymak: 50ms'lik request 200 span üretir, storage patlar, UI okunmaz.

Agent + library starter aynı anda: çift instrumentation, span duplicate.

Agent ile manuel SDK initialization birlikte: ağır çakışma.

Bölüm 10 — Java Agent Internalleri ve Bytecode Manipülasyonu

Java agent, OpenTelemetry'nin 'sihir' gibi çalışan parçasıdır. Çoğu mühendis için kara kutudur. Ancak agent'ın nasıl çalıştığını anlamak, prod'da yaşadığınız 'şu kütüphane neden instrument edilmedi', 'agent classloader exception attı', 'startup süresi 2x oldu' gibi sorunları teşhis için kritiktir.

10.1 Java Agent Mekanizması

Java Agent, JVM Tool Interface (JVMTI) yerine

```
Manifest-Version: 1.0
Premain-Class: io.opentelemetry.javaagent.OpenTelemetryAgent
Agent-Class: io.opentelemetry.javaagent.OpenTelemetryAgent
Can-Redefine-Classes: true
Can-Retransform-Classes: true
Boot-Class-Path: opentelemetry-javaagent.jar
```

JVM başlarken,

10.2 OpenTelemetry Agent'ın Mimarisi

OpenTelemetry Java agent içinde üç ana parça vardır:

- 10.
- 11.
- 12.

10.3 Bytecode Manipulation — Conceptual Örnek

Diyelim

```
// Orijinal kod
public class DispatcherServlet {
    protected void doDispatch(HttpServletRequest req, HttpServletResponse resp) {
        // ... handler logic
    }
}

// Agent'ın çalışmadan sonraki "soyut" hali
public class DispatcherServlet {
    protected void doDispatch(HttpServletRequest req, HttpServletResponse resp) {
        Span span = SpringWebMvcAdvice.onEnter(req); // bytecode prepend
        Throwable error = null;
        try {
            // ... orijinal handler logic
        } catch (Throwable t) {
            error = t;
            throw t;
        } finally {
            SpringWebMvcAdvice.onExit(span, resp, error); // bytecode append
        }
    }
}
```

Gerçekte byte kodu yeniden yazılır; Java kaynak dosyaya dokunulmaz. Bu yüzden 'kod değişikliği yok' demek doğrudur.

10.4 Classloader Sorunları

Modern Java uygulamalarında birden çok classloader vardır: Bootstrap, Platform (ext), System, Application. Spring Boot fat jar'ları kendi LaunchedURLClassLoader'larını kullanır. Agent'ın instrument ettiği sınıflar, instrument eden agent kodundan farklı bir classloader'da olabilir.

OpenTelemetry agent bu sorunu 'bootstrap classloader' stratejisi ile çözer: gerekli OTel API jar'larını JVM'in bootstrap classloader'ına ekler. Bu sayede her seviyedeki kullanıcı kodu OTel API'sini görebilir. Ancak nadiren OSGi, custom classloader, JBoss modules gibi sistemlerde bu strateji yetmez ve özel konfigürasyon gerekir.

DEBUGGING

Agent debug log: `-Dotel.javaagent.debug=true`

Hangi sınıflar instrument edildi: `-Dotel.javaagent.extensions=... + custom listener`

Classloader çakışması: `NoClassDefFoundError(io.opentelemetry...)` görüyorsanız bootstrap inject sorunu var.

Module path kullanan Java 9+ uygulamalarda `--add-opens` parametreleri gerekebilir.

10.5 Startup Süresi Etkisi

Java agent JVM startup'ında ek iş yapar: `ClassFileTransformer` kaydı, instrumentation modüllerinin yüklenmesi, byte kodu manipulation'ı her sınıf yüklendiğinde. Tipik etkiler:

- Spring Boot app: +500ms ile +3sn arası startup gecikmesi
- İlk istek (warmup): +20-50ms (sınıflar yeni yüklenirken instrument ediliyor)
- Tekrar eden istekler: <1ms ek latency

Bu cost, agent versiyonu, JVM versiyonu, uygulamanın kullandığı kütüphane sayısı ile değişir. Cloud-native ortamlarda startup süresi önemliyse (Kubernetes `CrashLoopBackOff`, autoscaling), CDS/AppCDS veya Quarkus gibi native-image friendly framework'leri değerlendirin.

10.6 Agent Extension'ları

Agent'ın davranışını kod yazarak değiştirmek istiyorsanız, 'extension' jar'ı yazabilirsiniz. Bunlar şunları yapabilir:

- Yeni instrumentation modülü ekle (örn. kendi RPC kütüphaneniz için)
- Custom Sampler implementasyonu plug et
- Custom Exporter ekle
- Custom Propagator ekle
- Resource detector ekle

```
# Extension'ı agent'a ekle
java -javaagent:opentelemetry-javaagent.jar \
  -Dotel.javaagent.extensions=/opt/argela-otel-ext.jar \
  -jar app.jar
```

10.7 Bölüm 10 — Tavsiyeler

PROD TAVSİYESİ

Agent versiyonunu her servis için aynı tutun; farklı versiyonlar farklı semantic convention attribute isimleri çıkarır.

Agent dosyasını CDN/registry'den her startup'ta indirmeyin; container image'a gömün.

Startup süresi kritikse: gerekmeyen instrumentation'ları devre dışı bırakın (otel.instrumentation.X.enabled=false).

Agent log'larını ELK'ya pipe etmeyin; gereksiz log noise.

Bölüm 11 — Spring Boot Entegrasyonu ve Micrometer İlişkisi

Java ekosisteminde Spring Boot baskın framework'tür. OpenTelemetry'nin Spring Boot ile nasıl konuştuğu — özellikle Micrometer ile ilişkisi — sıklıkla kafa karıştırır. Bu bölüm üç farklı entegrasyon yolunu netleştirir.

11.1 Üç Entegrasyon Yolu

Yol	Aktör	Pros	Cons
Java agent (zero-code)	Sadece JVM flag	En geniş kapsam, kod yok	Versiyon takibi, debug zor
OTel Spring Boot Starter	Maven dep	Spring tarzı config, autoconfig	Bazı düşük seviye instr. yok
Micrometer + Tracing	Spring Boot 3.x default	Native Spring Boot deneyimi	OTel-specific feature'lar zayıf

11.2 Spring Boot 3.x ve Micrometer Observation API

Spring Boot 3.0+ ile Micrometer 1.10, 'Observation API'sini tanıttı. Bu, Spring uygulamalarında metric ve trace'in tek bir API üzerinden üretildiği bir abstraction'dır. Default olarak Micrometer Tracing kütüphanesi gelir; backend olarak Brave (Zipkin) veya OpenTelemetry seçilebilir.

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

Bu konfigürasyonla Spring, gelen HTTP isteklerini ve giden WebClient çağrılarını Observation API üzerinden trace'e dönüştürür. Backend OpenTelemetry olduğu için Micrometer Observation API çağrıları altta OTEL Span'ına çevrilir.

11.3 Hangisini Seçmeli?

Karar şu sorulara cevap verir:

- **Mevcut sistemde Spring Boot 2.x mı?** → Micrometer Tracing yok. Java agent en doğru.
- **Spring Boot 3.x + sıfırdan başlıyorsan?** → Micrometer + OTEL bridge ekosistem ile uyumlu.
- **Polyglot ortam (Python, Go, Java karışık)?** → Java agent + manuel span; uyumluluk daha güvenli.

- **Kütüphane karmaşıklığı (Kafka, JMS, MongoDB, ...) yüksek?** → Java agent; daha geniş kapsam.

11.4 Argela Projeniz Örneği — Hybrid Yaklaşım

Sizin projenizde CLAUDE.md'ye göre Java agent kuruluyor. Bu Spring Boot 3 ile çakışır mı? Hayır —

```
# application.yml – Micrometer Tracing tarafını kapatmak için
management:
  tracing:
    enabled: false
  metrics:
    enable:
      all: true
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
```

11.5 Micrometer Metric → OpenTelemetry

Spring Actuator ile gelen JVM metric'leri (heap, GC, thread, connection pool, HTTP server) default Prometheus formatında

11.5.1 Pull Model: Prometheus Receiver

Collector,

```
# otel-collector-config.yaml
receivers:
  prometheus:
    config:
      scrape_configs:
        - job_name: 'fault-collector'
          scrape_interval: 15s
          static_configs:
            - targets: ['fault-collector:8080']
          metrics_path: /actuator/prometheus
```

11.5.2 Push Model: Micrometer OTel Registry

Micrometer'ın OTel registry'si, metric'leri doğrudan OTLP/gRPC ile collector'a push eder. Daha az latency, ama load balancer arkasında ek konfig gerekebilir.

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-otlp</artifactId>
</dependency>

management:
  otlp:
    metrics:
      export:
        url: http://otel-collector:4318/v1/metrics
        step: 15s
```

11.6 WebClient ve Reactive Tracing

Spring WebClient, agent veya starter ile otomatik instrument edilir. WebClient çağrısı yapıldığında automatic olarak:

- CLIENT kind span açılır
- traceparent header request'e eklenir
- response status code attribute olarak set edilir
- hata varsa span ERROR status alır

Önemli: WebClient reactive olduğu için context propagation Reactor üzerinden gerçekleşir. Agent reactor instrumentation'ı aktif değilse trace zinciri WebClient'tan sonra kopar.

11.7 Spring Data JPA / Hibernate

Agent, Hibernate session açılışını, query execution'ı ve JDBC çağrılarını ayrı span'lara çevirir. Tipik bir Spring Data

```
Span: POST /api/v1/process (SERVER)
└─ Span: AlarmProcessorService.process (INTERNAL, manuel)
    └─ Span: SessionImpl.flush (INTERNAL, Hibernate)
        └─ Span: alarms INSERT (CLIENT, JDBC)
```

Bu otomatik zincir, agent'ın en güçlü yanıdır. Manual instrumentation ile aynı detayı yakalamak yüzlerce satır kod yazmak demektir.

11.8 Bölüm 11 — Production Tavsiyeleri

PROD TAVSİYESİ

Spring Boot 3.x + Java agent: önce management.tracing.enabled=false yap; çift trace'i engelle.
JDBC instrumentation prod'da db.statement içeriyor; PII içeren tablolarda statement sanitizier şart.
WebClient çağrılarında her zaman timeout set edin; tracing exporter beklenirken request thread'i bloklamasın.
Spring Actuator /actuator/health'a tracing kapatın
(otel.instrumentation.spring-webmvc.experimental.span-suppression=true).

Bölüm 12 — Async, Reactive ve Virtual Thread'lerde Tracing

Async programlama modeli (CompletableFuture, @Async, Reactor, Coroutines, Virtual Threads), modern Java sistemlerinin standart yapı taşıdır. Aynı zamanda tracing ekiplerinin en sık yüz yüze geldiği sorun kaynağıdır: 'asenkrone metoda geçince trace kayboldu' şikayetinin tek nedeni context propagation eksikliğidir. Bu bölüm her async modelini ayrı ayrı işler.

12.1 Neden Async Tracing Zor?

Hatırlayın: OTEL Context, default olarak thread-local taşınır. Sync programlamada her request bir thread'de doğar ve aynı thread'de tüm iş yapılır; context propagation otomatiktir. Async programlamada ise iş başka bir thread'e devredilir; thread-local kopyalanmazsa context kaybolur. Span açma anında current context'in parent'ı, yanlış (eski, başka isteğin) span'ı gösterirse

12.2 CompletableFuture

```
// YANLIŞ – context async iş içinde kayıp
CompletableFuture.supplyAsync(() -> {
    Span span = tracer.spanBuilder("async-work").startSpan();
    // span.getParentSpanId() == 0
});

// DOĞRU – Context'i capture edip wrap et
Context current = Context.current();
CompletableFuture.supplyAsync(() -> {
    try (Scope s = current.makeCurrent()) {
        Span span = tracer.spanBuilder("async-work").startSpan();
        try { /* iş */ } finally { span.end(); }
    }
});

// EN İYİSİ – agent zaten yapıyor
// otel.instrumentation.java-util-concurrent.enabled=true (default true)
```

12.3 Spring @Async

Spring @Async, kendi TaskExecutor'unu kullanır. Default

12.3.1 ContextPropagatingTaskDecorator (Spring 6.1+)

```
@Configuration
@EnableAsync
public class AsyncConfig {
    @Bean(name = "tracedExecutor")
    public TaskExecutor tracedExecutor() {
        ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
        exec.setCorePoolSize(8);
        exec.setMaxPoolSize(32);
        exec.setQueueCapacity(200);
        exec.setTaskDecorator(new ContextPropagatingTaskDecorator()); // KRITİK
        exec.initialize();
        return exec;
    }
}
```

12.3.2 Manuel TaskDecorator

```
public class OtelTaskDecorator implements TaskDecorator {
    @Override
    public Runnable decorate(Runnable runnable) {
        Context captured = Context.current();
        return () -> {
            try (Scope s = captured.makeCurrent()) {
                runnable.run();
            }
        };
    }
}
```

12.4 Reactor (WebFlux)

Reactor 3.5+

```
// Reactor + OTel – modern (Spring Boot 3.2+)
@GetMapping("/alarms/{id}")
public Mono<AlarmDto> get(@PathVariable String id) {
    return alarmRepo.findById(id)
        .flatMap(this::enrich) // her aşamada context taşınır
        .map(AlarmDto::from);
}
```

Önemli not: Reactor scheduler'ları (Schedulers.boundedElastic(), parallel()) thread switch yapar. Eğer Reactor 3.5 öncesi versiyondaysanız,

12.5 Virtual Threads (Java 21+)

Virtual thread'ler context propagation açısından özel bir durum:

- ThreadLocal'lar virtual thread'lerde de çalışır — ama virtual thread yaratırken kopyalanır mı? Hayır, ThreadLocal child thread'e otomatik kopyalanmaz.
- InheritableThreadLocal virtual thread'de çalışır.
- Yeni ScopedValue API'si (preview, Java 21) immutable, daha verimli alternatif sunar.
- OpenTelemetry Java agent 1.32+ virtual thread instrumentation içerir.

```
// Java 21 virtual thread + manuel context capture
Context current = Context.current();
Thread vt = Thread.ofVirtual().start(() -> {
    try (Scope s = current.makeCurrent()) {
        Span span = tracer.spanBuilder("vt-work").startSpan();
        try { /* iş */ } finally { span.end(); }
    }
});
```

12.6 Kafka Consumer Async Pattern

Kafka consumer'da agent'ın açtığı 'process' span'ı consume metodu sonuna kadar yaşar. İçeride executor.submit() yapıp 'arka planda işleyeyim' dersiniz context kaybolur.

```
// YANLIŞ
@KafkaListener(topics = "alarms.in")
public void onAlarm(AlarmEvent ev) {
    executor.submit(() -> process(ev)); // CONTEXT KAYIP
}
```

```
// DOĞRU
@KafkaListener(topics = "alarms.in")
public void onAlarm(AlarmEvent ev) {
    Context current = Context.current();
    executor.submit(() -> {
        try (Scope s = current.makeCurrent()) {
            process(ev);
        }
    });
}
```

12.7 Bölüm 12 — Anti-Pattern'lar ve Çözümler

ANTI-PATTERN

ExecutorService kullanıp Context.taskWrapping unutmak

@Async ile context propagation decorator olmadan

Thread.start() ile native thread başlatıp context'i el ile taşımamak

Reactor pipeline'da subscribeOn/publishOn ile thread switch ama context propagation yok

Virtual thread'i factory ile yaratıp ScopedValue / Context.current() capture etmemek

Kafka consumer'da background executor'a iş atıp context taşımamak

DEBUGGING

Async kayıp tespiti: span'ın trace_id parent'ı ile aynı mı? Farklıysa context loss.

Aktif span beklenenden farklı mı? Context leak; önceki request'in span'ı thread-local'da kalmış.

Java agent ile çalışıyorsanız: otel.javaagent.debug=true ile hangi executor'ın instrument edildiğini görebilirsiniz.

Sürpriz: 'auto-config aktif ama @Async hala kopuk' = SimpleAsyncTaskExecutor kullanıyorsun;

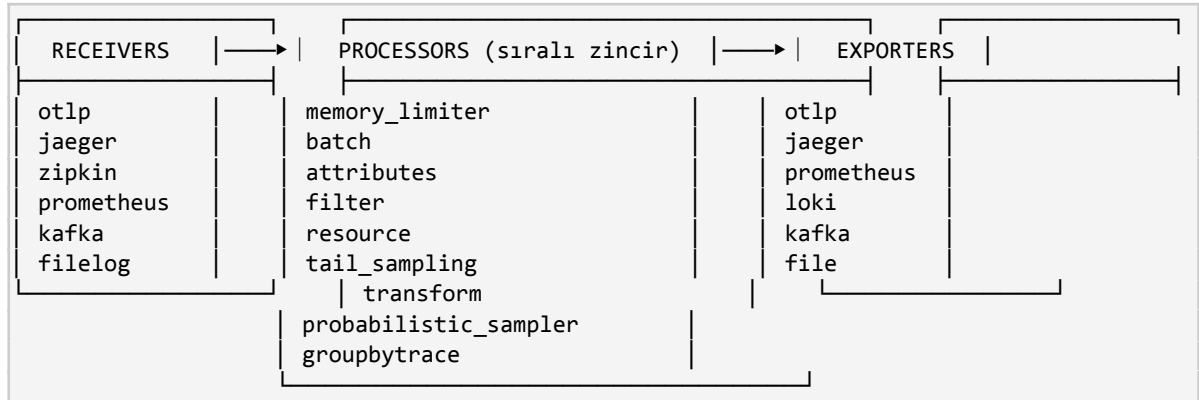
ThreadPoolTaskExecutor + decorator gerek.

Bölüm 13 — OpenTelemetry Collector Derin İnceleme

OpenTelemetry Collector, observability platformunuzun veri düzlemidir. Servislerden gelen telemetri verisini kabul eden, dönüştüren ve nihai backend'lere ileten ayrı bir process'tir. Doğru yapılandırılmadığı zaman tek başına tüm observability platformunuzu çökertebilir; iyi yapılandırıldığı zaman ise yüz binlerce span/saniye trafiği sorunsuz işleyebilir.

13.1 Collector Mimarisi

Collector iç mimarisi üç ana parça etrafında kurulur:



Collector iki ana dağıtımda gelir:

13.2 Pipeline Konfigürasyonu Anatomisi

```

# Tam tipik bir collector config
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
        max_recv_msg_size_mib: 16
      http:
        endpoint: 0.0.0.0:4318
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 1500
    spike_limit_mib: 250
  resource:
    attributes:
      - key: deployment.environment
        value: production
        action: upsert
  batch:
    timeout: 2s
    send_batch_size: 1024
    send_batch_max_size: 2048
exporters:
  otlp/jaeger:
    endpoint: jaeger-collector:4317
    tls:
  
```

```

    insecure: true
  prometheus:
    endpoint: 0.0.0.0:8889
    resource_to_telemetry_conversion:
      enabled: true
  extensions:
    health_check:
      endpoint: 0.0.0.0:13133
    pprof:
      endpoint: 0.0.0.0:1777
    zpages:
      endpoint: 0.0.0.0:55679
  service:
    extensions: [health_check, pprof, zpages]
    pipelines:
      traces:
        receivers: [otlp]
        processors: [memory_limiter, resource, batch]
        exporters: [otlp/jaeger]
      metrics:
        receivers: [otlp]
        processors: [memory_limiter, resource, batch]
        exporters: [prometheus]
    telemetry:
      logs:
        level: info
      metrics:
        level: detailed
        address: 0.0.0.0:8888

```

13.3 Processor Sırası — Hayati Bir Konu

Processor sırası, davranışı doğrudan etkiler. Genel kural:

13. memory_limiter HER ZAMAN ilk olmalı. OOMKilled riskinden korumak için.
14. resource / attributes / filter — veri temizliği ve enrichment.
15. tail_sampling — eğer kullanılıyorsa, batch'ten ÖNCE.
16. batch — HER ZAMAN sondan bir önceki processor.
17. exporters — son halka.

UYARI

batch processor'ı önde koymak: tail_sampling işe yaramaz çünkü kararlar trace tamamlanmadan alınır.
 memory_limiter'ı atlamak: pipeline overflow olduğunda collector OOM olur, tüm telemetri kaybolur.
 filter processor'ı resource'tan önce koymak: filter'ın kullandığı attribute henüz set edilmemiştir.

13.4 memory_limiter Processor

Collector'ın OOM olma riskini azaltır. Periyodik olarak (default 1sn) RAM kullanımını kontrol eder; eşiği aşarsa

```

processors:
  memory_limiter:
    check_interval: 1s

```

```

limit_mib: 1500      # process RAM limiti (soft)
spike_limit_mib: 250  # ani spike toleransı
# Toplam hard limit ~= limit_mib + spike_limit_mib

```

memory_limiter'in yapamadığı: GC tetikleyemez, native memory'i (Go runtime memory) sınırlandıramaz. K8s pod memory limit'i bu limit'ten %20-30 fazla olmalı; aksi halde memory_limiter'in 'refuse' tepkisi K8s OOMKill'inden geç kalır.

13.5 batch Processor

Batch processor, multiple span/metric/log'u tek bir export çağrısında gönderir. Bu, exporter tarafında network IO sayısını azaltır.

Parametre	Default	Anlam
timeout	200ms	Bu süre dolunca queue boş olsa da gönder
send_batch_size	8192	Bu sayıya ulaşıncaya hemen gönder
send_batch_max_size	0 (limitsiz)	Tek batch'in max boyutu

Argela boyutunda bir sistem için tipik değerler:

13.6 Attribute ve Resource Processor'ları

Span/metric attribute'larını ve resource attribute'larını eklemek, silmek, hash'lemek için kullanılır. PII redaction, format dönüşümü, vendor-specific attribute eklemek için kritiktir.

```

processors:
  attributes/redact:
    actions:
      - key: http.request.header.authorization
        action: delete
      - key: http.request.header.cookie
        action: delete
      - key: user.email
        action: hash
      - key: db.statement
        action: update
        from_attribute: db.statement.sanitized
      - pattern: ^url\.full$
        action: extract
        from_attribute: url.full
        regex: '^(?P<scheme>[^:]+):/(?P<host>[/]+)(?P<path>/[^\?]*)'

```

13.7 filter Processor

Belirli span/metric/log'ları drop etmek için kullanılır. Düzgün bir filter, telemetri maliyetini önemli ölçüde düşürür.

```

processors:
  filter/healthcheck:
    error_mode: ignore
    traces:
      span:
        - 'attributes["http.target"] == "/actuator/health"'

```

```
- 'attributes["http.target"] == "/actuator/prometheus"'
- 'name == "GET /favicon.ico"'
```

OTTL (OpenTelemetry Transformation Language) ile gelişmiş filter ifadeleri yazılabilir. OTTL, span/metric'lerin tipini, attribute'larını, status'unu, resource'unu sorgulayarak boolean ifade üretir.

13.8 transform Processor

Daha güçlü dönüşümler için: attribute hesaplama, span name değiştirme, severity değiştirme. OTTL ifadeleri kullanır.

```
processors:
  transform:
    trace_statements:
      - context: span
        statements:
          - set(attributes["service.canonical.name"],
                Concat([resource.attributes["service.namespace"],
                      resource.attributes["service.name"]], "/"))
          - replace_match(name, "GET /users/?*", "GET /users/:id")
          - delete_key(attributes, "url.full") where IsMatch(attributes["url.full"],
                ".*token=.*")
```

13.9 tail_sampling Processor

Head sampling (servis tarafında karar) yerine, trace'in tamamı geldikten sonra karar verir. 'Sadece hatalı veya yavaş trace'leri tut' gibi gelişmiş politikalar mümkün hale gelir.

```
processors:
  tail_sampling:
    decision_wait: 30s
    num_traces: 50000
    expected_new_traces_per_sec: 100
    policies:
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      - name: slow
        type: latency
        latency: { threshold_ms: 1000 }
      - name: critical_endpoint
        type: string_attribute
        string_attribute:
          key: http.route
          values: ["/api/v1/checkout", "/api/v1/payment"]
      - name: baseline
        type: probabilistic
        probabilistic: { sampling_percentage: 1 }
```

Tail sampling'in maliyeti: tüm trace'leri bellekte tutmak gerekir.

PERFORMANS ETKİSİ

Tail sampling 10k RPS'de 30sn decision_wait için yaklaşık 5-15GB RAM ihtiyacı doğurur.

Bunu single collector ile yapmak mümkün değildir; sharded gateway gerekir (load balancer ile trace_id'ye göre routing).

Tail sampling kullanıyorsanız gateway collector'ı stateful set edin.

13.10 groupbytrace Processor

Tail sampling'in altyapısı. Aynı trace'e ait span'ları tek bir batch'te biriktirir. Tail sampling olmadan da debugging için kullanılabilir.

13.11 Collector İçi Telemetri (Self-Observability)

Collector kendi metric'lerini Prometheus formatında

Metric	Anlam	Alert Tetikleyici
otelcol_receiver_accepted_spans	Receiver'a giren span sayısı	0'a düşerse: receiver bozuk
otelcol_receiver_refused_spans	Reddedilen span sayısı	>0 ise: backpressure/limit aşımı
otelcol_processor_dropped_spans	Processor drop etti	>0 ise: sampling veya filter agresif
otelcol_exporter_sent_spans	Backend'e başarılı export	Düşüş: backend down
otelcol_exporter_send_failed_spans	Export hatası	>0 sürekli: backend ulaşılmıyor
otelcol_exporter_queue_size	Exporter retry queue	Sürekli yüksek: backend yavaş
process_memory_rss	RAM kullanımı	Limit'e yakın: memory_limiter tetiklemeli

13.12 Bölüm 13 — Tavsiyeler ve Anti-Pattern'lar

PROD TAVSİYESİ

Collector'ı her zaman ayrı process/pod olarak çalıştırın; uygulama ile sidecar veya gateway.
memory_limiter ALWAYS, batch HER ZAMAN PROCESSOR ZİNCİRİNİN SONUNDA.
Collector self-metric'leri Prometheus'a scrape edilsin; queue/refused/dropped alarmları kurulsun.
Helm chart kullanın (open-telemetry/opentelemetry-collector); custom Deployment YAML 6 ay sonra çözülmez hale gelir.

ANTI-PATTERN

Collector'sız doğrudan backend'e push: backend down → uygulama down.
batch processor olmadan: her span tek tek gönderilir, network IO patlar.
Tail sampling tek collector ile yapmaya kalkmak: trace shard'ları yanlış collector'a gelir, decision yanlış.
Filter processor olmadan health check span'larını backend'e taşımak: %30-50 gereksiz veri saklanır.

Bölüm 14 — OTLP Protokolü ve gRPC vs HTTP

OTLP (OpenTelemetry Protocol), OpenTelemetry'nin tasarladığı ve üç sinyali (traces, metrics, logs) taşıyan binary protokoldür. Bu bölüm OTLP'nin yapısını, gRPC ve HTTP varyantlarını, retry mekanizmalarını ve prod-ortamı tradeoff'larını işler.

14.1 OTLP Nedir, Neden Var?

OpenTelemetry'den önce her vendor kendi wire formatına sahipti: Jaeger Thrift, Zipkin JSON, Datadog protobuf, vs. Vendor değiştirmek wire-level kod değişikliği demektir. OTLP, üç sinyali tek bir protobuf schema'sı altında birleştirir ve vendor-bağımsız bir 'lingua franca' yaratır.

OTLP protobuf schema'sı GitHub'da

14.2 Tek Protokol, İki Transport: gRPC ve HTTP

Özellik	OTLP/gRPC	OTLP/HTTP
Port	4317	4318
Encoding	Protobuf binary	Protobuf binary (default) veya JSON
Transport	HTTP/2 + gRPC framing	HTTP/1.1 veya HTTP/2
Streaming	Var (unary kullanılır pratikte)	Yok
Header taşıma	gRPC metadata	HTTP headers
Firewall dostluğu	Düşük (HTTP/2 gerekli, proxy'ler engel olabilir)	Yüksek
Performans	Daha hızlı (binary framing, multiplexing)	Biraz daha yavaş
Resmi destek	Tüm SDK'larda	Tüm SDK'larda

14.2.1 Endpoint Yapısı

```
# gRPC
grpc://otel-collector:4317
# Service path otomatik: /opentelemetry.proto.collector.trace.v1.TraceService/Export

# HTTP
http://otel-collector:4318/v1/traces
http://otel-collector:4318/v1/metrics
http://otel-collector:4318/v1/logs
```

14.3 OTLP Mesaj Yapısı

Bir OTLP trace mesajının protobuf hiyerarşisi:

```
ExportTraceServiceRequest
├─ resource_spans (ResourceSpans[])
│   └─ resource (Resource) ← service.name, service.version, k8s.* etc.
│       └─ attributes (KeyValue[])
│   └─ scope_spans (ScopeSpans[]) ← her instrumentation library için bir tane
│       └─ scope (InstrumentationScope) ← örn. "io.opentelemetry.spring-webmvc-3.1"
│       └─ spans (Span[])
```

```

├─ trace_id (16 byte)
├─ span_id (8 byte)
├─ parent_span_id (8 byte, optional)
├─ name
├─ kind (SpanKind)
├─ start_time_unix_nano
├─ end_time_unix_nano
├─ attributes (KeyValue[])
├─ events (Event[])
├─ links (Link[])
└─ status (Status)

```

Resource ve scope, batch içinde DEDUPE edilir. Yani 1000 span aynı service.name'i taşıyorsa, resource sadece 1 kere serialize edilir. Bu OTLP'nin bandwidth verimliliğinin temelidir.

14.4 gRPC vs HTTP — Pratik Karar

14.4.1 Ne zaman gRPC tercih edilir?

- Düşük latency hedefi var, network içi (sidecar / aynı node)
- Çok yüksek throughput (10k+ span/saniye/instance)
- mTLS ile servis kimliklendirme — gRPC'nin SAN matching mantığı daha katı
- HTTP/2 destekleyen load balancer (Envoy, Linkerd) zaten varsa

14.4.2 Ne zaman HTTP tercih edilir?

- Eski proxy / firewall / WAF arkasındaysanız
- Multi-region setup; gRPC long-lived connection sticky olabilir
- Frontend / browser instrumentation (gRPC-Web ekstra adım)
- Lambda / Cloud Functions gibi short-lived runtime (connection reuse zor)

14.5 Retry ve Backoff Mekanizmaları

OTLP exporter, transient hatalarda retry yapar. Default davranış:

```

# Java SDK exporter default retry config
otel.exporter.otlp.timeout = 10s
otel.exporter.otlp.retry.enabled = true
otel.exporter.otlp.retry.initial-backoff = 1s
otel.exporter.otlp.retry.max-backoff = 5s
otel.exporter.otlp.retry.max-attempts = 5

```

Hangi hatalar retry edilir? gRPC standardına göre

UYARI

Retry storm tehlikesi: backend down olunca tüm client'lar aynı anda retry başlatır.

Exponential backoff + jitter zorunlu (OTel SDK default'ta jitter yapar).

Backend down süresi uzarsa: BSP queue dolar, span drop başlar.

Collector kullanıyorsanız retry policy collector'a delegated; client side timeout daha düşük tutulmalı.

14.6 Compression

OTLP gRPC default

14.7 mTLS ve Authentication

```
# Client tarafı (Java SDK)
otel.exporter.otlp.endpoint=https://otel-gateway:4317
otel.exporter.otlp.certificate=/etc/certs/ca.pem
otel.exporter.otlp.client.certificate=/etc/certs/client-cert.pem
otel.exporter.otlp.client.key=/etc/certs/client-key.pem
otel.exporter.otlp.headers=Authorization=Bearer YOUR_TOKEN_HERE
```

Collector tarafı:

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
        tls:
          cert_file: /etc/certs/server-cert.pem
          key_file: /etc/certs/server-key.pem
          client_ca_file: /etc/certs/ca.pem # mTLS için
        auth:
          authenticator: oidc/keycloak
```

14.8 Bölüm 14 — Production Notları

PROD TAVSİYESİ

Cluster içi: gRPC + insecure (cluster mesh güvende ise). Cross-cluster: mTLS + gRPC.

Cross-region veya internet üzerinden: HTTP + Bearer auth + TLS. Connection reuse'ı incele.

Compression hep açık (gzip). CPU bedeli %2-5; bandwidth tasarrufu %70+.

Client timeout < Collector receive timeout < Backend timeout (zincir kuralı).

Bölüm 15 — Collector Pipeline Tasarım Pattern'ları

Tek bir collector instance'ı küçük workload için yeterlidir; ama prod'da pipeline tasarımı stratejik kararlar gerektirir. Bu bölüm gerçek dünya pattern'larını ve neden böyle tasarlandıklarını işler.

15.1 Pattern 1: Single-Tier Sidecar

Her pod'da bir collector container. Uygulama localhost'a OTLP gönderir. Sidecar OTLP'yi backend'e iletir.

- ✓ Basitlik: tek katman
- ✓ Network: localhost
- ✗ Tail sampling yok (collector cross-trace görmüyor)
- ✗ Config update zor (her pod'u yeniden başlat)
- ✗ Resource tüketimi 1 collector/pod

Küçük ekipler ve <50 pod ortamlar için makul. Argela demo projesi bu pattern'a yakındır.

15.2 Pattern 2: DaemonSet Agent

Her K8s node'unda bir collector pod (DaemonSet). Uygulamalar host'un dahili IP'sine gönderir.

- ✓ Sidecar'dan daha az resource (her pod yerine her node)
- ✓ Node-level enrichment (k8s.node.name vs.)
- ✗ Hala tail sampling yok
- ✗ Pod restart node'u etkilemez ama node restart tüm pod'lar geçici tracing kaybeder

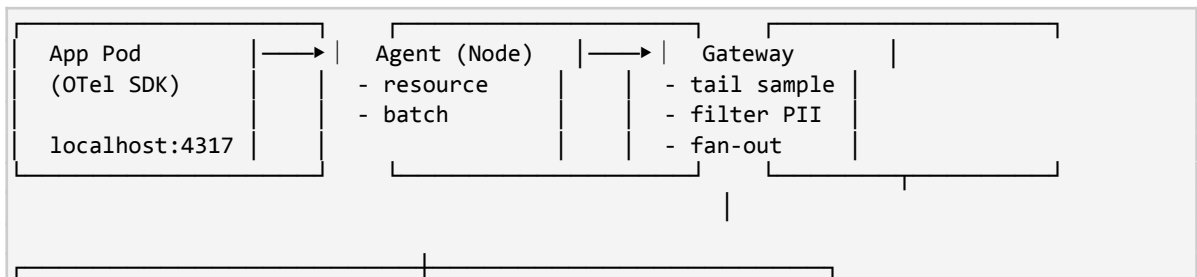
15.3 Pattern 3: Gateway Collector (Centralized)

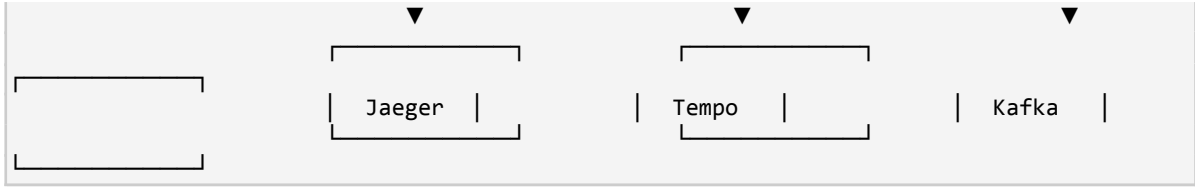
Birden çok collector replica'sı bir LB arkasında. Tüm uygulamalar buraya gönderir.

- ✓ Tail sampling, attribute redaction, multi-backend fan-out
- ✓ Config tek noktada
- ✓ Scaling horizontal
- ✗ Network hop ekstra
- ✗ Single point of failure (LB + collector)

15.4 Pattern 4: Two-Tier (Agent + Gateway) — Endüstri Standardı

Node'da DaemonSet agent (basit batching, resource enrichment) + merkezi gateway cluster (tail sampling, fan-out).





Avantaj: tek bir uygulama pod'unun gönderdiği veri önce node-local agent'ta tampon edilir (network güvenli), sonra gateway'de cross-trace agregasyonu yapılır. Gateway down olsa agent local buffer'da tutar (limit dahilinde).

15.5 Pattern 5: Sharded Gateway (Tail Sampling İçin)

Tail sampling, aynı trace'in tüm span'larının aynı collector instance'ına gelmesini gerektirir. Bu, basit LB ile mümkün değil — round-robin trace'i parçalar. Çözüm:

```
# Front-tier gateway: trace_id'ye göre back-tier gateway'e yönlendirir
exporters:
  loadbalancing:
    routing_key: traceID
    protocol:
      otel:
        tls: { insecure: true }
    resolver:
      static:
        hostnames:
          - gateway-backend-0.gateway:4317
          - gateway-backend-1.gateway:4317
          - gateway-backend-2.gateway:4317

# back-tier gateway: tail_sampling yapar
processors:
  groupbytrace: { wait_duration: 10s, num_traces: 100000 }
  tail_sampling: ...
```

15.6 Pattern 6: Lambda / Serverless Tracing

Lambda gibi serverless ortamlarda Java agent çalışır ama her cold start agent yüklemesi 1-3sn ekler. Çözümler:

- Lambda Layer olarak agent'ı yükle (AWS Distro for OpenTelemetry / ADOT)
- Cold start'ı azaltmak için 'snap-start' veya provisioned concurrency
- BSP timeout düşük tut (200ms); Lambda execution sonrası flush
- Lambda extension olarak collector çalıştır (sidecar)

15.7 Bölüm 15 — Topology Seçim Tablosu

Senaryo	Önerilen Pattern	Neden
Demo / küçük ekip (<10 servis)	Sidecar veya DaemonSet	Basitlik öncelikli
Orta ölçek (10-50 servis)	DaemonSet + Gateway	Config yönetimi, ilk tail sampling
Büyük ölçek (50-500 servis)	Two-tier, sharded gateway	Tail sampling, fan-out
Multi-region	Per-region gateway + global tier	Latency ve compliance

Senaryo	Önerilen Pattern	Neden
Multi-tenant SaaS	Sharded + per-tenant attribute routing	İzolasyon, faturalama
Serverless heavy	Lambda extension + central gateway	Cold start, ephemeral compute

Bölüm 16 — Exporter'lar ve Backend Entegrasyonu

Exporter, collector pipeline'inin son halkasıdır: işlenmiş telemetri verisini nihai backend'lere gönderir. Collector'da 100+ exporter mevcut; bu bölüm en sık kullanılanları ve dikkat noktalarını işler.

16.1 OTLP Exporter — En Yaygın

Backend OTLP konuşuyorsa (Jaeger 1.42+, Tempo, Honeycomb, Lightstep, vs.) OTLP exporter en doğru seçim.

```
exporters:
  otlp:
    endpoint: jaeger-collector:4317
    tls:
      insecure: true
    sending_queue:
      enabled: true
      num_consumers: 4
      queue_size: 5000
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
      max_elapsed_time: 5m
```

Önemli parametre: `sending_queue.queue_size`. Backend yavaşlarsa retry'lar bu kuyrukta birikir. Limitsiz bırakırsanız OOM riski; çok düşük tutarsanız transient sorunda veri kaybı. $RPS \times 30s \times \text{span/trace}$ iyi bir başlangıç değeri.

16.2 Jaeger Exporter (Legacy)

Jaeger 1.42 öncesi versiyonlar OTLP konuşmazdı; Thrift protokolü ile çalışıyordu.

```
# DEPRECATED – yeni Jaeger sürümleri için OTLP kullan
exporters:
  jaeger:
    endpoint: jaeger-collector:14250
    tls: { insecure: true }
```

16.3 Prometheus Exporter (Pull)

Metric'leri Prometheus formatında bir endpoint'ten serve eder. Prometheus server scrape eder.

```
exporters:
  prometheus:
    endpoint: 0.0.0.0:8889
    namespace: argela
    const_labels:
      cluster: prod-eu
    resource_to_telemetry_conversion:
      enabled: true # resource attribute'larını label olarak ekle
```

Avantaj: Prometheus tarafında zaten kurulu scraping altyapısı kullanılır. Dezavantaj: collector instance ölünce o instance'taki son metric'ler kayıp.

16.4 PrometheusRemoteWrite Exporter (Push)

Doğrudan Prometheus remote-write API'sine push eder. Mimir, Thanos, Cortex gibi storage'lar için yaygın.

```
exporters:
  prometheusremotewrite:
    endpoint: http://mimir:9009/api/v1/push
    headers:
      X-Scope-OrgID: argela-tenant
    resource_to_telemetry_conversion:
      enabled: true
    external_labels:
      cluster: prod-eu
```

16.5 Loki Exporter

Log'ları Grafana Loki'ye push eder. Hâlâ deneysel; çoğu durumda promtail/Alloy ile log toplamak daha olgun.

16.6 Kafka Exporter

Telemetri verisini Kafka topic'ine yazar. Trace data lake, custom processing pipeline, multi-backend fan-out için kullanılır.

```
exporters:
  kafka:
    brokers: [kafka-1:9092, kafka-2:9092, kafka-3:9092]
    topic: otlp_spans
    encoding: otlp_proto
    producer:
      compression: snappy
      max_message_bytes: 1048576
      flush_max_messages: 1000
```

16.7 ClickHouse Exporter

Trace + log + metric'leri ClickHouse'a yazar. SigNoz, Uptrace gibi açık kaynak observability platformlarının altyapısıdır.

16.8 File Exporter

Local dosyaya yazma. Debug, replay, audit, regülasyon uyum için kullanışlı.

```
exporters:
  file:
    path: /var/log/otelcol/spans.jsonl
    rotation:
      max_megabytes: 100
      max_days: 7
      max_backups: 5
      localtime: false
```

16.9 Vendor-Specific Exporters

Datadog, New Relic, Dynatrace, Honeycomb, Lightstep, AWS X-Ray, Azure Monitor, Google Cloud Trace gibi vendor'lar için ayrı exporter'lar mevcut. Yeni vendor'a geçerken: client kodu değişmez, sadece collector exporter blok'u güncellenir. Bu, OpenTelemetry'nin asıl vaadidir.

16.10 Multi-Exporter Pattern: Fan-Out

Aynı veriyi birden fazla backend'e göndermek (örn. Jaeger + Tempo + Kafka):

```
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlp/jaeger, otlp/tempo, kafka]
```

Önemli: bir exporter yavaşladığında, batch processor diğerlerinin de yavaşlamasına neden olabilir. Critical olmayan exporter'ları ayrı pipeline'da tutun:

```
service:
  pipelines:
    traces/critical:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlp/jaeger] # ana backend
    traces/archive:
      receivers: [otlp]
      processors: [memory_limiter, batch/archive]
      exporters: [kafka, file] # arşiv, daha tolerans
```

16.11 Bölüm 16 — Production Tavsiyeleri

PROD TAVSİYESİ

Birden fazla backend kullanıyorsanız critical/non-critical exporter'ları ayrı pipeline'a koyun.
 OTLP exporter'ında sending_queue + retry_on_failure HER ZAMAN aktif olsun.
 Vendor exporter'ları kullanırken vendor SDK versiyonunu CI'da test edin; breaking change'ler sıktır.
 File exporter ile her şeyi yazmak: disk patlatır. Sampling + rotation şart.

ANTI-PATTERN

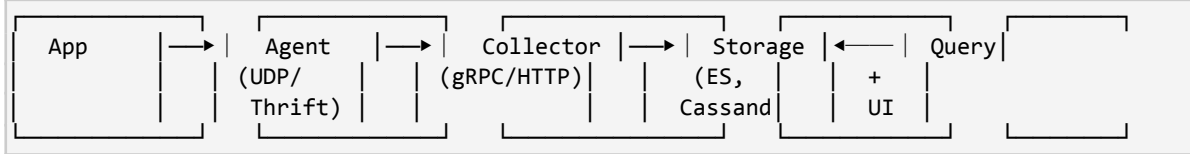
Tek pipeline'a 5 farklı backend exporter koymak: en yavaş olanı batch'i kilitler.
 Sending_queue size limitsiz: backend down → OOM.
 Exporter retry'ı kapatmak: transient hatada veri silinir.
 Production'da file exporter ile her şeyi log'lamak: disk yarın dolmuş olur.

Bölüm 17 — Jaeger Mimarisi ve Bileşenleri

Jaeger, Uber tarafından 2015'te başlatılan ve 2017'de CNCF'e bağışlanan, açık kaynak distributed tracing platformudur. CNCF Graduated statüsündedir. Bu bölüm Jaeger'ın iç mimarisini, hangi bileşenin neyi yaptığını, eski v1 ve yeni v2 arasındaki farkları işler.

17.1 Jaeger v1 Mimarisi (Klasik)

Jaeger v1 (1.x sürümleri) klasik distributed tracing pipeline'ını implement eder:



Bileşen	Sorumluluk	Tipik Port
jaeger-agent	App'ten UDP/Thrift al, batch'le, collector'a forward et	6831/UDP, 6832/UDP
jaeger-collector	Span'ları al, validate et, storage'a yaz	14268 (HTTP), 14250 (gRPC), 4317 (OTLP)
jaeger-query	Storage'tan span okur, UI'a serve eder	16686 (UI), 16685 (gRPC)
jaeger-ingester	Kafka'dan span tüketip storage'a yazar (opsiyonel)	-

17.2 Jaeger v2 — OpenTelemetry Collector Tabanlı (2024+)

Jaeger v2 (2024'te GA), kendi binary'sini tamamen baştan yazmak yerine OpenTelemetry Collector'ı baz aldı. Yani jaeger v2 = OTel Collector + Jaeger-specific receiver/exporter/extension'lar. Bu, Jaeger ekibinin OTel ile dual stack maintenance'i sona erdirmeye stratejisinin sonucu.

Pratik sonuç:

- jaeger-agent kaldırıldı (artık OTel Collector agent rolü oynar)
- jaeger-collector ve jaeger-query tek bir 'jaeger' binary'sinde birleşti
- Konfigürasyon OTel Collector YAML formatında
- OTLP native receiver'a sahip; başka tracing client kütüphanesi gerekmez

```

# Jaeger v2 docker-compose örneği
services:
  jaeger:
    image: jaegertracing/jaeger:2.0.0
    ports:
      - "16686:16686" # UI
      - "4317:4317"   # OTLP gRPC
      - "4318:4318"   # OTLP HTTP
    environment:
      - SPAN_STORAGE_TYPE=memory # demo
    # Prod için config dosyası mount:
    # volumes: ["/jaeger-config.yaml:/etc/jaeger/config.yaml"]
  
```

PROD TAVSİYESİ

Yeni projelerde Jaeger v2'ye geçin; Jaeger v1 maintenance moduna alındı.

Argela'daki demo docker-compose'da Jaeger v1 olabilir; OTel Collector zaten ayrı; tek değişiklik image tag.

Migration tek seferlik: storage formatı uyumlu, sadece config farkı.

17.3 Jaeger UI

Jaeger UI,

- Service ve operation dropdown ile trace arama
- Tag filter (örn. http.status_code=500)
- Min duration filter ('en az X ms süren trace'ler')
- Trace timeline view: span'ların gantt chart'ı
- Service Performance Monitoring (SPM) — RED metric grafikleri
- Trace comparison — iki trace'i yan yana
- System Architecture (Service Dependency Diagram) — servisler arası akışın graph görünümü

17.4 Service Performance Monitoring (SPM)

Jaeger v1.30+ ve v2'de gelen SPM, trace verisinden RED (Rate, Errors, Duration) metric'leri türetir.

Prometheus formatında bir metric exporter (genelde

```
# spanmetrics processor – collector'da
processors:
  spanmetrics:
    metrics_exporter: prometheus
    latency_histogram_buckets: [10ms, 50ms, 100ms, 500ms, 1s, 5s]
    dimensions:
      - name: http.method
      - name: http.status_code

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [spanmetrics, batch]
      exporters: [otlp/jaeger]
    metrics:
      receivers: [spanmetrics]
      exporters: [prometheus]
```

Bu konfigle Jaeger UI'da her servis için otomatik RED dashboard görünür. Argela projenizde manuel custom metric yerine spanmetrics processor düşünebilirsiniz.

17.5 Sampling Stratejileri — Jaeger Tarafı

Jaeger collector, client'lardan periyodik olarak 'sampling strategy' fetch isteklerini cevaplayabilir. Bu, sampling rate'ini merkezi olarak yönetmenizi sağlar.

```
# strategies.json – Jaeger collector'a verilir
{
```

```

"default_strategy": {
  "type": "probabilistic",
  "param": 0.1
},
"service_strategies": [
  {
    "service": "fault-collector",
    "type": "ratelimiting",
    "param": 100 // RPS
  },
  {
    "service": "fault-processor",
    "type": "probabilistic",
    "param": 0.5,
    "operation_strategies": [
      {
        "operation": "POST /api/v1/process",
        "type": "probabilistic",
        "param": 1.0 // her zaman sample
      }
    ]
  }
]
}

```

17.6 Bölüm 17 — Tavsiyeler

PROD TAVSİYESİ

Yeni kurulumda Jaeger v2 + OTEL Collector pipeline kullanın; tek binary, tek config.

Demo veya test ortamı: in-memory storage. Prod: Elasticsearch/OpenSearch veya Cassandra.

SPM aktifleştirin; ekstra Prometheus tarafına spanmetrics push edilsin.

Sampling stratejisini Jaeger'dan değil, OTEL SDK + Collector tail_sampling üzerinden yönetin (daha modern).

Bölüm 18 — Jaeger Storage Backend'leri: Elasticsearch, OpenSearch, Cassandra

Jaeger'in belki en kritik tasarım kararı, hangi storage backend'in kullanılacağıdır. Storage seçimi; retention, query performance, operasyonel kompleksite ve maliyetin hepsini etkiler. Bu bölüm dört ana seçeneği derinlemesine işler: Elasticsearch, OpenSearch, Cassandra, ve ClickHouse (community).

18.1 In-Memory Storage — Sadece Demo

Jaeger varsayılan olarak in-memory storage ile gelir. Bu, RAM'de bir HashMap'tir. Avantaj: kurulum yok. Dezavantaj: pod restart = tüm trace kaybı. CLAUDE.md'de belirtildiği üzere bu Argela projesinin de bilinen kısıtıdır.

18.2 Elasticsearch (En Yaygın)

Jaeger'in en yaygın prod storage backend'i Elasticsearch'tür. Span'lar tipik olarak günlük index'lere yazılır (

18.2.1 Index Şeması

```
jaeger-span-2026-05-23      # her gün için ayrı index
{
  "traceID": "4bf92f3577b34da6a3ce929d0e0e4736",
  "spanID": "00f067aa0ba902b7",
  "parentSpanID": "e457b5a2e4d86bd1",
  "operationName": "POST /api/v1/process",
  "process": {
    "serviceName": "fault-processor",
    "tags": [...]
  },
  "startTime": 1716461663142000,
  "duration": 23142,
  "tags": [
    { "key": "http.status_code", "type": "int64", "value": 200 },
    { "key": "alarm.severity", "type": "string", "value": "HIGH" }
  ],
  "logs": [...],
  "references": [...]
}
```

18.2.2 Index Sharding ve Sizing

Genel kural: günde 1M trace $\approx$ 30-50GB raw, 60-100GB Elasticsearch (replica dahil).

- Shard count: günlük index için 5 shard tipik; çok büyük cluster'da 10-20.
- Replica: prod 1 replica zorunlu; mission-critical 2 replica.
- Heap: ES node başına 16-31GB (32GB üstü = JVM compressed-oops kaybı).
- Disk: NVMe SSD; HDD ES'i kullanılamaz hale getirir.

18.2.3 Retention ve ILM

Index Lifecycle Management ile günlük index'ler otomatik silinir. Tipik strateji:

- Hot tier: 0-3 gün (SSD, sık erişim)

- Warm tier: 3-7 gün (SSD ama daha az replica)
- Cold tier: 7-30 gün (HDD veya searchable snapshot)
- Delete: 30+ gün

```
# Jaeger ES config örneği
SPAN_STORAGE_TYPE=elasticsearch
ES_SERVER_URLS=https://es-master:9200
ES_USERNAME=jaeger
ES_PASSWORD=...
ES_INDEX_PREFIX=jaeger
ES_TAGS_AS_FIELDS_ALL=true      # tag'leri ayrı field yap - query hızı +, mapping size +
ES_NUM_SHARDS=5
ES_NUM_REPLICAS=1
ES_TIME_RANGE=72h              # query'lerde max range
```

18.2.4 ES Mapping Patlaması — Yaygın Sorun

UYARI

ES_TAGS_AS_FIELDS_ALL=true ile her tag yeni bir field olur.

Yüksek cardinality attribute (user_id, session_id) field count'u patlatır.

ES default field limiti 1000; aştığınızda yeni index oluşmaz, span'lar kayıp.

Çözüm: tags_as_fields_include ile sadece sorgulanan tag'leri field yap, gerisi 'tags' nested object'te kalsın.

18.3 OpenSearch

OpenSearch, AWS'in 2021'de Elasticsearch'ün fork'undan başlattığı, Apache 2.0 lisanslı bir search engine. Jaeger OpenSearch'ü tam destekler; konfigürasyon Elasticsearch ile %95 aynıdır.

OpenSearch'ün avantajları:

- Apache 2.0 lisansı — Elasticsearch'ün eski Apache 2.0 sürümleri 7.10'da donduruldu, sonrası SSPL/Elastic License.
- AWS managed (Amazon OpenSearch Service)
- Anomaly detection, alerting plugin'leri Elasticsearch X-Pack'a alternatif

CLAUDE.md'de doğru not edildiği üzere Jaeger için kalıcı storage olarak OpenSearch/Elasticsearch ekip tarafından önerilir.

18.4 Cassandra

Uber'in original storage tercihi. Çok yüksek yazma throughput'u gerekiyorsa hâlâ caziptir.

Boyut	Cassandra	Elasticsearch
Yazma throughput	Çok yüksek (LSM tree)	Orta (index update)
Okuma flexibility	Düşük (key bazlı)	Çok yüksek (full-text)
Operational kompleksite	Yüksek (repair, compaction)	Orta
Disk verimliliği	Orta (denormalized)	İyi (compressed)
Trace arama UX	Sınırlı	Tam metin + aggregation

Cassandra trace'leri trace_id key'i ile depolar; service+operation index'leri ayrı tablodadır. Karmaşık trace arama (örn. 'attribute X içeren ve >1sn süren trace'ler') Cassandra'da yavaştır. Bu yüzden çoğu prod kurulum Elasticsearch'e geçiş yaptı.

18.5 ClickHouse (Community)

ClickHouse, OLAP odaklı column-oriented bir veritabanıdır. Jaeger çekirdek olarak desteklemez ama community storage backend'leri vardır. SigNoz gibi projeler trace storage olarak ClickHouse kullanır.

ClickHouse'un cazibesi:

- Compression çok yüksek (10-20x)
- Aggregation sorguları çok hızlı
- Disk maliyeti ES'in 1/5'i
- Petabyte ölçeğine ulaşmış prod implementasyonları var (Uber, Cloudflare)

Dezavantaj: Jaeger official desteklemez, third-party adapter gerekir. Maintenance riski.

18.6 Storage Backend Seçim Kılavuzu

Senaryo	Önerilen	Neden
Demo / test	In-memory	Sıfır kurulum
Küçük prod (<100GB/gün)	Elasticsearch/OpenSearch	Kolay query, esnek mapping
Orta prod (100GB-1TB/gün)	OpenSearch + ILM tiers	Maliyet/performans dengeli
Çok büyük (>1TB/gün)	ClickHouse veya Cassandra	Storage maliyeti, yazma throughput
AWS native	Amazon OpenSearch Service	Managed, IAM entegre
GCP native	Tempo / Cloud Trace	Native GCP backend
Cost-conscious + custom	ClickHouse + Grafana Tempo alternatifi	Maximum compression

18.7 Bölüm 18 — Sık Karşılaşılan Storage Sorunları

DEBUGGING

'Trace UI'da görünmüyor': ES query timeout, mapping limit aşımı, veya index lifecycle silmiş.

'Disk doldu, Jaeger çöktü': ILM kurulmamış, eski index'ler silinmiyor.

'Trace yavaş açılıyor (10+ sn)': ES heap düşük, GC sık; veya cross-cluster search düşük performansla.

'Mapping exception in logs': field count limit'i aşıldı; tags_as_fields'i kısıtla.

Bölüm 19 — Tempo, Zipkin ve Diğer Tracing Backend'leri

Distributed tracing pazarında Jaeger tek seçenek değil. Bu bölüm en sık karşılaştırılan açık kaynak alternatiflerini ve seçim kriterlerini işler.

19.1 Grafana Tempo

Grafana Labs'ın açık kaynak trace backend'i (Apache 2.0). Felsefe: 'sadece trace store, search yok.' Tempo, trace'leri object storage'a (S3, GCS, Azure Blob) yazar; trace_id ile lookup yapar. Search için Loki ile log'lardan veya Mimir ile metric exemplar'larından gelir.

Boyut	Jaeger	Tempo
Storage	ES/OpenSearch/Cassandra	Object storage (S3 vs.)
Cost / TB	Yüksek	Çok düşük
Search by attribute	Native	TraceQL (2.x), sınırlı
Search by trace_id	Var	Native, hızlı
UI	Jaeger UI veya Grafana	Sadece Grafana
Multi-tenancy	Sınırlı	Native (X-Scope-OrgID)
İşletme kolaylığı	Orta	Yüksek (sadece object store)

Tempo'nun asıl gücü maliyettir. 1TB trace storage'ı Tempo'da S3'te aylık ~\$23, Elasticsearch'te 10x fazla. Dezavantaj: 'şu attribute'a sahip trace'leri bana göster' tarzı arama Jaeger kadar olgun değil (TraceQL hızla gelişiyor).

19.2 Zipkin

Twitter'ın 2012'de açtığı, ilk popüler open-source tracing backend'i. Hâlâ aktif ama OTEL sonrası ekosistemde daha az tercih ediliyor.

Zipkin'in artıları:

- Çok hafif (tek JAR, in-memory veya MySQL/ES storage)
- B3 propagation format'ı endüstri standardı olarak yaygın
- Yıllardır prod-tested, stabil

Eksileri:

- OpenTelemetry semantic conventions ile %100 uyumlu değil
- Modern dashboard / UI Jaeger'a göre geride
- Service map gibi gelişmiş özellikler zayıf

19.3 SigNoz

Açık kaynak (Apache 2.0), ClickHouse tabanlı, all-in-one observability platformu. Tracing + metric + log + APM tek platformda. Datadog/New Relic'e alternatif olarak konumlanıyor. OTEL-native.

19.4 Uptrace

ClickHouse tabanlı, OTel native, daha küçük ölçekli ama hızlı gelişen bir alternatif.

19.5 Vendor (Ticari) Çözümler

Platform	Pros	Cons
Datadog APM	Olgun, geniş integration	Pahalı, vendor lock-in
New Relic	Tüm sinyaller, ücretsiz tier	Yüksek hacimde maliyet artıyor
Dynatrace	AI tabanlı root cause, agent zengin	En pahalı segment
Honeycomb	Yüksek cardinality friendly, BubbleUp	Daha küçük ekip, Asia bölgesi sınırlı
Lightstep / ServiceNow	Statistical analysis odaklı	Üretici desteği değişiyor
Splunk Observability	Splunk ile entegre	License ücreti

Bütün bu vendor'lar OTLP destekler. Yani OTEL SDK + Collector ile yazdığınız uygulamayı vendor değiştirmek tek bir collector exporter blok değişikliğiyle yapılır. Bu OpenTelemetry'nin asıl ekonomik vaadidir.

19.6 Karar Matrisi

Öncelik	Önerilen
Açık kaynak + Jaeger-tarzı UI	Jaeger v2
Maliyet düşük + Grafana ekosistemi	Tempo + Grafana
All-in-one (trace+metric+log)	SigNoz veya commercial
Yüksek cardinality + iyi UX	Honeycomb
Enterprise + AI insights	Datadog/Dynatrace
Self-hosted + sınırsız retention	Tempo + S3

Bölüm 20 — Prometheus, Grafana, Loki ile Tam Stack

Observability stack'i sadece tracing'den ibaret değil. Modern Grafana ekosisteminde Prometheus (metric), Loki (log), Tempo/Jaeger (trace) ve Mimir (long-term metric) birlikte çalışır. Bu bölüm bu komponentlerin OpenTelemetry ile nasıl birleştiğini ve correlate edildiğini işler.

20.1 Prometheus — Metric Store

Prometheus, time-series metric storage'da defacto standarttır. Mimari özellikleri:

- Pull-based: Prometheus, scrape target'lardan metric çeker
- PromQL: güçlü query dili, alerting kuralları yazabilir
- Local storage: TSDB on-disk; uzun saklama için Mimir/Thanos/Cortex
- Exemplar'lar: metric noktasından trace_id'ye link verebilir

20.1.1 OTel Metric'leri → Prometheus

İki yol var (Bölüm 11'de gördük): pull (Prometheus collector'ı scrape'ler) veya push (Mimir remote-write).

20.1.2 PromQL Örnekleri

```
# Argela projesi için tipik PromQL'ler

# Alarm geliş hızı (servise göre)
rate(alarms_received_total[5m])

# Severity dağılımı (son 1 saat)
sum by (severity) (
  increase(alarms_processed_total[1h])
)

# p99 processing duration
histogram_quantile(0.99,
  sum by (le) (rate(alarm_processing_duration_bucket[5m]))
)

# Pending alarm gauge
alarms_pending_count

# Hata oranı
sum(rate(http_server_requests_seconds_count{outcome="SERVER_ERROR"}[5m]))
/
sum(rate(http_server_requests_seconds_count[5m]))
```

20.1.3 Exemplar'lar — Metric ile Trace Köprüsü

Exemplar, metric histogram'ında 'tipik trace örneği' olarak işaretlenmiş bir nokta. Grafana metric grafiğinde tıklarsanız ilgili trace'e gidirsiniz.

```
# OpenTelemetry SDK exemplar'ı otomatik üretir; Prometheus exposition format:
http_server_requests_seconds_bucket{le="0.1"} 1234 # {trace_id="4bf92f3577..." } 0.087
1716461663142
http_server_requests_seconds_bucket{le="0.5"} 4321 # {trace_id="abcdef..." } 0.342
1716461663800
```

Prometheus 2.43+ exemplar storage'ı destekler. Grafana Cloud, Mimir 2.0+ aynı şekilde. Exemplar'lar açıkken Grafana'da metric grafiğinde küçük noktalar belirir; tıklıldığında Tempo/Jaeger panel'ine atlatır.

20.2 Grafana — Tek Pane of Glass

Grafana, üç sinyali tek arayüzde birleştirir. Data source'lar:

- Prometheus / Mimir — metric
- Loki — log
- Tempo / Jaeger — trace
- Pyroscope — profile

20.2.1 Trace ID ile Loki'den Log'a Atlama

Grafana'da bir trace'i incelerken span attribute'larında 'Logs for this span' butonu görünür. Bu, Loki'de o trace_id'yi içeren log'ları otomatik filtreler. Bunun çalışması için:

- Servisten gelen log'larda trace_id label/field olarak var olmalı
- Tempo data source'unda 'Trace to logs' konfigi yapılmış olmalı

```
# Tempo data source – trace to logs config
{
  "tracesToLogsV2": {
    "datasourceUid": "loki",
    "tags": [
      { "key": "service.name", "value": "service_name" }
    ],
    "filterBySpanID": false,
    "filterByTraceID": true,
    "spanStartTimeShift": "-30s",
    "spanEndTimeShift": "30s"
  }
}
```

20.3 Loki — Log Aggregation

Loki, Prometheus'un log eşdeğeri. Loglar 'label' tabanlı index'lenir; içerik raw kalır. Storage object store'da (S3, GCS); çok ucuz.

Loki ile OTel log'u entegrasyonu:

- OTel Collector'da Loki exporter (deneysel) — direkt push
- Daha olgun: Loki için Promtail/Alloy agent — file/journal scraping
- Spring Boot servislerinde Logback JSON encoder + Promtail = en yaygın

20.3.1 Spring Boot Logback Konfigi (Argela için ideal)

```
<!-- logback-spring.xml -->
<configuration>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <providers>
        <timestamp/>
        <version/>
      </providers>
    </encoder>
  </appender>
</configuration>
```

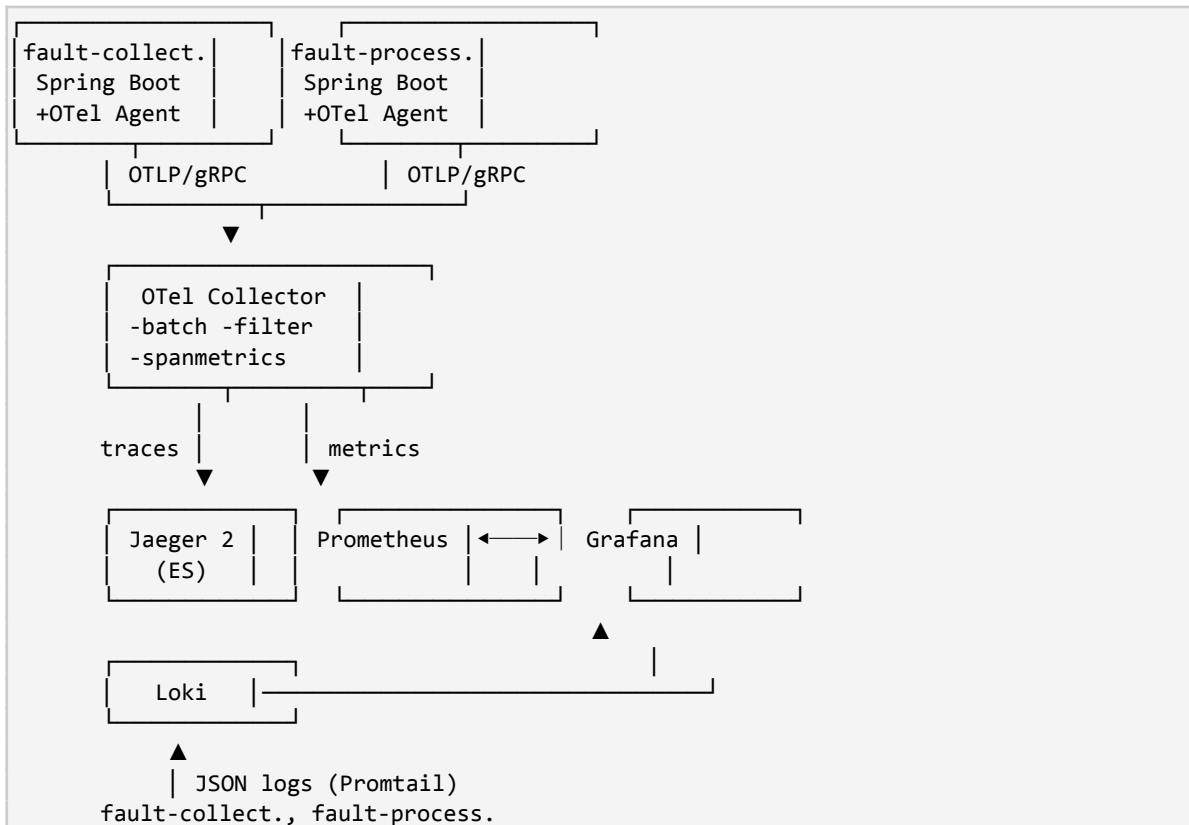
```

<logLevel/>
<loggerName/>
<mdc/>
<pattern>
  <pattern>
    {
      "service": "${spring.application.name:-}",
      "trace_id": "%X{trace_id:-}",
      "span_id": "%X{span_id:-}",
      "thread": "%thread"
    }
  </pattern>
</pattern>
<message/>
<stackTrace/>
</providers>
</encoder>
</appender>
<root level="INFO">
  <appender-ref ref="STDOUT"/>
</root>
</configuration>

```

OTel Java agent, Logback'a otomatik olarak

20.4 Tam Stack: Argela Önerilen Setup



20.5 Bölüm 20 — Tavsiyeler

PROD TAVSİYESİ

Grafana'da Tempo/Jaeger + Loki + Prometheus data source'larını birbirine bağlayın (trace↔log↔metric).

Exemplar'ları aktif edin: metric anomalisinden trace'e bir tıkla geçiş.

Log'larda trace_id label'ı zorunlu: olmazsa correlation sıfır.

Loki için Promtail agent + Alloy: collector ile çakıştırmayın; tek bir log path olsun.

ANTI-PATTERN

Aynı log'u hem Promtail hem OTel Collector ile toplamak: çift indeks, maliyet 2x.

Grafana data source isimlendirmesini standartlaştırmamak: dashboard'lar import edilemez.

Trace UI'ı Jaeger'ın native UI'ı + Grafana'da Tempo data source ile çift kullanmak: kullanıcı kafa karışıklığı.

Bölüm 21 — Sampling: Head, Tail, Adaptive ve Bütünleşik Stratejiler

Distributed tracing'in en sancılı pratik kararı sampling'dir. 'Her trace'i sakla' demek prod'da maliyet ve performans patlamasıdır. 'Sadece %1 sakla' demek ise sorun anında elinde veri olmaması demektir. Olgun ekipler bu iki uca da gitmez; akıllı sampling stratejileri tasarlar. Bu bölüm sampling'in matematiksel temeli, türleri ve gerçek dünya tradeoff'larını işler.

21.1 Sampling Neden Gerekli?

Sampling olmadan üç sorun yaşarsınız:

- 18.
- 19.
- 20.

21.2 Head Sampling — Karar Erken Verilir

Head sampling, trace'in BAŞINDA (root span üretildiğinde) sample edip etmeme kararının verilmesidir. Karar trace boyunca tüm child span'larda korunur. Trace context'teki

21.2.1 AlwaysOn / AlwaysOff

Karar trivial: ya hep al, ya hep alma. Demo veya test için.

21.2.2 TraceIdRatioBased (Probabilistic)

Default ve en yaygın strateji. Trace ID'nin matematiksel olarak deterministic bir property'sine bakarak (örn.

```
# OTEL SDK env
OTEL_TRACES_SAMPLER=traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1 # %10
```

Önemli özellik: aynı trace_id farklı servislerde aynı sonucu üretir. Yani trace cross-service'te tutarlı sample/drop kararı alır. Tüm servislerin aynı ratio kullanması gerekmez; ama tüm servisler ParentBased ile parent kararına saygı duymalıdır.

21.2.3 ParentBased Sampler

Parent span sample edildiyse, child da edilir. Parent yoksa (root span), inner sampler'a düşer. Bu,

```
# Default önerilen Java SDK ayarı
OTEL_TRACES_SAMPLER=parentbased_traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1

# İçeride:
# - parent.sampled=true → take (root karar verirken zaten alındı)
# - parent.sampled=false → drop
# - root (parent yok) → traceIdRatioBased(0.1) ile karar
```

21.2.4 Rule-Based Head Sampling

Bazı servisler önemli (örn. checkout) %100 sample, diğerleri %1. OpenTelemetry SDK'sının kendi içinde 'rule-based' default sampler yok; Jaeger native remote sampler veya custom Sampler implementasyonu gerekir.

```
# Jaeger remote sampling (Bölüm 17'de gördük)
OTEL_TRACES_SAMPLER=jaeger_remote
OTEL_TRACES_SAMPLER_ARG=endpoint=http://jaeger-collector:5778,pollingIntervalMs=30000
```

21.3 Head Sampling'in Problemi

Head sampling karar trace'in başında alındığı için 'ilginç olmayacağına dair karar verilmiş' bir trace, sonradan hata patlayınca elinizde verisi olmaz. Tipik senaryo:

- Trace başlar, sampler 'drop' der.
- Trace içinde DB sorgusu 30sn sürer.
- Trace error ile biter.
- Drop'lu olduğu için backend'de yok; debug yapamazsınız.

Bu sorunun çözümü

21.4 Tail Sampling — Karar Trace Bittikten Sonra

Tail sampling, trace'in tamamlandıktan sonra (veya yeterli süre geçtikten sonra) tüm span'ların incelenip karar verilmesidir. Avantaj: 'Her hatalı trace'i tut, başarılı trace'lerin %1'ini tut' gibi gelişmiş politikalar.

21.4.1 Tail Sampling Pipeline'i

Tail sampling collector tarafında

21. Gelen span'lar trace_id'ye göre buffer'da gruplandırılır.
22. Bir 'decision_wait' süresi (typically 10-30sn) beklenir.
23. Süre dolunca veya trace'in 'bittiği' sinyali gelirse, kurallar trace'in tamamına uygulanır.
24. Karar 'sample' veya 'drop'tur; ardından span'lar exporter'a giderlerle veya silinirler.

21.4.2 Tail Sampling Policy Tipleri

Policy	Anlam
status_code	Trace'in herhangi bir span'ı ERROR ise sample
latency	Trace toplam süresi X ms üzerinde ise sample
probabilistic	Trace'in random %N'i sample
rate_limiting	Saniyede max N trace sample (rate limiter)
string_attribute	Belirli attribute belirli değerde ise
numeric_attribute	Sayısal attribute aralığa girerse
always_sample	Bütün trace'ler
composite	AND/OR ile birden çok policy birleştir
and (compound)	Birden çok policy hep birden eşleşirse

21.4.3 Production'da Tipik Tail Sampling Konfigi

```
processors:
  tail_sampling:
    decision_wait: 20s
    num_traces: 100000
    expected_new_traces_per_sec: 200
    policies:
      # 1. Bütün hatalı trace'leri al
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }

      # 2. Yavaş trace'leri al
      - name: slow_traces
        type: latency
        latency: { threshold_ms: 1000 }

      # 3. Critical endpoint'leri %100 al
      - name: critical_endpoints
        type: string_attribute
        string_attribute:
          key: http.route
          values:
            - /api/v1/checkout
            - /api/v1/payment
            - /api/v1/auth/login

      # 4. Customer X için tüm trace'leri al (debug)
      - name: tenant_debug
        type: string_attribute
        string_attribute:
          key: tenant.id
          values: [customer-x]

      # 5. Geriye kalan trafikten %1 baseline
      - name: baseline
        type: probabilistic
        probabilistic: { sampling_percentage: 1 }
```

21.4.4 Tail Sampling'in Maliyeti

PERFORMANS ETKİSİ

Tail sampling tüm trace'i bellekte tutar — collector RAM kullanımı 10x artabilir.

Tipik hesap: $200 \text{ trace/sn} \times 30 \text{sn decision_wait} \times 8 \text{ span/trace} \times 5 \text{KB/span} \approx 250 \text{MB}$ sürekli bellek.

Yüksek RPS'te (5k+) shard'lı gateway zorunlu (Bölüm 15.5).

Single collector + tail sampling = OOMKill kaçınılmaz.

21.5 Adaptive Sampling

Statik bir oran yerine, sistem yüküne göre dinamik olarak ayarlanan sampling. Jaeger'ın adaptive sampler'ı, operation başına RPS'i ölçer ve target QPS'e ulaşacak şekilde oranı ayarlar.

```
# Jaeger adaptive sampler
{
  "default_strategy": {
    "type": "ratelimiting",
```



```

    "param": 100      // hedef: saniyede 100 trace
  }
}

```

21.6 Sampling Stratejisi Tasarımı — Karar Akış Şeması

```

Servis çok düşük RPS (< 10/sn)?
├─ EVET → AlwaysOn (her trace'i al)
├─ HAYIR
│   └─ Çok yüksek RPS (> 5000/sn)?
│       ├── EVET → Tail sampling + sharded gateway
│       └─ HAYIR
│           └─ Hata oranı önemli mi? Critical endpoint'ler var mı?
│               ├── EVET → Tail sampling (errors + latency + critical)
│               └─ HAYIR → ParentBased + TraceIdRatio %10 (head)

```

21.7 Bölüm 21 — Tavsiyeler ve Yaygın Hatalar

PROD TAVSİYESİ

Yeni servis prod'a alırken: parentbased_traceidratio %10 ile başlayın; ölçüp ayarlayın.

Critical akışlar (ödeme, auth) %100 sample edilsin (tail sampling rule).

Tail sampling'i ancak monitoring + RAM headroom + sharded gateway varsa devreye alın.

Sampling oranını /actuator/info gibi bir endpoint'ten görünür yapın; ekipler 'ben sample edildim mi?' diye sormasın.

ANTI-PATTERN

Head sampling %1 + hata anında 'trace yok' panik anlık değişiklik = sistemi kararsızlaştırır.

Servisler farklı parent-based stratejisi kullanır: trace zinciri yarıda kopar.

Tail sampling tek collector ile: OOM + yanlış sampling decision.

Sampler kararını 'her trace'i al' + filter ile sonra at: BSP overflow'a kadar veri kaybı yaşarsınız.

Bölüm 22 — JVM Üzerinde OpenTelemetry'nin Performans Maliyeti

Tracing 'free' değildir. CPU cycle, memory allocation, GC pressure ve network IO yaratır. Bu bölüm Java agent + manual instrumentation'ın gerçek prod overhead'ini ölçüsel olarak işler ve nasıl optimize edileceğini gösterir.

22.1 Tipik Overhead Rakamları

Üretici-bağımsız benchmark'lara göre OpenTelemetry Java agent'ın overhead profili:

Metrik	Tipik Etki	Notlar
Throughput	%2-5 düşüş	Auto-instrument + BSP
p50 latency	+1-3ms	Span yaratma maliyeti
p99 latency	+5-15ms	BSP export çakışması, GC
RAM (heap)	+50-200MB	Agent + BSP buffer
CPU	+5-10%	Allocation + serialization
Startup süresi	+500ms ile +3sn	Bytecode transform

Bu rakamlar 'tipik'; gerçek değer servisin yapısına göre değişir. Çok 'instrumented' kütüphane kullanan servisler (JDBC + Redis + Kafka + gRPC karışık) daha yüksek overhead görür.

22.2 Memory Allocation

Her span üretiminde Java SDK kabaca:

- 1 Span object (SdkSpan)
- 1 AttributesMap (varsayılan 16 entry)
- Eğer event varsa: List<EventData>
- Eğer link varsa: List<LinkData>
- BSP queue entry

Toplamda tek span başına ~1-2KB allocation. Saniyede 1000 span = ~2MB/sn allocation rate. Bu G1GC'de young generation'ı zorlar ama büyük problem değildir; ZGC veya Shenandoah'ta daha az gözle görülür.

22.3 GC Pressure ve Tuning

PROD TAVSİYESİ

OpenTelemetry overhead'i en çok young GC sıklığını artırır.

G1GC default'ta yeterli; -XX:+UseG1GC -XX:MaxGCPauseMillis=100

Yüksek throughput servislerde ZGC/Shenandoah avantajlıdır (allocation-pressure'a daha toleranslı).

JFR/async-profiler ile allocation hotspot'larını ölçün; gereksiz span'ları kapatın.

22.4 CPU Overhead Detayı

CPU cycle'ları üç ana yere harcanır:

25.

26.

27.

22.5 Hot Path Önemli Mi?

Span yaratımının kendisi hızlıdır (~500-1000ns) ama 'hot path' (saniyede 100k+ çağrılan metod) için bile fark eder. Strateji:

- Hot path için @WithSpan KULLANMAYIN; sadece çağırılan metoda koyun.
- Attribute eklemeyi minimize edin (max 5-10 critical attribute).
- String concatenation'dan kaçınin;
- Loop içinde span açmayın; loop dışında parent span + her iteration'da event.

```
// YANLIŞ – döngü içinde span = 1M span/saniye = exporter patlar
items.forEach(item -> {
    Span s = tracer.spanBuilder("process-item").startSpan();
    process(item);
    s.end();
});

// DOĞRU – döngü dışı span + event'ler
Span batch = tracer.spanBuilder("process-batch")
    .setAttribute("batch.size", items.size())
    .startSpan();
try (Scope sc = batch.makeCurrent()) {
    items.forEach(item -> {
        try { process(item); }
        catch (Exception e) {
            batch.addEvent("item-failed",
                Attributes.of(AttributeKey.stringKey("item.id"), item.id()));
        }
    });
} finally { batch.end(); }
```

22.6 Virtual Thread'lerin Overhead'e Etkisi

Java 21 virtual thread'leri ile her request'i ayrı bir VT'ye veren mimari yaygınlaşıyor. OTEL agent'ın etkisi:

- Span yaratma maliyeti aynı (~500ns)
- Context propagation hassasiyeti artar (VT'ler arası yapılan switch yüksek)
- BSP queue contention azalır (multiple VT'den enqueue daha ucuz)
- CPU profillemeye zorlaşır; eBPF tabanlı profil daha doğru sonuç verir

22.7 Bölüm 22 — Tavsiyeler

PROD TAVSİYESİ

Tipik servis için overhead %5'in altında olmalı; aşıyorsa instrumentation incelemesi şart.

Container memory limit'ini +20% artırın (agent ek heap için).

JVM args: -XX:+UseG1GC -XX:MaxGCPauseMillis=100 -XX:+ParallelRefProcEnabled

Production sample rate başlangıç %10 + ölçüm + tail sampling = optimal.

Bölüm 23 — Cardinality, Metric Explosion ve Cost Bombs

Metric tarafında en korkulan kelime: cardinality. Bir metric'in label'larının olası kombinasyonu cardinality'sidir. Yüksek cardinality, Prometheus/Mimir/Datadog'u tek başına çökertebilir. Bu bölüm cardinality matematiğini, tehlike işaretlerini ve önleme yöntemlerini işler.

23.1 Cardinality Tanımı

Diyeelim metric'iniz şu:

method:	GET, POST, PUT, DELETE, ...	≈ 8
status_code:	200, 201, 400, 401, 403, 500, ...	≈ 20
endpoint:	/users, /products, /checkout, ...	≈ 50
Toplam:	$8 \times 20 \times 50 = 8,000$ time series	

Bu makul bir cardinality'dir. Sorun, label'lardan birine

```
http_requests_total{method, status_code, endpoint, user_id}
user_id: 10,000 farklı kullanıcı

10,000 × 8,000 = 80,000,000 time series!
```

Prometheus, time series count ile lineer olarak memory ve CPU kullanır. 80M time series prod-grade bir Prometheus instance'ı tek başına dize çevirir.

23.2 Cardinality Patlaması Nereden Gelir?

Risk Kaynağı	Tehlike Seviyesi	Çözüm
URL path raw (/users/12345)	Çok yüksek	Route template (/users/:id)
user_id, session_id, request_id	Çok yüksek	Label OLMA — sadece span attribute
IP address	Yüksek	CIDR'a normalize
timestamp	Felaket	Label OLMA
error message text	Yüksek	error_type label'ı (sınırlı set)
pod name	Orta	Sadece ihtiyaç varsa
build_sha / version	Düşük	OK ama her commit yeni series

23.3 Cardinality Audit — Prod'da Erken Uyarı

```
# Prometheus'ta toplam series sayısı
prometheus_tsdb_head_series

# Metric başına series count (en yüksek 20)
topk(20, count by (__name__){__name__=~".+"}))

# Belirli metric için label cardinality
count(count by (label_X) (metric_name))

# Belirli label'ın değer sayısı
group by (label_X) (metric_name)
```

Bu sorguları haftalık/günlük çalıştırıp cardinality artışını takip edin. Aniden 100k series eklenmesi muhtemelen bir geliştirici yanlış label koymuş demektir.

23.4 Cardinality Limitleri — Defansif Yapılandırma

Prometheus 2.43+

```
# Mimir tenant limits
limits:
  max_global_series_per_user: 1500000
  max_label_names_per_series: 30
  max_label_value_length: 2048
  max_metadata_length: 1024
```

23.5 OTEL Tarafında Cardinality Kontrolü

Cardinality bomb'unu collector tarafında defuse etmek mümkün.

```
processors:
  transform/cardinality:
    metric_statements:
      - context: datapoint
        statements:
          # URL path'i route template'ine indir
          - replace_pattern(attributes["http.target"],
            "/users/\\d+", "/users/:id")
          - replace_pattern(attributes["http.target"],
            "/orders/[a-f0-9-]+", "/orders/:uuid")

          # IP'yi CIDR'a indir
          - replace_pattern(attributes["client.address"],
            "(\\d+\\.\\d+\\.\\d+\\.\\d+)", "$1.0.0/16")

          # user_id sil (eğer yanlışlıkla geldiyse)
          - delete_key(attributes, "user.id")
```

23.6 Bölüm 23 — Anti-Pattern'lar

ANTI-PATTERN

user_id, session_id, request_id'yi metric label yapmak: cardinality patlar.

Raw URL path'i metric label yapmak: her path varyantı yeni series.

Cardinality limit'i set etmemek: production'da fark ettiğinizde geç olur.

Span attribute'unu doğrudan metric'e çevirip cardinality görmemezlikten gelmek (spanmetrics processor).

Bölüm 24 — Capacity Planning ve Maliyet Optimizasyonu

Observability platformu maliyetli olabilir. Bir Datadog veya New Relic faturası, yıllık 6 haneli rakamlara ulaşabilir. Self-hosted çözüm bile altyapı, mühendis efor, storage'ta hatırı sayılır maliyet doğurur. Bu bölüm capacity planning matematiğini ve maliyet kontrol stratejilerini işler.

24.1 Veri Hacmi Tahmini

Bir tracing platformunun ihtiyacını planlamak için temel formül:

```
Günlük span hacmi (sampled) =
  RPS × span_per_request × sampling_rate × 86400

Tipik prod servis:
  500 RPS × 8 span/request × 0.1 sampling × 86400 = 34,560,000 span/gün
  ≈ 35M span/gün

Span boyutu:
  Avg span (10 attribute) ≈ 2-3 KB raw
  ES indexed (+_source + indices) ≈ 5-8 KB

Storage:
  35M × 6 KB = 210 GB/gün raw
  ES'te (1 replica): 420 GB/gün
  ClickHouse'ta (10x compression): 21 GB/gün
```

Bu rakamı 7-14 gün retention ile çarparak total storage hesabı çıkar.

24.2 Maliyet Düşürme Stratejileri

24.2.1 Sampling Doğru Ayarlanması

Sampling rate'i %20'den %5'e düşürmek storage'ı 4x düşürür. Ama tail sampling ile akıllı sampling, sabit head sampling'den her zaman üstündür: hatalı %100, normal %1 = total ~%2 sample, ama tüm hatalar elinizde.

24.2.2 Storage Tier'ları

Hot/Warm/Cold tier ayrımı maliyeti 5-10x düşürür. Son 3 günü SSD, sonrası HDD, 30+ gün object store snapshot.

24.2.3 Backend Seçimi

Backend	TB başına aylık maliyet (kabaca)
Datadog APM	\$5,000-15,000
New Relic	\$2,000-8,000
Elasticsearch (self-host AWS)	\$500-2,000
Tempo + S3	\$50-200
ClickHouse (self-host)	\$100-400

24.2.4 Attribute Trimming

Collector'da

24.2.5 Compression

OTLP gzip compression 3-5x, ClickHouse storage compression 10-20x. Sıkıştırılmamış span göndermek bandwidth boşa harcamaktır.

24.3 Operasyonel Maliyetler

Storage haricinde gizli maliyetler:

- Collector instance'ları (CPU + RAM)
- Network egress (cross-AZ, cross-region pahalı)
- Mühendis efor (config, troubleshooting, sampling tuning)
- Eğitim ve onboarding
- On-call rotation impact

24.4 Cost Budget Yaklaşımı

Mature ekipler observability'ye 'cost budget' uygular. Tipik formül:

- Toplam infrastructure budget'ın %3-7'si observability'ye
- Bu budget'ın %50'si metric, %30'u trace, %20'si log
- Aylık review: hangi servis aşırı tüketiyor? Hangi metric label kontrol dışı?

24.5 Bölüm 24 — Tavsiyeler

PROD TAVSİYESİ

Her servis için sampling rate ve metric cardinality dashboard'u kurun.

Aylık cost review'lerinde 'top 10 most expensive service' grafiği bulunsun.

Vendor değişimi 12-18 aylık ROI hesabı yapın; OTLP standartlaştırıldığı için 'lock-in' artık zayıf bir excuse.

Storage tier'larını otomatize edin; manuel index management = unutulur, disk dolar.

ANTI-PATTERN

'Her trace'i 90 gün tutalım' - storage 10x daha pahalı, %95 trace hiç bakılmaz.

Tüm metric'ler 1sn scrape interval - %95 metric için 15sn yeterli.

Cross-region telemetry push - egress maliyeti backend'i geçer.

Backend down olunca data loss yerine queue limitsiz - OOM, kontrol kaybı.

Bölüm 25 — Docker Compose ile Tam Observability Stack

Docker Compose, küçük ekipler ve geliştirme ortamları için observability stack'i ayağa kaldırmanın en hızlı yoludur. Argela demo projeniz zaten bu modeli kullanıyor. Bu bölüm prod-grade düşünülmüş, tam donanımlı bir Compose dosyasını adım adım açıklar; sonra production'a uyarlama notları verir.

25.1 Compose'un Sınırları

Docker Compose ile prod yapmak teknik olarak mümkündür, ama önerilmez. Sınırları:

- Tek host'a bağımlı; HA için orchestration gerek (Swarm/K8s)
- Rolling update zayıf; service restart = downtime
- Resource limit'leri kabaca; advanced scheduling yok
- Network policy, RBAC, secret rotation sınırlı
- Persistent volume management el ile

Compose'un asıl yeri: geliştirme, staging, demo, küçük internal tool'lar. Argela 'Fault Management' staj projesi tam bu kategoride.

25.2 Tam docker-compose.yml — Production-Inspired

```
version: "3.9"

x-logging: &default-logging
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"
    tag: "{{.Name}}/{{.ID}}"

networks:
  observability:
    driver: bridge
  fm-internal:
    driver: bridge

volumes:
  postgres-data:
  prometheus-data:
  grafana-data:
  es-data:
  jaeger-tmp:

services:
  # =====
  # ALTYAPI
  # =====
  postgres:
    image: postgres:16-alpine
    container_name: fm-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: argela_fm
      POSTGRES_USER: fm_user
      POSTGRES_PASSWORD_FILE: /run/secrets/pg_password
    volumes:
```

```

    - postgres-data:/var/lib/postgresql/data
secrets:
  - pg_password
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U fm_user -d argela_fm"]
  interval: 10s
  retries: 5
  start_period: 30s
networks: [fm-internal]
logging: *default-logging

# =====
# ELASTICSEARCH (Jaeger storage)
# =====
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.13.4
  container_name: fm-es
  restart: unless-stopped
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - "ES_JAVA_OPTS=-Xms2g -Xmx2g"
    - bootstrap.memory_lock=true
  ulimits:
    memlock: { soft: -1, hard: -1 }
    nofile: { soft: 65536, hard: 65536 }
  volumes:
    - es-data:/usr/share/elasticsearch/data
  healthcheck:
    test: ["CMD-SHELL", "curl -fs http://localhost:9200/_cluster/health"]
    interval: 15s
    retries: 10
    start_period: 60s
  networks: [observability]
  logging: *default-logging

# =====
# OTel COLLECTOR
# =====
otel-collector:
  image: otel/opentelemetry-collector-contrib:0.108.0
  container_name: fm-otel-collector
  restart: unless-stopped
  command: ["--config=/etc/otelcol/config.yaml"]
  volumes:
    - ./infrastructure/otel/collector-config.yaml:/etc/otelcol/config.yaml:ro
  ports:
    - "4317:4317" # OTLP gRPC
    - "4318:4318" # OTLP HTTP
    - "8888:8888" # collector self-metrics
    - "8889:8889" # prometheus exporter
    - "13133:13133" # health
    - "55679:55679" # zpages
  depends_on:
    jaeger:
      condition: service_healthy
  networks: [observability, fm-internal]
  deploy:
    resources:
      limits:
        memory: 1024M

```

```

    reservations:
      memory: 256M
    logging: *default-logging

# =====
# JAEGER v2 (OTLP native)
# =====
jaeger:
  image: jaegertracing/jaeger:2.0.0
  container_name: fm-jaeger
  restart: unless-stopped
  environment:
    - SPAN_STORAGE_TYPE=elasticsearch
    - ES_SERVER_URLS=http://elasticsearch:9200
    - ES_TAGS_AS_FIELDS_ALL=false
    - ES_NUM_SHARDS=3
    - ES_NUM_REPLICAS=0
  ports:
    - "16686:16686" # UI
  depends_on:
    elasticsearch:
      condition: service_healthy
  healthcheck:
    test: ["CMD", "wget", "-qO-", "http://localhost:14269/"]
    interval: 15s
    retries: 5
  networks: [observability]
  logging: *default-logging

# =====
# PROMETHEUS
# =====
prometheus:
  image: prom/prometheus:v2.54.1
  container_name: fm-prometheus
  restart: unless-stopped
  command:
    - --config.file=/etc/prometheus/prometheus.yml
    - --storage.tsdb.retention.time=15d
    - --storage.tsdb.retention.size=10GB
    - --web.enable-lifecycle
    - --enable-feature=exemplar-storage
  volumes:
    - ./infrastructure/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus-data:/prometheus
  ports:
    - "9090:9090"
  networks: [observability]
  logging: *default-logging

# =====
# GRAFANA
# =====
grafana:
  image: grafana/grafana:11.2.0
  container_name: fm-grafana
  restart: unless-stopped
  environment:
    - GF_SECURITY_ADMIN_PASSWORD__FILE=/run/secrets/grafana_admin_pw
    - GF_USERS_ALLOW_SIGN_UP=false
    - GF_INSTALL_PLUGINS=grafana-piechart-panel

```

```

secrets:
  - grafana_admin_pw
volumes:
  - grafana-data:/var/lib/grafana
  - ./infrastructure/grafana/provisioning:/etc/grafana/provisioning:ro
  - ./infrastructure/grafana/dashboards:/var/lib/grafana/dashboards:ro
ports:
  - "3000:3000"
depends_on: [prometheus, jaeger]
networks: [observability]
logging: *default-logging

# =====
# UYGULAMA SERVISLERI
# =====
fault-collector:
  build: ./fault-collector
  container_name: fault-collector
  restart: unless-stopped
  environment:
    OTEL_SERVICE_NAME: fault-collector
    OTEL_RESOURCE_ATTRIBUTES:
"service.namespace=argela-fm,deployment.environment=local"
    OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4317
    OTEL_EXPORTER_OTLP_PROTOCOL: grpc
    OTEL_TRACES_SAMPLER: parentbased_traceidratio
    OTEL_TRACES_SAMPLER_ARG: "1.0" # demo: hep al
    OTEL_BSP_MAX_QUEUE_SIZE: "4096"
    FAULT_PROCESSOR_URL: http://fault-processor:8081
    JAVA_TOOL_OPTIONS: "-javaagent:/app/otel-agent.jar"
  ports: ["8080:8080"]
  depends_on:
    otel-collector: { condition: service_started }
    fault-processor: { condition: service_healthy }
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health || exit 1"]
    interval: 15s
    retries: 5
    start_period: 40s
  networks: [observability, fm-internal]
  logging: *default-logging

fault-processor:
  build: ./fault-processor
  container_name: fault-processor
  restart: unless-stopped
  environment:
    OTEL_SERVICE_NAME: fault-processor
    OTEL_RESOURCE_ATTRIBUTES:
"service.namespace=argela-fm,deployment.environment=local"
    OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4317
    OTEL_EXPORTER_OTLP_PROTOCOL: grpc
    OTEL_TRACES_SAMPLER: parentbased_traceidratio
    OTEL_TRACES_SAMPLER_ARG: "1.0"
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/argela_fm
    SPRING_DATASOURCE_USERNAME: fm_user
    SPRING_DATASOURCE_PASSWORD_FILE: /run/secrets/pg_password
    JAVA_TOOL_OPTIONS: "-javaagent:/app/otel-agent.jar"
  ports: ["8081:8081"]
  depends_on:
    postgres: { condition: service_healthy }

```

```

    otel-collector: { condition: service_started }
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:8081/actuator/health || exit 1"]
    interval: 15s
    retries: 5
    start_period: 40s
  secrets: [pg_password]
  networks: [observability, fm-internal]
  logging: *default-logging

secrets:
  pg_password:
    file: ./secrets/pg_password.txt
  grafana_admin_pw:
    file: ./secrets/grafana_admin_pw.txt

```

25.3 Sık Karşılaşılan Compose Sorunları

DEBUGGING

'depends_on' default'ta sadece container start sırasını garanti eder, healthy bekleme YAPMAZ. condition: service_healthy şart.

Tek bridge network'te tüm servisler birbirini görür ama prod-benzeri network izolasyonu yok. fm-internal + observability ayrımı doğru pattern.

Volume tanımlamadan ES çalıştırırsan: container restart'ta tüm trace'ler silinir.

ES için ulimits memlock şart; aksi halde bootstrap.memory_lock=true fail eder ve ES start olmaz.

25.4 Compose'tan Production'a Geçiş Checklist'i

- Plaintext password kaldırıldı, Docker Secrets veya Vault kullanılıyor mu?
- ES single-node mode prod değil; en az 3 node cluster gerekli
- Volume'lar host disk değil; NFS/EBS/PVC olmalı
- Servisler resource limit'lerine sahip mi? (cpu/memory)
- Log driver json-file değil; cluster log aggregator'a forward
- TLS kapalı bırakılmış mı? Cluster-internal'da bile gerekli olabilir
- Healthcheck'ler doğru endpoint'leri probe ediyor mu?
- Image tag 'latest' değil, semver pinleme yapılmış mı?

Bölüm 26 — Kubernetes Üzerinde OpenTelemetry

Kubernetes, modern observability stack'inin doğal habitatıdır. Bu bölüm K8s üzerinde OTel collector dağıtım pattern'larını, OpenTelemetry Operator'ı, sidecar injection'ı ve K8s'a özgü problemleri işler.

26.1 K8s'ta Üç Deployment Pattern'ı

Pattern	K8s Workload	Tipik Kullanım
Sidecar Collector	Pod'da extra container	Tek servis için izole, küçük yük
DaemonSet Agent	Her node'da 1 pod	Node-level toplama, k8s metadata
Gateway Deployment	Replicated Deployment + Service	Merkezi pipeline, tail sampling
StatefulSet Gateway	Stateful set + headless svc	Sharded tail sampling, consistent hash

26.2 OpenTelemetry Operator

OpenTelemetry Operator, K8s'ta OTel kaynaklarını yönetmek için en olgun yoldur. CRD'leri (Custom Resource Definitions) ile gelir:

- `OpenTelemetryCollector` — Collector deployment'ını declarative tanımlar
- `Instrumentation` — Auto-instrumentation injection için
- `OpAMPBridge` — Remote configuration için

```
# Operator kurulumu (cert-manager prerequisite)
kubectl apply -f
https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.yaml
kubectl apply -f
https://github.com/open-telemetry/opentelemetry-operator/releases/latest/download/opentelemetry-operator.yaml
```

26.3 OpenTelemetryCollector CRD Örneği

```
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata:
  name: gateway
  namespace: observability
spec:
  mode: deployment # daemonset | sidecar | statefulset | deployment
  image: otel/opentelemetry-collector-contrib:0.108.0
  replicas: 3
  resources:
    limits:
      memory: 2Gi
      cpu: 1000m
    requests:
      memory: 512Mi
      cpu: 200m
  envFrom:
    - configMapRef:
        name: otelcol-env
  config: |
    receivers:
```

```

otlp:
  protocols:
    grpc: { endpoint: 0.0.0.0:4317 }
    http: { endpoint: 0.0.0.0:4318 }
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 1500
    spike_limit_mib: 250
  k8sattributes:
    auth_type: serviceAccount
    passthrough: false
    extract:
      metadata:
        - k8s.namespace.name
        - k8s.pod.name
        - k8s.pod.uid
        - k8s.deployment.name
        - k8s.node.name
      pod_association:
        - sources:
            - from: resource_attribute
              name: k8s.pod.ip
  batch:
    timeout: 2s
    send_batch_size: 1024
exporters:
  otlp/jaeger:
    endpoint: jaeger-collector.observability.svc.cluster.local:4317
    tls: { insecure: true }
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, k8sattributes, batch]
      exporters: [otlp/jaeger]
telemetry:
  metrics:
    address: 0.0.0.0:8888

```

26.4 Auto-Instrumentation Injection (Sidecar Pattern)

Operator'ın Instrumentation CRD'si ile, deployment'a annotation eklemek 'init container' enjeksiyonu ile Java agent'ı otomatik mount eder. Hiç Dockerfile değiştirmeden:

```

apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: argela-java
  namespace: argela-fm
spec:
  exporter:
    endpoint: http://gateway-collector.observability.svc:4317
  propagators: [tracecontext, baggage]
  sampler:
    type: parentbased_traceidratio
    argument: "0.1"
  java:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-java:2.10.0
    env:

```

```
- name: OTEL_JAVAAGENT_DEBUG
  value: "false"
- name: OTEL_INSTRUMENTATION_LOGBACK_APPENDER_EXPERIMENTAL_CAPTURE_MDC_ATTRIBUTES
  value: "**"
```

Sonra Deployment manifestinde annotation:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fault-collector
  namespace: argela-fm
spec:
  template:
    metadata:
      annotations:
        instrumentation.opentelemetry.io/inject-java: "argela-fm/argela-java"
```

Operator, init container ile Java agent jar'ını paylaşımlı emptyDir'e kopyalar; main container'da

26.5 K8s Networking ve OTLP

K8s'ta OTLP trafiği için dikkat noktaları:

- Service DNS:
- gRPC + ClusterIP Service çalışır. NodePort/LoadBalancer gerekirse HTTP/2 desteklediğinden emin olun.
- Istio veya Linkerd service mesh varsa: sidecar proxy gRPC'yi HTTP/1'e düşürebilir. mTLS + L7 proxy konfigürasyonu kritik.
- Network policy: collector'ı sadece application namespace'lerinden erişime aç.

26.6 Bölüm 26 — Tavsiyeler

PROD TAVSİYESİ

Production'da OpenTelemetry Operator + Instrumentation CRD pattern'ı en az manuel iş gerektirir. k8sattributes processor gateway'de ZORUNLU; bu olmadan span'larda pod/namespace bilgisi yok. Operator versiyonu ile autoinstrumentation image versiyonunu eşleştirin; uyumsuzluklar sessiz bug üretir.

Gateway collector'ı en az 3 replica + PodDisruptionBudget ile yapılandırın.

ANTI-PATTERN

Her pod'a sidecar collector koymak: 1000 pod = 1000 collector container, kaynak israfı.

Operator yokken Helm chart ile statik collector deploy: konfig drift, manuel sync zorluğu.

K8s service mesh varken auto-instrumentation: çift trace span'ı (proxy + agent) üretebilir.

Bölüm 27 — Helm Chart'lar, Kustomize ve GitOps Akışı

Production observability stack'i ürettikten sonra onu sürdürmek ayrı bir disiplin. Bu bölüm Helm, Kustomize ve GitOps (ArgoCD/Flux) yaklaşımlarının observability'e nasıl uygulandığını işler.

27.1 Resmi Helm Chart'lar

Chart	Repo	Kullanım
opentelemetry-collector	open-telemetry/opentelemetry-helm-charts	Standalone collector deploy
opentelemetry-operator	open-telemetry/opentelemetry-helm-charts	Operator + CRD'ler
jaeger	jaegertracing/helm-charts	Jaeger v1 (legacy)
jaeger-operator	jaegertracing/helm-charts	Jaeger v2 operator
kube-prometheus-stack	prometheus-community/helm-charts	Prometheus + Grafana + AlertManager
loki-stack	grafana/helm-charts	Loki + Promtail
tempo-distributed	grafana/helm-charts	HA Tempo deployment

27.2 Helm Values Örneği — Collector Gateway

```
# values.yaml – opentelemetry-collector chart için
mode: deployment
replicaCount: 3

image:
  repository: otel/opentelemetry-collector-contrib
  tag: 0.108.0

resources:
  limits: { cpu: 1, memory: 2Gi }
  requests: { cpu: 200m, memory: 512Mi }

podDisruptionBudget:
  enabled: true
  minAvailable: 2

ports:
  otlp: { enabled: true, containerPort: 4317, servicePort: 4317, protocol: TCP }
  otlp-http: { enabled: true, containerPort: 4318, servicePort: 4318, protocol: TCP }
  metrics: { enabled: true, containerPort: 8888, servicePort: 8888 }

config:
  receivers:
    otlp:
      protocols:
        grpc: { endpoint: ${env:MY_POD_IP}:4317 }
        http: { endpoint: ${env:MY_POD_IP}:4318 }
  processors:
    memory_limiter:
      check_interval: 1s
      limit_mib: 1700
```

```

k8sattributes: {}
batch:
  timeout: 2s
  send_batch_size: 1024
exporters:
  otlp/jaeger:
    endpoint: jaeger-collector:4317
    tls: { insecure: true }
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, k8sattributes, batch]
      exporters: [otlp/jaeger]

```

27.3 GitOps Pattern

Observability config'leri Git'te yaşar; ArgoCD/Flux otomatik apply eder. Bu yaklaşımın avantajları:

- Tüm değişikliklerin audit trail'i Git history
- Pull request review'i config değişiklikleri için
- Rollback tek commit revert
- Multi-cluster sync (her cluster aynı config)

```

# Klasör yapısı
gitops/
├── base/
│   ├── otel-collector/
│   │   ├── kustomization.yaml
│   │   └── collector.yaml
│   └── jaeger/
├── overlays/
│   ├── prod/
│   │   ├── kustomization.yaml    ← prod-specific patch
│   │   └── values.yaml
│   ├── staging/
│   └── dev/
├── argocd/
└── application.yaml

```

27.4 Bölüm 27 — Production Notları

PROD TAVSİYESİ

Helm chart upgrade'lerini staging'te bir hafta tut, prod'a sonra al; breaking change'leri yakala.

kustomize overlays ile dev/staging/prod farklarını yönet; values.yaml duplicate etme.

ArgoCD sync wave kullan: önce CRD, sonra operator, sonra app instances.

Helm release name'lerini ekipler arası standart tut: 'gateway' yerine 'otel-gateway-platform'.

Bölüm 28 — High Availability, Disaster Recovery ve Scaling

Observability platformu da bir prod servistir; uptime SLO'su ve DR planı olmalı. Tracing kesintisi 'sessiz' bir kesintidir — uygulamalar çalışır ama mühendisler kör. Bu bölüm HA/DR/scaling stratejilerini işler.

28.1 Collector HA

Collector'ı HA yapmak iki katmanda düşünülür:

28.1.1 Replica Count + PDB

Gateway collector'ı en az 3 replica, PodDisruptionBudget ile

28.1.2 Service-Level Load Balancing

ClusterIP service round-robin yapar; gRPC için L7 LB (Envoy, Istio) tercih edilir çünkü HTTP/2 connection pinning olabilir. Service mesh varsa onun LB algoritmasını kullanın.

28.1.3 Sharded Gateway (Tail Sampling İçin)

Tail sampling kullanıyorsanız HA + sharding birlikte gerekir.

28.2 Jaeger / Storage HA

Jaeger collector ve query stateless'tır; storage ise stateful.

Katman	HA Stratejisi
Jaeger collector	K8s Deployment + 2+ replica + service
Jaeger query	K8s Deployment + 2+ replica
Elasticsearch	3+ master node, dedicated data nodes, replica > 0
OpenSearch	Same as ES; cross-AZ deployment
Cassandra	RF=3 minimum, ConsistencyLevel=QUORUM

28.3 Disaster Recovery

Tracing data DR planı için sorulması gereken sorular:

28. Hangi süredeki trace'leri kaybetmeyi göze alabilirsin? (RPO)
29. Tracing tamamen down olursa ne kadar süre çalışmadan duracak? (RTO)
30. Birincil region down olursa ikincil'e nasıl failover?

Pratik DR stratejileri:

- Backend snapshot'ları S3'e (ES snapshot API + S3 repository)
- Cross-region replica (ES Cross-Cluster Replication)
- Tempo + S3 zaten DR-friendly (object storage replication)
- Collector data buffer: persistent queue (file_storage extension)

```
# Collector persistent queue
extensions:
  file_storage:
    directory: /var/lib/otelcol/data
    timeout: 1s

exporters:
  otlp:
    endpoint: backend:4317
    sending_queue:
      enabled: true
      storage: file_storage # backend down olunca diske yaz
      num_consumers: 4
      queue_size: 10000

service:
  extensions: [file_storage]
```

28.4 Scaling Stratejileri

28.4.1 Horizontal Pod Autoscaler

HPA + custom metric (CPU + receiver_accepted_spans_rate) ile collector otomatik scale eder.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: otel-gateway
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: otel-gateway
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
    - type: Pods
      pods:
        metric: { name: otelcol_receiver_accepted_spans_rate }
        target: { type: AverageValue, averageValue: "5000" }
```

28.4.2 Vertical Scaling

Collector tek instance'ta 50k span/sn handle edebilir (2 CPU + 4GB RAM). Bunun üstüne çıkmak zor; HPA ile horizontal scaling tercih edilir. Tail sampling pod'ları VPA ile vertical scaling daha mantıklıdır (RAM bağımlı).

28.4.3 Storage Scaling

Elasticsearch'te shard count yatay scaling birim'idir. Yeni node eklemek otomatik shard rebalance tetikler.

28.5 Capacity Planning Formülü

```
# Collector
RPS_per_collector = 5000-15000 span/sn (CPU bağımlı)
Required replicas = TotalRPS / RPS_per_collector × 1.5 (headroom)
```

```
# Memory
Memory_per_collector = limit_mib + spike_mib + 20% (Go runtime overhead)

# Tail sampling
TailSampling_RAM_GB = (RPS × decision_wait_sec × avg_span_per_trace × avg_span_size_KB) /
(1024*1024)

# Elasticsearch
Daily_span_bytes = RPS × sampling_rate × avg_span_indexed_size × 86400
ES_disk_required = Daily_span_bytes × retention_days × (1 + replica_count)
ES_node_count = ES_disk_required / disk_per_node + 1 (master node)
```

28.6 Bölüm 28 — Tavsiyeler

PROD TAVSİYESİ

Tracing platformunun SLO'su prod uygulamalardan daha az olabilir (ör. %99 yerine %99.5); ama 'down' demek 'kör' demek olduğunu unutmayın.

DR test'i ayda en az bir kez: snapshot restore + cluster failover drill.

Collector ve backend resource kullanımını ayrı dashboard'larda izleyin; capacity headroom %30+.

HPA scale-down policy'sini agresif tutmayın; trafik dalgalanması tracing kaybına yol açar.

ANTI-PATTERN

Tracing'i 'best effort' kabul edip HA atlamak: incident'ta kör kalırsınız.

Tail sampling cluster'ını single AZ'de toplamak: AZ failure → tüm trace kaybı.

ES için yalnızca 1 replica: master + 1 data node ile prod yapmak — kesinlikle değil.

Bölüm 29 — TLS, mTLS ve Authentication

Telemetri verisi 'iç' veri sayılır ama 2024 sonrası 'Zero Trust' yaklaşımı bunu da güvende tutmayı zorunlu kılıyor. Trace verisi HTTP header'lar, SQL statement'ları, attribute olarak PII içerebilir; sızdığına ciddi sonuçlar doğurur. Bu bölüm OTEL pipeline'ında güvenliği işler.

29.1 TLS — Minimum Standart

Cluster-internal trafik için bile TLS önerilir. Service mesh (Istio, Linkerd) mTLS otomatik halleder; mesh yoksa elle yapılandırma gerekir.

```
# Collector TLS receiver
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      tls:
        cert_file: /etc/certs/server.crt
        key_file: /etc/certs/server.key
        ca_file: /etc/certs/ca.crt
        min_version: "1.3"
        reload_interval: 1h      # cert rotation desteği

# Client SDK
OTEL_EXPORTER_OTLP_ENDPOINT=https://otel-gateway:4317
OTEL_EXPORTER_OTLP_CERTIFICATE=/etc/certs/ca.crt
```

29.2 mTLS — İki Yönlü Doğrulama

Collector, hangi client'ın bağlandığını bilmek istiyorsa mTLS gerekir. Client kendi cert'ini sunar; collector

```
# Server tarafı
tls:
  cert_file: /etc/certs/server.crt
  key_file: /etc/certs/server.key
  client_ca_file: /etc/certs/client-ca.crt  # client cert'leri doğrula

# Client tarafı (Java)
OTEL_EXPORTER_OTLP_CLIENT_CERTIFICATE=/etc/certs/client.crt
OTEL_EXPORTER_OTLP_CLIENT_KEY=/etc/certs/client.key
```

29.3 Authentication Extension'lar

Cert tabanlı kimliklendirme yetmediğinde collector

- `bearertokenauth` — HTTP Authorization header'da Bearer token
- `basicauth` — username/password
- `oidc` — OIDC/JWT (Keycloak, Auth0)
- `headers_setter` — egress için header enjeksiyonu
- `sigv4auth` — AWS Signature v4 (X-Ray)

```
extensions:
  bearertokenauth:
```

```
scheme: "Bearer"
tokens: ["argela-very-long-secret-token"]

receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
        auth: { authenticator: bearertokenauth }

service:
  extensions: [bearertokenauth]
```

29.4 Cert Rotation

PROD TAVSİYESİ

Cert rotation collector'da reload_interval ile otomatik (0.83+).

Cert expiry alarmı kur: ortalama 30 gün öncesinden uyarı.

cert-manager + Let's Encrypt veya internal CA ile otomatik renewal.

Compromise senaryosu: cert revocation list (CRL) veya kısa TTL (24h) cert'ler.

Bölüm 30 — PII, GDPR ve Veri Maskeleme

Telemetri verisinin GDPR/KVKK/HIPAA gibi regülasyonlardan muaf olduğunu zannetmek yaygın bir hatadır. Trace verisi `user_id`, email, IP gibi 'kişisel veri' tanımına giren bilgiler içerebilir. Bu bölüm hangi alanların PII riski taşıdığını ve hangi katmanda maskeleneceğini işler.

30.1 OTel Verisinde Yaygın PII Sızıntı Noktaları

Alan	Risk	Örnek
<code>http.request.header.authorization</code>	Çok yüksek	Bearer eyJ... — token sızar
<code>http.request.header.cookie</code>	Çok yüksek	session ID, JWT
<code>http.request.body</code>	Çok yüksek	Login body içinde password
<code>url.full / url.query</code>	Yüksek	?token=... query param
<code>db.statement</code>	Yüksek	SELECT ... WHERE email='alice@x.com'
<code>http.user_agent</code>	Düşük-Orta	Bazı device fingerprint
<code>client.address / net.peer.ip</code>	Orta	IP adresi GDPR'da kişisel veri
<code>span.event.message</code>	Yüksek	Stack trace'te email görünebilir
<code>baggage.user_id</code>	Çok yüksek	Cross-service yayılır

30.2 Maskeleme Stratejileri

30.2.1 Source-Side (SDK) Filter

SpanProcessor ile attribute'ları yazılmadan önce filtrele:

```
public class PIIRedactingProcessor implements SpanProcessor {
    private static final Set<String> SENSITIVE_KEYS = Set.of(
        "http.request.header.authorization",
        "http.request.header.cookie",
        "http.user_agent"
    );

    @Override
    public void onStart(Context parentContext, ReadWriteSpan span) {
        SENSITIVE_KEYS.forEach(k -> span.setAttribute(k, "[REDACTED]"));
    }

    @Override public void onEnd(ReadableSpan span) {}
    @Override public boolean isStartRequired() { return true; }
    @Override public boolean isEndRequired() { return false; }
    @Override public CompletableResultCode shutdown() {
        return CompletableResultCode.ofSuccess();
    }
    @Override public CompletableResultCode forceFlush() {
        return CompletableResultCode.ofSuccess();
    }
}
```

30.2.2 Collector-Side Redaction

Daha merkezi ve esnek yöntem.

```
processors:
  attributes/pii:
    actions:
      - key: http.request.header.authorization
        action: delete
      - key: http.request.header.cookie
        action: delete
      - key: user.email
        action: hash # SHA-256 hash
      - key: client.address
        action: update
        from_attribute: client.address.masked # önceden masked

  transform/redact:
    error_mode: ignore
    trace_statements:
      - context: span
        statements:
          # URL query'den hassas param'ları temizle
          - replace_pattern(attributes["url.query"],
            "(token|password|api_key)=[^&]+", "$1=[REDACTED]")
          # SQL'den string literal'ları temizle
          - replace_pattern(attributes["db.statement"],
            "'[^']*'", "'?'")
          # Email pattern'i evrensel olarak hash'le
          - replace_pattern(attributes["alarm.description"],
            "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}",
            "[EMAIL_REDACTED]")

  redaction/secrets:
    allow_all_keys: true
    blocked_values:
      - "AKIA[0-9A-Z]{16}" # AWS access key
      - "ghp_[a-zA-Z0-9]{36}" # GitHub PAT
      - "sk_live_[a-zA-Z0-9]{24,}" # Stripe
```

30.3 IP Adresi ve GDPR

GDPR ve KVKK, IP adresini 'doğrudan veya dolaylı olarak kimliklendirilebilen' veri olarak görür. Tipik strateji: IP'yi /16 veya /24 CIDR'a indirgemek.

```
# IPv4'ü /16'ya indir
- replace_pattern(attributes["client.address"],
  "^(\\d+\\.\\d+\\.\\d+\\.\\d+)$", "$1.0.0/16")

# IPv6'yı /48'e indir
- replace_pattern(attributes["client.address"],
  "^[0-9a-f]{4}:([0-9a-f]{4}):([0-9a-f]{4}):.*", "$1::/48")
```

30.4 Right to be Forgotten

GDPR Madde 17 'unutulma hakkı' der: kullanıcı isteğinde tüm verisi silinmeli. Tracing data açısından bu zor bir gereksinim; çünkü trace storage tipik olarak 'delete by query' destekler ama performans pahalıdır.

Mitigasyon: `user_id`'yi attribute olarak ASLA TUTMAYIN. Onun yerine hash'lenmiş `user_id` veya geçici `session_id` kullanın. Bu sayede 'unutma' = hash mapping'i silmek; trace storage'a dokunmak gerekmez.

30.5 Audit Log

Veri koruma compliance'i için collector erişim log'ları tutulmalı: kim hangi token ile veri gönderdi/aldı, hangi config değişti. Collector access log için

30.6 Bölüm 30 — Tavsiyeler

PROD TAVSİYESİ

PII redaction'ı collector'da merkezi yap; SDK'ya bırakırsan ekiplerin biri unutur.
Yeni attribute eklemekten önce: 'Bu PII mi?' checklist'i code review'da olsun.
Storage tarafında encryption at rest zorunlu (ES KMS, ClickHouse TDE).
Cross-region transfer GDPR uyumlu mu? EU verisi US backend'e gidiyor mu?

ANTI-PATTERN

`db.statement`'ı raw bırakmak: SQL içinde email/password literal görünür.
URL'den query string'i silmemek: `?token=...` şeklinde access token'lar trace'te.
Baggage'a `tenant_id` KOYMAK OK ama email KOYMAK büyük hata.
Stack trace'i sansürlü tutmak: `' at User(email=alice@x.com)'` Java toString sızır.

Bölüm 31 — Multi-Tenant Mimari ve RBAC

Tek bir observability platformuna birden fazla takım/ekip/müşteri yazıyorsa, izolasyon kritik. Tenant A'nın trace'i Tenant B tarafından görülmemeli. Bu bölüm multi-tenancy pattern'larını ve RBAC stratejilerini işler.

31.1 Tenant İzolasyon Seviyeleri

Seviye	İzolasyon	Maliyet	Kullanım
Soft (label-based)	Tek backend, tenant_id label'i	Düşük	Internal ekipler
Medium (namespace)	Aynı cluster, ayrı namespace/index	Orta	Kurumsal multi-team
Hard (cluster)	Tenant başına ayrı cluster	Yüksek	SaaS, regulated

31.2 Soft Multi-Tenancy

Tüm tenant'lar aynı collector + storage'ı kullanır; tenant_id attribute'u ile izole edilir. Grafana'da row-level security ile dashboard'lar tenant'a göre filtrelenir.

```
# Collector – tenant header'ı resource attribute'una çevir
processors:
  transform/tenant:
    metric_statements:
      - context: resource
        statements:
          - set(attributes["tenant.id"],
                resource.attributes["http.request.header.x-tenant-id"])
```

Risk: 'noisy neighbor' — bir tenant'ın yüksek trafiği diğerlerini etkileyebilir.

31.3 Mimir/Tempo Native Multi-Tenancy

Grafana stack'i (Mimir, Tempo, Loki) native multi-tenancy destekler.

```
# Tempo distributor – tenant header'a göre route
multitenancy_enabled: true

# Limits
overrides:
  argela-prod:
    max_traces_per_user: 1000000
    ingestion_rate_limit_bytes: 50_000_000
  argela-staging:
    max_traces_per_user: 100000
    ingestion_rate_limit_bytes: 5_000_000
```

31.4 Grafana RBAC

Grafana Enterprise (veya OSS folder permission'ları) ile dashboard ve data source'lara erişim sınırlanır. Tipik politika:

- Folder/'argela-fm-prod' → 'fm-team' rolü

- Data source/'tempo-prod' → admin only
- Service account token'ları 7 gün TTL, otomatik rotation

31.5 Bölüm 31 — Tavsiyeler

PROD TAVSİYESİ

Yeni ekip onboarding süreci: tenant_id + RBAC + quota üçlü.

Quota olmadan multi-tenant: bir ekip diğerini boğar; per-tenant rate limit zorunlu.

Cross-tenant query'lere izin verme; SOX/HIPAA compliance'ta sorun yaratır.

Bölüm 32 — Observability Governance, SLO/SLI ve Incident Response

Observability bir mühendislik kültürü meselesidir. Araçlar kurulsun da, takımın bu araçları kullanma disiplini olmazsa platform değer üretmez. Bu bölüm governance, SLO tasarımı ve incident response'a uygun observability pattern'larını işler.

32.1 Observability Governance Framework

Olgun ekipler observability için 'governance committee' veya 'platform guild' kurar. Bu yapının sorumlulukları:

- Semantic conventions standardı (her ekibin attribute naming uyumu)
- Sampling rate policy (kim ne kadar trace tutar)
- Cost allocation (her ekibe storage maliyeti)
- New metric/log review (cardinality patlamasını önler)
- Backend upgrade ve breaking change communication

32.2 SLI, SLO, SLA — Tanımlar

Terim	Anlam	Örnek
SLI (Service Level Indicator)	Ölçülen değer	p99 latency, success rate, availability
SLO (Service Level Objective)	Hedef değer	p99 < 200ms son 30 gün
SLA (Service Level Agreement)	Müşteriye taahhüt	%99.9 uptime, ihlalde refund
Error Budget	1 - SLO; tolerans payı	%99.9 SLO = 43.2 dk/ay error budget

32.3 OTEL Verisinden SLI Üretmek

Argela 'fault-collector' için örnek SLI'ler:

```
# Availability SLI: başarılı request oranı
sum(rate(http_server_requests_seconds_count{
  service="fault-collector", outcome="SUCCESS"
}[5m]))
/
sum(rate(http_server_requests_seconds_count{
  service="fault-collector"
}[5m]))

# Latency SLI: p99 < 500ms
histogram_quantile(0.99,
  sum by (le) (rate(http_server_requests_seconds_bucket{
    service="fault-collector"
  }[5m]))
) < 0.5

# Alarm processing SLI: kuyrukta beklemeyen alarm yüzdesi
1 - (alarms_pending_count / alarms_received_total)
```

32.4 Burn Rate Alerting

Naive SLO alerting ('%99.9 ihlal edildi') yanlış zamanda alarm verir. Burn rate, error budget'in ne kadar hızlı tüketildiğini ölçer; multi-window alerting önerilir:

```
groups:
  - name: slo-fault-collector
    rules:
      # Fast burn: son 1 saatte 14x budget tüketimi → 2 günde tüm budget biter
      - alert: FastBurn
        expr: |
          (1 - (
            sum(rate(http_server_requests_seconds_count{
              service="fault-collector", outcome="SUCCESS"}[1h]))
            /
            sum(rate(http_server_requests_seconds_count{
              service="fault-collector"}[1h]))
          )) > (14 * 0.001)
        for: 2m
        labels: { severity: critical, page: "true" }

      # Slow burn: 6 saatte 6x → 5 günde budget biter
      - alert: SlowBurn
        expr: |
          (1 - (...)) > (6 * 0.001)
        for: 15m
        labels: { severity: warning }
```

32.5 Incident Response — Observability Yönü

Incident esnasında tracing + log + metric'in birlikte kullanılması:

31. Alert tetiklendi (metric anomalisi)
32. Exemplar veya trace ID ile spike'a sebep olan trace'lere atla
33. Trace içinde error span'ı bul; service.name + operation
34. Trace'in log'larına Grafana'dan tek tıkla geç (trace_id filter)
35. Stack trace'i çözümü, mitigasyon uygula
36. Postmortem'de tracing verisini kanıt olarak kullan

32.6 Postmortem'de Telemetry Kullanımı

Postmortem doc'larında 'timeline' bölümü, trace ID ve link içermeli. Örnek:

```
## Timeline (UTC)

14:23:11 - Alert FastBurn tetiklendi (fault-processor)
14:23:34 - Tracing kontrol: trace_id=4bf92f3577b34da6...
           → fault-processor → severity.calculate spans
           → SeverityStrategy.evaluate'da NullPointerException
14:24:02 - Log kontrol: NPE @ HighLatencyStrategy.java:42
           Sebep: alarmRequest.description == null
14:25:30 - Rollback başlatıldı v1.2.1 → v1.2.0
14:28:45 - Trafik normalize, alert clear
```

32.7 Bölüm 32 — Tavsiyeler

PROD TAVSİYESİ

Her servis için en az 1 availability + 1 latency SLO; SLI'ler OTel verisinden türetilsin.

Burn rate multi-window alerting kur; tek pencere alert spam'i yaratır.

Postmortem template'i trace_id alanı içersin; postmortem'lerde tracing kullanımını ölç.

Observability guild ayda 1 toplantı; semantic conv + sampling review.

ANTI-PATTERN

Her metric için bir alert kurmak: ortalama on-call 200 alert görürse hepsini ignore eder.

SLO'yu mühendis intuisyonuyla yazmak (müşteri ile konuşmadan): yanlış hedef.

Tracing'i sadece 'broken' anında kullanmak: proaktif inceleme kültürü kaybolur.

Alert tetiklediğinde sadece metric'e bakmak; trace+log korelasyonunu unutmak.

Bölüm 33 — Troubleshooting Handbook: 100+ Gerçek Vaka

Bu bölüm OpenTelemetry + Jaeger + Spring Boot ortamında karşılaşılan gerçek prod sorunlarını ve çözümlerini katalog halinde sunar. CTRL+F dostu, her vaka 'belirti → kök neden → çözüm' formatında. Bölüm 100+ vaka içerdiği için iki kısma bölünmüştür: 33A (vaka 1-60) ve 33B (vaka 61-130).

33A — Vakalar 1-60: Trace, Context, Collector, Exporter

A. Trace Görünmüyor / Kayıp Senaryoları

#1 — Jaeger UI'da hiç servis görünmüyor

BELİRTİ: Servis dropdown boş, refresh yapsam da bir şey yok.

KÖK NEDEN: OTEL agent yüklendi ama exporter endpoint'i yanlış; veya collector down; ya da collector'dan Jaeger'a gönderim hatalı.

ÇÖZÜM:

- Servisten collector'a network: `docker exec fault-collector wget -qO- http://otel-collector:13133`
- Collector self-metric: `curl http://otel-collector:8888/metrics | grep otelcol_exporter_send_failed_spans`
- Jaeger collector: `docker logs fm-jaeger | grep -i error`
- `OTEL_EXPORTER_OTLP_ENDPOINT` doğru protokol mü (grpc → :4317, http → :4318/v1/traces)?
- Sampling: `OTEL_TRACES_SAMPLER_ARG=0` koymadıysan veya parentbased ile parent=0 değilse spans üretiliyor mu kontrol et

#2 — Belirli bir servisin trace'i yok ama diğerleri var

BELİRTİ: fault-collector trace gönderiyor; fault-processor yok.

KÖK NEDEN: Sadece o servise OTEL agent atanmamış, ya da service.name override edilmemiş, ya da `JAVA_TOOL_OPTIONS` env unutulmuş.

ÇÖZÜM:

- `docker exec fault-processor sh -c 'echo $JAVA_TOOL_OPTIONS'`
- Beklenen: `-javaagent:/app/opentelemetry-javaagent.jar`
- Logback log'larında satırlarda `trace_id` görünüyor mu? Yoksa agent yüklenmedi.
- Agent debug: `-Dotel.javaagent.debug=true` → 'OpenTelemetry Javaagent: starting agent' log line aranır

#3 — Trace'in sadece bir kısmı görünüyor (parent yok)

BELİRTİ: fault-processor span'ı var ama fault-collector görünmüyor; trace orphan başlıyor.

KÖK NEDEN: Context propagation kopuk: traceparent header gönderilmiyor veya alıcı tarafta okunmuyor.

ÇÖZÜM:

- RestTemplate kullanıyorsan: agent ile auto-instrument edilir, ama eski Spring Boot 2 + manuel HTTP client'larda problem olabilir.
- WebClient'i custom builder ile yaratıyorsan: agent extension hook'unu kaçırma; .build() metodunu Bean olarak expose et.
- Inbound tarafta filter chain'in agent'tan önce trace context'i tüketip atması: Spring Security filter sırasını kontrol et.
- Reverse proxy/nginx propagation header drop ediyor mu? curl -v ile traceparent header'ı isteğe ekle, downstream'de görüyor musun?

#4 — İki ayrı trace'e bölünmüş, normalde tek olmalıydı

BELİRTİ: fault-collector POST /api/v1/alarms tek trace; fault-processor POST /api/v1/process ayrı trace olmuş.

KÖK NEDEN: Outbound HTTP çağrısında traceparent eklenemedi. Yeni span açıldı ama parent yok; yeni trace_id üretildi.

ÇÖZÜM:

- fault-collector'da WebClient agent ile instrument edildi mi?
otel.instrumentation.spring-webflux.enabled=true
- Custom HTTP client (Apache HttpClient, OkHttp, JDK java.net.http) için ayrı instrumentation aktif mi?
- Manuel HTTP yapıyorsan:
OpenTelemetry.getGlobalPropagators().getTextMapPropagator().inject(...) ile header'ı el ile ekle

#5 — Trace var ama operation isimleri 'unknown'

BELİRTİ: Jaeger'da operation 'HTTP GET' veya '/' gibi gözüküyor; route template yok.

KÖK NEDEN: Spring agent http.route attribute'unu doldurmuyor; veya semantic conv versiyon uyumsuzluğu.

ÇÖZÜM:

- Agent versiyonunu güncelle (en az 2.10.0)
- Spring Boot 3'te /actuator/health gibi endpoint'ler için /actuator/{name} pattern'i agent tarafında destekleniyor olmalı
- Custom @RequestMapping'lerde URI template kullan: @GetMapping('/alarms/{id}') gibi

#6 — Trace görünüyor ama 30 saniye gecikmeli

BELİRTİ: İstek atıldı, Jaeger'da 30 saniye sonra görüldü.

KÖK NEDEN: BatchSpanProcessor scheduleDelay default 5sn; collector batch processor timeout default 200ms ama send_batch_size hiç dolmuyorsa daha geç gönderir.

ÇÖZÜM:

- BSP: OTEL_BSP_SCHEDULE_DELAY=2000 (2sn'ye düşür)
- Collector batch: timeout: 2s, send_batch_size: 256 (düşük yük için)
- Düşük throughput sistemde scheduleDelay düşük tutmak doğru; high throughput'ta tersine yüksek olmalı

#7 — Yoğun saatte trace drop oluyor

BELİRTİ: Sabah hours'unda Jaeger'da %30 trace eksik.

KÖK NEDEN: BSP queue dolu, span drop ediliyor; veya collector memory_limiter refuse ediyor; veya exporter sending_queue dolu.

ÇÖZÜM:

- Spring app metric: otel.bsp.dropped_spans > 0
- Collector metric: otelcol_processor_dropped_spans, otelcol_receiver_refused_spans
- OTEL_BSP_MAX_QUEUE_SIZE=4096 (default 2048'den artır)
- Collector replica sayısını arttır (HPA aç)
- Sampling rate'i düşür

#8 — Sadece prod'da trace yok, dev'de var

BELİRTİ: Geliştirme ortamında trace görünüyor, prod'da hiç gelmemiş.

KÖK NEDEN: Sampling rate dev=1.0, prod=0.01 olabilir; veya prod collector'ın IP/DNS'i farklı; veya prod firewall 4317 portunu engelliyor.

ÇÖZÜM:

- ConfigMap'ten OTEL_TRACES_SAMPLER_ARG değerini kontrol et
- OTEL_EXPORTER_OTLP_ENDPOINT prod-collector.svc DNS'ini resolve ediyor mu? kubect! exec pod -- nslookup
- Network policy 4317 port'unu allow ediyor mu?
- VPC peering / security group cross-account erişim açık mı?

#9 — Trace var ama duration 0 veya negative

BELİRTİ: Span süresi 0ms veya negatif; UI'da yanıltıcı timeline.

KÖK NEDEN: Clock skew: container içindeki saat host'la senkron değil; veya manuel span'da start_time / end_time yanlış set edilmiş.

ÇÖZÜM:

- Container'da: date komutu host ile aynı mı kontrol et
- NTP/Chrony container'da çalışıyor mu? Cloud (AWS) ise time-sync service aktif olmalı
- Manuel span'da .startSpan() yerine .startSpan(Instant.now()) gibi explicit timestamp kullanılmışsa epoch'la milli/nano karışıklığı olabilir

#10 — Trace timeline'da boş süre var (gap)

BELİRTİ: Span A bitiyor 10:00:00.500'de, Span B başlıyor 10:00:01.200'de; arada 700ms 'kayıp'.

KÖK NEDEN: Network/proxy/serialization süresi; veya async iş yapıldı ama hiç span açılmadı; veya GC pause.

ÇÖZÜM:

- GC log'a bak: jstat -gcutil pid 1000
- JFR profil al: jcmd pid JFR.start duration=60s filename=/tmp/profile.jfr
- Async iş varsa: o işin span'ını ekle (manuel veya @WithSpan)
- JDBC connection pool wait süresi: HikariCP metric'lerinde görünür

#11 — DB span'ı yok, sadece servis span'ları görünüyor

BELİRTİ: fault-processor'ın HTTP span'ı var ama JDBC / Hibernate span'ı görünmüyor.

KÖK NEDEN: JDBC instrumentation kapalı; veya DataSource proxy'lenmemiş; veya Spring Data agent versiyonu eski.

ÇÖZÜM:

- otel.instrumentation.jdbc.enabled=true
- otel.instrumentation.hibernate.enabled=true
- HikariCP DataSource: agent otomatik instrument eder; custom bean ile yarattıysan @Bean DataSource döndürdüğüne emin ol
- Spring Data Repository: agent 2.x üstünde otomatik

#12 — Aynı trace'te aynı span iki kez görünüyor (duplicate)

BELİRTİ: Span ağacında 'GET /api/v1/process' iki kez aynı duration ile var.

KÖK NEDEN: Çift instrumentation: Java agent + library starter aynı anda aktif; veya Spring Boot 3 Micrometer Tracing + OTEL bridge + agent üçü aktif.

ÇÖZÜM:

- Spring Boot 3'te management.tracing.enabled=false yap, agent zaten span üretir
- Maven dependency tree'sinde opentelemetry-spring-boot-starter VE javaagent ikisi varsa birini kaldır
- otel.instrumentation.spring-webmvc.enabled=false yapıp Micrometer'a bırakabilirsin (alternatif)

B. Context Propagation Sorunları

#13 — @Async metodunda parent span kayboluyor

BELİRTİ: Sync metod span ağacı doğru; @Async ile çağrılan metod span'ı orphan veya hiç yok.

KÖK NEDEN: Spring TaskExecutor context propagation yapmıyor.

ÇÖZÜM:

- ThreadPoolTaskExecutor + ContextPropagatingTaskDecorator (Spring 6.1+)
- Veya manuel OtelTaskDecorator (Bölüm 12.3.2)

- SimpleAsyncTaskExecutor kullanma; her seferinde yeni thread + context loss

#14 — Reactor WebFlux'ta context'i kaybediyorum

BELİRTİ: .flatMap içine girdiğimde Span.current() invalid (default).

KÖK NEDEN: Reactor 3.5 öncesi: Schedulers thread switch yaparken ThreadLocal kopyalanmıyor.

ÇÖZÜM:

- Spring Boot 3.2+ kullan (Reactor 3.6 ile gelir)
- Eski versiyonlarda: Hooks.onEachOperator(Operators.lift(...)) ile global hook
- Veya: Mono.deferContextual ile manuel Context.current() taşı

#15 — Kafka consumer'da trace zinciri kopuyor

BELİRTİ: Producer span var, consumer span var, ama parent ilişkisi yok; iki ayrı trace gibi görünüyor.

KÖK NEDEN: Producer mesaja traceparent header eklemiyor; veya consumer header okumuyor.

ÇÖZÜM:

- Producer: ProducerRecord.headers() boş mu? Agent açıkken otomatik eklenir, custom binder kullanıyorsan kapalı olabilir
- Consumer: @KafkaListener'ın container factory'sinde recordInterceptor agent tarafında kayıt mı?
- Spring Kafka 3.x ile native OTel integration var;
otel.instrumentation.kafka.experimental-span-attributes=true
- Kafka headers preservation: bazı broker config (record.replication...) header'ı silebilir

#16 — nginx ingress traceparent header'ı drop ediyor

BELİRTİ: Client traceparent ile istek atıyor, backend'de görünmüyor.

KÖK NEDEN: Nginx default'ta küçük header'ları kaldırmaz ama bazı eski config'lerde proxy_set_header eksik olabilir.

ÇÖZÜM:

- Nginx config: proxy_pass_request_headers on; (default zaten on)
- Eski config: proxy_set_header traceparent \$http_traceparent;
- Cloudflare/CDN katmanı varsa: 'Bypass Cache by Cookie' vs custom header forwarding aktif mi?

#17 — Envoy / Istio sidecar trace context'i değiştiriyor

BELİRTİ: Service mesh varken trace zinciri farklı; pod'a giren traceparent ile çıkan farklı.

KÖK NEDEN: Istio default tracing-config'i kendi span'ını açar ve trace context'i kendine göre rewrite eder.

ÇÖZÜM:

- Istio meshConfig.defaultConfig.tracing'i OTel'e uyumlu yapılandır
- Veya tracing kapatıp app-level tracing'e bırak: meshConfig.enableTracing=false

- Istio + OTel hybrid çalışırken: B3 ve W3C propagator ikisini de aktif et

#18 — gRPC metadata trace context'i kayboluyor

BELİRTİ: gRPC client → server zincirinde span kopuk.

KÖK NEDEN: gRPC interceptor agent tarafından eklenmedi; veya custom interceptor agent'ı override etti.

ÇÖZÜM:

- `otel.instrumentation.grpc.enabled=true`
- Custom ServerInterceptor'lar Tracing interceptor'tan SONRA ekleniyor mu?
.intercept(otelInterceptor, customInterceptor) sırasına dikkat
- Channel'i custom yaratıyorsan: `GrpcTelemetry.create(openTelemetry).newClientInterceptor()` ile sarmala

#19 — Virtual thread'lerde context kayboluyor

BELİRTİ: `Thread.ofVirtual().start()` ile başlayan iş'te `Span.current()` invalid.

KÖK NEDEN: `ThreadLocal` otomatik kopyalanmaz; agent 1.32 öncesi virtual thread instrumentation yok.

ÇÖZÜM:

- Agent versiyonu 1.32+ olmalı
- Manuel capture: `Context current = Context.current(); ... try (Scope s = current.makeCurrent()) {...}`
- `ScopedValue` API'sine geçişi planla (Java 21 preview, ileride stable)

#20 — Scheduled task (Quartz, @Scheduled) trace'i yok

BELİRTİ: Cron job çalışıyor, ama Jaeger'da hiç span görünmüyor.

KÖK NEDEN: Scheduled job'ın doğal bir parent span'ı yok (kullanıcı isteği değil); agent her `@Scheduled` için span açmaz default.

ÇÖZÜM:

- `@WithSpan` annotation'ı koy: span manuel açılır
- Veya `tracer.spanBuilder('cron.daily-cleanup').setSpanKind(SpanKind.INTERNAL).startSpan()`
- Cron job trace'leri için ayrı sampling stratejisi düşün (genelde hepsini almak istersin)

#21 — CompletableFuture chain'i trace'i bozuyor

BELİRTİ: `supplyAsync` ile başlayan chain'de child span'lar parent yanlış gösteriyor.

KÖK NEDEN: Agent `java-util-concurrent` instrumentation'ı aktif değil veya `CommonPool` kullanılıyor.

ÇÖZÜM:

- `otel.instrumentation.java-util-concurrent.enabled=true` (default true)
- Custom executor: `Context.taskWrapping(executor)` ile sarmala
- `ForkJoinPool.commonPool()` agent ile instrumented; özel pool yarat ve sarmala

#22 — Baggage downstream'e ulaşmıyor

BELİRTİ: Gateway'de tenant_id baggage'a koydum; downstream servis `Baggage.current().getEntryValue('tenant_id')` null döner.

KÖK NEDEN: Propagator config'inde baggage yok; sadece tracecontext aktif.

ÇÖZÜM:

- `OTEL_PROPAGATORS=tracecontext,baggage`
- Spring Boot Micrometer kullanıyorsan: `management.tracing.propagation.produce/consume` listesine baggage ekle
- Baggage filter'lar collector'da silmiş mi? attributes processor'da `Baggage_` prefix'i kontrol et

#23 — Baggage value'sunda Türkçe karakter sorun yaratıyor

BELİRTİ: Baggage'a 'tenant.name=Türkiye İşbankası' koyunca header bozuluyor, downstream parse hatası.

KÖK NEDEN: Baggage RFC'ye göre percent-encoded ASCII olmalı; SDK encode etmez.

ÇÖZÜM:

- Baggage value'larını URL-encode et: `URLEncoder.encode(value, UTF_8)`
- Downstream'de `URLDecoder` ile decode et
- Veya tenant ID gibi ASCII-safe değer kullan, isim için ayrı lookup yap

#24 — Cross-language trace context format farkı

BELİRTİ: Java servis Go servise istek atıyor; Go tarafında parent span görünmüyor.

KÖK NEDEN: Java W3C tracecontext, Go OpenCensus B3 propagator kullanıyor olabilir.

ÇÖZÜM:

- Her iki tarafta da: W3C tracecontext + B3 multi her ikisini aktif et
- `OTEL_PROPAGATORS=tracecontext,baggage,b3multi`
- Go side: `otelpropagator dependency`'sinde `compositeTextMapPropagator(b3.New(), propagation.TraceContext{})`

C. Collector Sorunları

#25 — Collector OOMKilled oluyor

BELİRTİ: Pod sürekli restart, exit code 137, K8s event: OOMKilled.

KÖK NEDEN: memory_limiter kurulmamış veya pod limit çok düşük; veya tail_sampling buffer dolu.

ÇÖZÜM:

- memory_limiter processor: limit_mib pod limit'inin %75-80'i
- Pod memory limit'i collector binary'sinin ihtiyacının +%30 fazlası
- Tail sampling kullanıyorsan: `num_traces` × ortalama trace size hesapla; RAM yetmiyorsa `decision_wait` azalt
- Receiver `max_rcv_msg_size_mib` çok yüksekse (>100): tek mesaj OOM yaratabilir

#26 — Collector CPU %100, throughput düşük

BELİRTİ: Container CPU sürekli max, ama span/sn beklenenden az.

KÖK NEDEN: Single thread bottleneck; gRPC receiver tek bir worker'da; veya processor zincirinde regex-heavy transform.

ÇÖZÜM:

- Collector'ı multi-replica yap (HPA ile)
- transform processor'da regex pattern'leri optimize et; her span için derlenmesin
- pprof endpoint'ten profile al: curl http://collector:1777/debug/pprof/profile
- GOMAXPROCS ortam değişkeni container CPU limit'iyle aynı olmalı (auto detect)

#27 — Collector'dan backend'e bağlantı sürekli kopuyor

BELİRTİ: exporter_send_failed_spans sürekli artıyor; backend healthy ama collector retry yapıyor.

KÖK NEDEN: Collector ile backend arasında L4 LB connection idle timeout var; gRPC long-lived connection kesiliyor.

ÇÖZÜM:

- exporter keepalive ayarla: keepalive: { time: 30s, timeout: 5s, permit_without_stream: true }
- AWS NLB: idle timeout 350sn (sabit); gRPC keepalive bunun altında olmalı
- Backend tarafında: server keepalive enforce minimum policy gevşek olmalı

#28 — Tail sampling decision yanlış: hata trace'i drop ediliyor

BELİRTİ: Trace'te ERROR span var ama tail_sampling drop kararı verdi.

KÖK NEDEN: Sharded gateway'de aynı trace'in span'ları farklı collector'lara gitti; her biri eksik trace gördü.

ÇÖZÜM:

- Loadbalancer exporter ile front-tier: routing_key: traceID
- Back-tier statefulset, scaling agresif olmamalı (replica change → shard rebalance → karar bozulur)
- decision_wait yetmemiş olabilir: trace'in son span'ı geç gelmiş, karar zaten verilmiş

#29 — Collector restart'tan sonra trace gap'i

BELİRTİ: Collector restart oldu; restart süresince gelen span'lar kayıp.

KÖK NEDEN: Persistent queue yok; restart sırasında in-memory queue uçtu.

ÇÖZÜM:

- file_storage extension + sending_queue.storage: file_storage
- Persistent queue disk hızı bottleneck olmaz, ama disk space planla
- Rolling update yap: replica > 1 ise tek collector restart olur, diğerleri trafiği alır

#30 — Collector log'ları 'connection reset by peer' spam

BELİRTİ: Collector log'unda her birkaç saniyede bir connection reset.

KÖK NEDEN: Client BSP timeout'u collector receiver timeout'undan kısa; veya client side TCP RST gönderiyor.

ÇÖZÜM:

- Client OTEL_EXPORTER_OTLP_TIMEOUT > collector receiver timeout
- Mesh varsa: sidecar TCP idle timeout'unu kontrol et
- Bazı durumlarda log level WARN'a indirip ignore: receivers.otlp.grpc bağlı; sorun değil

#31 — Collector 'too many open files' hatası

BELİRTİ: Log'da 'accept: too many open files'.

KÖK NEDEN: ulimit'ler container içinde düşük; her connection FD tüketiyor.

ÇÖZÜM:

- Pod spec'te: securityContext.sysctls'da fs.file-max yükselt veya init container ile ulimit ayarla
- K8s node sysctl: fs.file-max=2097152
- Collector'ın connection pool'unu küçült: receiver tarafında max_connection_idle

#32 — Collector'a sürekli 'unknown service' hatası dönüyor

BELİRTİ: Client log'unda: 'OTel exporter: status=Status{code=UNIMPLEMENTED}'.

KÖK NEDEN: Client OTLP/gRPC ama collector OTLP/HTTP açık; ya da port yanlış.

ÇÖZÜM:

- 4317 = gRPC, 4318 = HTTP; karıştırma
- OTEL_EXPORTER_OTLP_PROTOCOL=grpc veya http/protobuf
- Collector receiver hem grpc hem http açık olmalı: ikisini de tanımla

#33 — Collector multi-pipeline'da bazı veri yanlış pipeline'a gidiyor

BELİRTİ: Trace pipeline'a metric verisi gelmiş; veya tersine.

KÖK NEDEN: Pipeline tanımında karışıklık; receiver'ın hangi sinyal döndürdüğü gözden kaçmış.

ÇÖZÜM:

- service.pipelines'da traces/metrics/logs ayrımı net olmalı
- Multi-instance receiver: 'otlp/grpc' ve 'otlp/http' farklı isimle tanımlanmalı
- Pipeline name'lerini standardize et: 'traces/main', 'metrics/prometheus'

#34 — memory_limiter aktif ama yine de OOM oluyor

BELİRTİ: memory_limiter tetikleniyor (log'da görünüyor) ama pod yine OOMKilled.

KÖK NEDEN: memory_limiter Go heap'i ölçer; Go runtime'ın native memory'sini, goroutine stack'leri, syscall buffer'ları kapsamaz.

ÇÖZÜM:

- Pod limit'i memory_limiter.limit_mib + memory_limiter.spike_limit_mib + %30 olmalı

- Örnek: limit_mib=1500, spike_limit_mib=250 → pod limit en az 2200Mi
- GOMEMLIMIT environment ile Go GC'yi pod limit'inin altında tut

D. Exporter ve Backend Sorunları

#35 — Jaeger UI 'no traces found' diyor ama backend hayatta

BELİRTİ: Search yapıyorum, trace gelmiyor; lookup by trace_id da boş.

KÖK NEDEN: Time range yanlış (default son 1 saat), veya servis name dropdown'da yok, veya ES mapping bozuk.

ÇÖZÜM:

- Time range'i 'Last 24h' yap
- Servis dropdown'da fault-collector görünüyor mu? Yoksa hiç trace ulaşmamış demek
- Jaeger collector log: 'spans indexed' artıyor mu?
- ES: curl 'http://es:9200/_cat/indices/jaeger-*?v' — günlük index var mı?

#36 — ES mapping exception: 'mapper of [X] cannot be changed'

BELİRTİ: Jaeger collector log'unda mapping exception, span'lar drop ediliyor.

KÖK NEDEN: Bir attribute'un type'ı değişti (string → number); ES günlük index'te aynı field için iki tip görüyor.

ÇÖZÜM:

- Jaeger ES_TAGS_AS_FIELDS_ALL=false yap (default), sadece sorgulanan tag'leri field yap
- Mevcut index'i sil veya yeni günlük index'ten itibaren düzelt: DELETE jaeger-span-2026-05-23
- Application tarafında attribute'un tipini sabitleyin: 'alarm.severity' her zaman string, integer atma

#37 — ES disk doldu, Jaeger çalışmıyor

BELİRTİ: ES read-only moda geçti; ', cluster.routing.allocation.disk.flood_stage' alarm.

KÖK NEDEN: Index Lifecycle Management kurulmamış; eski index'ler silinmiyor.

ÇÖZÜM:

- Eski index'leri manuel sil: curl -XDELETE 'http://es:9200/jaeger-span-2026-04-*)
- ILM kur: 7 günde delete; warm tier 3 günde
- ES disk watermark'larını disk büyüklüğüne göre ayarla: cluster.routing.allocation.disk.watermark.low=85%

#38 — Jaeger UI yavaş, search'ler timeout

BELİRTİ: Trace listesi 30+ saniye yükleniyor.

KÖK NEDEN: ES query timeout; shard sayısı yetersiz; veya tag-based search index'siz alan üzerinde yapılıyor.

ÇÖZÜM:

- ES_NUM_SHARDS=5 (default 1); shard sayısını arttır
- Daha sık aranan tag'leri tag_as_fields_include listesine ekle (alarm.severity vs.)
- Time range'i daralt; 7 günlük search yerine 24 saatlik
- ES heap düşükse: -Xms4g -Xmx4g (max 31g)

#39 — Jaeger collector start olmuyor, ES'e bağlanamıyor

BELİRTİ: jaeger-collector log'unda 'Failed to connect to elasticsearch'.

KÖK NEDEN: ES henüz hazır değil ama collector start oldu; veya ES_USERNAME/PASSWORD eksik (X-Pack security).

ÇÖZÜM:

- depends_on: condition: service_healthy ile ES sağlık beklemesi şart
- X-Pack security açıksa: ES_USERNAME=jaeger, ES_PASSWORD=secret
- TLS: ES_TLS_ENABLED=true, ES_TLS_CA gerekli
- Initial cluster: ES single-node'da single-node config; production ES'te master/data ayrımı

#40 — Span attribute'lar Jaeger UI'da görünmüyor ama trace var

BELİRTİ: Trace listesi geliyor, ama detayında tag/attribute sekmesi boş.

KÖK NEDEN: Eski Jaeger versiyonu yeni OTel semantic conv attribute isimlerini tanımıyor; veya tag'ler nested object'te kalmış.

ÇÖZÜM:

- Jaeger v1.42+ veya v2 kullan
- ES_TAGS_AS_FIELDS_INCLUDE ile önemli tag'leri field yap; nested olmaktan kurtar
- Spans index template'ini güncel tut

#41 — OTLP exporter timeout sürekli

BELİRTİ: Collector log: 'context deadline exceeded' her 30sn.

KÖK NEDEN: Backend yavaş yanıt veriyor; veya network latency yüksek (cross-region).

ÇÖZÜM:

- exporter timeout: 30s → 60s
- Backend tarafında bottleneck (ES) sorgu/insert latency'sini incele
- Cross-region telemetry push: kötü pratik; her region kendi backend'ine yazmalı
- retry_on_failure max_elapsed_time'ı yükselt

#42 — Prometheus exporter'dan metric scrape 'context deadline'

BELİRTİ: Collector :8889/metrics scrape'i Prometheus tarafında timeout.

KÖK NEDEN: Çok yüksek cardinality (50k+ series), scrape uzun sürüyor.

ÇÖZÜM:

- scrape_timeout: 30s
- scrape_interval'i de yükselt (60s)

- `metric_relabel_configs` ile gereksiz `metric`'leri drop et
- Push-based (`remoteWrite`) alternatifine geç

#43 — Trace var ama `service.name` 'unknown_service:java'

BELİRTİ: Jaeger'da servis adı 'unknown_service:java' veya 'spring-boot'.

KÖK NEDEN: `OTEL_SERVICE_NAME` set edilmemiş; default değer kullanılıyor.

ÇÖZÜM:

- `OTEL_SERVICE_NAME=fault-collector` (env)
- Veya `OTEL_RESOURCE_ATTRIBUTES=service.name=fault-collector`
- Spring Boot 3'te: `spring.application.name` agent tarafından okunmaz; explicit ayarla

#44 — OTLP HTTP yerine gRPC kullansam daha hızlı mı?

BELİRTİ: Performans testlerinde HTTP vs gRPC fark var mı?

KÖK NEDEN: Genel olarak gRPC daha hızlı (binary framing, multiplexing), ama küçük yükte fark <%5.

ÇÖZÜM:

- Aynı pod içinde sidecar collector: ikisi de yeterli
- Cross-region veya internet üzerinden: gRPC long-lived connection avantajlı
- Connection problem'i varsa (proxy/firewall): HTTP'a düş

E. Sampling Sorunları

#45 — Hata trace'leri sample edilmiyor

BELİRTİ: Production'da %10 sampling var; hatalı trace'lerin %90'ı kayıp.

KÖK NEDEN: Head sampling: karar trace başında alındı; hata sonradan oluştu.

ÇÖZÜM:

- Head sampling yerine tail sampling kullan
- Tail sampling policy: `status_code: ERROR` → her zaman al
- Veya: `errors_only` sampler (özel) — SDK manuel implementasyon

#46 — Sampling rate %10 ayarladım ama %100 trace görüyorum

BELİRTİ: `OTEL_TRACES_SAMPLER_ARG=0.1` set ettim, hala her şey görünüyor.

KÖK NEDEN: ParentBased sampler kullanılıyor; parent karar zaten `sampled=true` olduğu için child da alınıyor.

ÇÖZÜM:

- Root span'lar gerçekten kim tarafından üretiliyor? Eğer gateway/upstream'den traceparent geliyorsa karar orada veriliyor
- Tüm servislerde aynı sampling rate set et
- Yeni trace'ler senin servisinden başlıyorsa `OTEL_TRACES_SAMPLER=traceidratio` (parent-less)

#47 — Yeni servis trace üretmiyor sanıyorum, sampler %0**BELİRTİ:** OTEL_TRACES_SAMPLER=alwaysOff yanlışlıkla bırakılmış.**KÖK NEDEN:** Demo / test sırasında geçici set edildi, prod'a geçince unutuldu.**ÇÖZÜM:**

- ConfigMap audit: OTEL_TRACES_SAMPLER ve _ARG değerlerini standardize et
- OTEL_TRACES_SAMPLER=parentbased_traceidratio default
- OTEL_TRACES_SAMPLER_ARG=0.1 prod, 1.0 dev/staging

#48 — Tail sampling 'expected_new_traces_per_sec' nasıl ayarlanır?**BELİRTİ:** Documentation belirsiz; bu parametre tail_sampling kararı için kullanılıyor mu?**KÖK NEDEN:** expected_new_traces_per_sec, internal LRU cache sizing içindir; actual decision'ı etkilemez ama yanlış ayarlanırsa bellek tüketimi kötüleşir.**ÇÖZÜM:**

- Gerçek RPS'in 2-3 katını gir (200 RPS sistemde: 500)
- num_traces ile birlikte düşün: $\text{num_traces} = \text{expected_new_traces_per_sec} \times \text{decision_wait} \times 1.5$

F. Metric ve Cardinality Sorunları**#49 — Prometheus disk %90 doldu birden bire****BELİRTİ:** Stable çalışan sistem ani disk patlaması.**KÖK NEDEN:** Bir geliştirici user_id'yi metric label yapmış; cardinality patladı.**ÇÖZÜM:**

- `topk(20, count by (__name__){__name__=~'.+'}))` ile en büyük metric'leri bul
- Yeni eklenen metric'i geri al; `relabel_config` ile drop
- Cardinality limit'leri zorunlu kıl (Prometheus 2.43+ veya Mimir)

#50 — Spanmetrics'ten gelen metric'lerde cardinality patlıyor**BELİRTİ:** spanmetrics processor aktifleştirildikten sonra Prometheus series count 5x.**KÖK NEDEN:** Spanmetrics 'dimensions' listesinde `http.target` gibi yüksek-cardinality attribute var.**ÇÖZÜM:**

- dimensions yerine `http.route` kullan (template)
- `spanmetrics.metrics_flush_interval`'ı düşür; aggregation pencere küçülür
- `exclude_dimensions` ile drop edilecek attribute'ları belirt

#51 — OTel metric'lerinde 'name conflict' hatası**BELİRTİ:** Collector log'unda: 'metric X already registered with different type'.**KÖK NEDEN:** Aynı metric ismi farklı type ile (Counter vs Gauge) iki yerden geliyor.

ÇÖZÜM:

- Custom metric'leri isimle prefix'le: argela.alarm.received_total
- spanmetrics processor name çakışmasını önle: namespace ayarla
- Micrometer + OTEL bridge: bazı default metric'ler iki kez registered olabilir

#52 — Histogram bucket'ları yanlış: p99 değeri max'a sıkışmış

BELİRTİ: histogram_quantile(0.99,...) her zaman 10s gibi düz çizgi.

KÖK NEDEN: Histogram bucket boundary'leri çok düşük; gerçek değer en üst bucket'a düşüyor.

ÇÖZÜM:

- ExplicitBucketHistogramAggregation ile bucket'ları genişlet: [10ms, 50ms, 100ms, 500ms, 1s, 5s, 10s]
- Veya ExponentialHistogramAggregation kullan (OTel exponential): bucket boundary'ler otomatik

G. Performans ve JVM Sorunları**#53 — OTEL agent yüklendikten sonra startup süresi 2x oldu**

BELİRTİ: Spring Boot startup 30sn → 60sn.

KÖK NEDEN: Agent bytecode transform her sınıf yüklenirken pahalı.

ÇÖZÜM:

- Gereksiz instrumentation'ları kapat: otel.instrumentation.X.enabled=false (jdbi, batik, vs.)
- AppCDS / CDS kullan: java -XX:ArchiveClassesAtExit=app-cds.jsa
- Cloud-native: Quarkus/Micronaut native image (ama tracing'in farklı kurulması gerek)

#54 — p99 latency agent ile %20 arttı

BELİRTİ: Tracing yokken p99 100ms; agent ile 120ms.

KÖK NEDEN: BSP scheduleDelay default 5s; export sırasında GC pause veya export thread contention.

ÇÖZÜM:

- BSP_MAX_QUEUE_SIZE=8192 (artır)
- BSP_MAX_EXPORT_BATCH_SIZE=2048
- BSP_SCHEDULE_DELAY=2000
- G1GC pause target: -XX:MaxGCPauseMillis=50
- Sampling rate düşür

#55 — JVM heap agent ile 200MB fazla harcıyor

BELİRTİ: Container memory limit'i agent ile aşıyor.

KÖK NEDEN: BSP queue + attribute string'leri + serialization buffer'lar.

ÇÖZÜM:

- Container memory request/limit'ini +200MB artır

- Span attribute count'unu kısıtla: OTEL_ATTRIBUTE_COUNT_LIMIT=64
- Attribute value length: OTEL_ATTRIBUTE_VALUE_LENGTH_LIMIT=512

#56 — Allocation rate agent ile 2x oldu

BELİRTİ: JFR'da young GC sıklığı 5sn'den 2sn'ye düştü.

KÖK NEDEN: Her span attribute eklemesi yeni String/array allocate ediyor.

ÇÖZÜM:

- Hot path'te @WithSpan kullanma
- AttributeKey.stringKey(...) constant olarak tanımla, her seferinde yeniden create etme
- Span event log'larını sadece error durumunda ekle

#57 — GC pause süresi agent ile artmış

BELİRTİ: p99'da GC pause spike'ları görüyorum.

KÖK NEDEN: Allocation pressure + heap fragmentation.

ÇÖZÜM:

- ZGC veya Shenandoah dene (low-latency GC)
- -XX:+UseZGC -Xmx4g (Java 21)
- JFR ile allocation hotspot'larını bul

#58 — Thread count agent ile 10+ thread artmış

BELİRTİ: jstack ile 'BSP', 'OtelExporter' thread'leri görüyorum.

KÖK NEDEN: BSP worker, exporter thread, metric reader thread default'ta.

ÇÖZÜM:

- Bu thread'ler normaldir; thread count artışı 10-15 kadar.
- JVM thread limit problem değilse görmezden gel
- Eğer çok fazlaysa: birden fazla SdkTracerProvider initialize olmuş olabilir; tek global olmalı

#59 — Direct memory (off-heap) agent ile büyüyor

BELİRTİ: Container OOM ama heap düşük; native memory tüketimi.

KÖK NEDEN: Netty/gRPC off-heap buffer'lar; OTLP gRPC exporter Netty kullanır.

ÇÖZÜM:

- -XX:MaxDirectMemorySize=256m ile sınırla
- Native Memory Tracking: -XX:NativeMemoryTracking=summary, jcmd pid VM.native_memory
- OTLP HTTP'a geç (Netty kullanmaz)

#60 — Java 21 virtual thread'le agent çakışıyor

BELİRTİ: Agent versiyonu 1.30 ile VT'de pinning veya context loss.

KÖK NEDEN: Agent 1.32 öncesi VT instrumentation eksik.

ÇÖZÜM:

- Agent 2.10.0+ kullan
- JFR'da Pinned Thread event'leri kontrol: -XX:+UnlockExperimentalVMOptions
- Manuel Context.current() capture pattern'i ile bypass et

Bölüm 33B — Troubleshooting Handbook (devam): Vakalar 61-130

H. Spring Boot Özelinde Sorunlar

#61 — Spring Boot 3 + Micrometer Tracing aktif, OTEL agent ile çift trace üretiyor

BELİRTİ: Span ağacı her seviyede 2 span'a bölünmüş.

KÖK NEDEN: Hem Micrometer Tracing (observation API) hem OTEL agent aktif; iki ayrı tracing pipeline.

ÇÖZÜM:

- `management.tracing.enabled=false` yap (Micrometer Tracing kapat)
- Agent'ı bırak; o zaten observation API'yi de instrument eder
- Veya tersine: agent'ı kaldır, `micrometer-tracing-bridge-otel` kullan (tek pipeline)

#62 — WebClient çağrısı trace etmiyor, agent açık

BELİRTİ: RestTemplate ile yapılan istekler trace'leniyor ama WebClient ile yapılanlar parent'sız.

KÖK NEDEN: WebClient.Builder bean'i agent'ın gördüğü standart yoldan yaratılmamış (örn. lambda içinde `new WebClient.Builder()`).

ÇÖZÜM:

- @Bean WebClient.Builder döndür: agent inject eder
- Veya @Autowired WebClient.Builder builder ile inject et
- Custom builder zincirinde `.filter(...)` yerine `.filters(...)` kullanırsan agent'ın filter'ını override etme

#63 — @Transactional metodun span'ı yok ama içindeki JDBC span'ı var

BELİRTİ: DB span'ı görünüyor ama @Transactional kullanan service span'ı yok.

KÖK NEDEN: @Transactional, AOP proxy gerektirir; aynı sınıf içinden çağrılırsa proxy bypass edilir; span da kaybolur.

ÇÖZÜM:

- Inter-service call yap: `serviceA.txMethod()` (serviceB içinden)
- Manuel span ekle: `tracer.spanBuilder('process-tx').startSpan()`
- @WithSpan ile çift annotation (proxy zaten kuruluysa AOP zincirinde sırayla)

#64 — Spring Security filter chain trace context'i bozuyor

BELİRTİ: /login endpoint'ine giden trace var ama protected endpoint'lerde trace başlamış gibi görünmüyor.

KÖK NEDEN: Spring Security filter'ı OTEL filter'tan ÖNCE; OTEL başlayamıyor.

ÇÖZÜM:

- OTEL agent'in doRequest'i Tomcat filter chain'in en başında olmalı (default böyledir)
- Custom SecurityFilterChain'de OncePerRequestFilter yazıyorsan: OTEL filter'tan SONRA gelsin
- Spring Boot 3.1+ default sıralama uyumlu

#65 — Spring Data JPA repository span'ı yok

BELİRTİ: AlarmRepository.save() çağırısı için INSERT span görünmüyor.

KÖK NEDEN: Hibernate/JPA instrumentation kapalı; veya transaction manager farklı yapılandırılmış.

ÇÖZÜM:

- otel.instrumentation.hibernate.enabled=true
- otel.instrumentation.spring-data.enabled=true
- Hibernate 6.x: spring-data-jpa 3.x uyumlu agent versiyonu en az 2.0.0

#66 — Spring Cloud Gateway trace context'i bozuyor

BELİRTİ: Gateway'den gelen istekte yeni traceparent üretiliyor.

KÖK NEDEN: Spring Cloud Gateway reactor-netty altında çalışır; OTEL reactor instrumentation aktif değil veya gateway custom filter'ı header'ı silmiş.

ÇÖZÜM:

- spring-cloud-gateway için otel.instrumentation.spring-cloud-gateway.enabled=true
- Custom GatewayFilter'da: exchange.getRequest().getHeaders() bozma, sadece read
- Spring Cloud Sleuth yerine sadece OTEL kullan (Sleuth deprecated)

#67 — Spring Boot actuator /prometheus endpoint'inde OTEL metric'leri yok

BELİRTİ: Sadece JVM ve Spring default metric'leri görünüyor; argela.alarm.received_total yok.

KÖK NEDEN: OTEL SDK metric'leri Prometheus exposition'a basılmıyor; ayrı bir pipeline gerekli.

ÇÖZÜM:

- Micrometer OTEL registry: micrometer-registry-otlp eklenirse push model
- Veya Prometheus receiver collector'da: /actuator/prometheus'ı scrape et
- Veya OTEL metric'leri ayrı endpoint'te (otlp-collector → prometheus exporter)

#68 — Spring Boot graceful shutdown'da son trace'ler kayıp

BELİRTİ: Pod terminate olurken son birkaç saniyenin span'ları gelmiyor.

KÖK NEDEN: BSP flush ediliyor ama JVM shutdown hook tarafından bekleme süresi yetersiz.

ÇÖZÜM:

- server.shutdown=graceful + spring.lifecycle.timeout-per-shutdown-phase=30s
- OTEL SDK shutdown hook: TracerProvider.shutdown() en az 10s timeout
- K8s preStop hook: sleep 10 + actuator shutdown

#69 — Logback JSON encoder + OTEL agent trace_id basmıyor

BELİRTİ: JSON log'da trace_id field'ı yok.

KÖK NEDEN: Logback'in agent tarafından instrument edilmemesi; veya MDC injection kapalı.

ÇÖZÜM:

- `otel.instrumentation.logback-appender.enabled=true`
- `otel.instrumentation.logback-mdc.enabled=true` (deprecated, yerine logback-appender)
- LogstashEncoder + `<mdc/>` provider içerir; `trace_id` otomatik basar

#70 — Spring Boot 3 + Java 21 virtual thread + OTel — context kayboluyor

BELİRTİ: `spring.threads.virtual.enabled=true` ile request başına VT; trace zinciri kopuk.

KÖK NEDEN: Agent versiyonu eski; VT instrumentation eksik.

ÇÖZÜM:

- Agent 2.10.0+
- `spring.threads.virtual.enabled=true` ile VT açıkken Reactor + OTel instrumentation versiyon uyumu
- Test: `jstack` ile virtual thread'leri kontrol et, `ScopedValue/Context.current()` debug log'u

I. Kafka / Mesajlaşma

#71 — Kafka producer span'ı var, consumer span'ı yok

BELİRTİ: `ProducerRecord` ile gönderildi; `@KafkaListener`'da hiç span görünmüyor.

KÖK NEDEN: Spring Kafka instrumentation kapalı; veya `ConsumerFactory` custom yapılandırılmış.

ÇÖZÜM:

- `otel.instrumentation.kafka.enabled=true`
- `otel.instrumentation.spring-kafka.enabled=true`
- Custom `ConcurrentKafkaListenerContainerFactory`: `setObservationEnabled(true)` (Spring Kafka 3.x)

#72 — Kafka consumer span'ı tek mesaj için açılıyor ama batch consume var

BELİRTİ: Batch consumer 100 mesaj alıyor, sadece 1 span görünüyor.

KÖK NEDEN: Batch consumer için span model: link'lerle tek consume span + her mesaja link.

ÇÖZÜM:

- Bu OTel'in beklenen davranışı (batch consumer)
- Her mesaj için ayrı span istiyorsan: manuel olarak record başına span aç
- Span event ile her mesajın id'sini ekle: `addEvent('message-processed', attributes)`

#73 — Kafka header propagation broker tarafında siliniyor

BELİRTİ: Producer traceparent header ekledi, consumer'da yok.

KÖK NEDEN: Broker compaction veya re-replication header'ı silebilir (rare); ya da producer config `'headers.delivery.strategy'` ayarı.

ÇÖZÜM:

- Kafka 0.11+ header destekler; broker downgrade olmadı mı?
- Consumer'da: `record.headers().toArray()` ile gerçekten geliyor mu kontrol et
- Compacted topic: header korunur normal şartlarda; debezium/mirror maker araları silebilir

#74 — Kafka Streams trace zinciri kopuyor

BELİRTİ: `KStream.map().to()` chain'inde her node'da yeni trace başlıyor.

KÖK NEDEN: Kafka Streams'in record'tan-record'a context'i implicit taşıması; OTel agent stream API'sini henüz tam instrument etmiyor.

ÇÖZÜM:

- Kafka Streams 3.5+ ile observability hook'ları geliştirdi
- Manuel: `ProcessorContext.headers()` ile trace context'i her node'da taşı
- Veya: state store ile `trace_id`'yi key+trace_id olarak persist et

#75 — RabbitMQ consumer span'ı var ama parent yanlış

BELİRTİ: Producer A → broker → Consumer B; B'nin parent'ı A değil.

KÖK NEDEN: Default propagation 'follows_from'; consumer span 'CONSUMER kind' + producer span'a link.

ÇÖZÜM:

- Bu doğru davranıştır; link yapısı 'temporal relationship'i ifade eder
- Eğer parent-child isteniyorsa:
`otel.instrumentation.rabbitmq.experimental.span-suppression-strategy` değiştir
- Producer `trace_id`'yi message property'ye koy; consumer'da
`Span.current().setLink(producerContext)`

#76 — JMS / ActiveMQ consumer trace'i yok

BELİRTİ: JMS listener'da hiç span görünmüyor.

KÖK NEDEN: `otel.instrumentation.jms.enabled=false` default; veya ActiveMQ versiyonu eski.

ÇÖZÜM:

- `otel.instrumentation.jms.enabled=true`
- ActiveMQ 5.16+ veya Artemis kullan
- Spring JMS 6.x ile observation API entegre

#77 — AWS SQS consumer'da trace context kayboluyor

BELİRTİ: SDK v2 ile SQS consume; trace yok.

KÖK NEDEN: SQS message attributes max 10 limiti; traceparent + tracestate + baggage 3 attribute kaplar.

ÇÖZÜM:

- AWS SDK v2 instrumentation aktif: `otel.instrumentation.aws-sdk-2.2.enabled=true`

- Eğer kendi consumer kod'u yazıyorsan: `messageAttributes`'tan `traceparent` oku, `Context.current().with(...).makeCurrent()`
- Diğer business attribute'ları minimize et (10 limit)

J. Docker / Compose

#78 — Container içinden 'connection refused' otel-collector'a

BELİRTİ: fault-collector log: 'failed to connect to otel-collector:4317'.

KÖK NEDEN: Compose service name'ler yanlış; veya networks aynı değil.

ÇÖZÜM:

- Aynı network'te mi? `docker network inspect <network>`
- Service name eşleşiyor mu? otel-collector mi yoksa otelcol mu?
- `depends_on`: `condition: service_started` ile başlama sırası garanti

#79 — Compose volume'da Jaeger data persiste etmiyor

BELİRTİ: docker-compose down + up sonrası eski trace'ler kayıp.

KÖK NEDEN: Volume tanımlanmamış; veya `SPAN_STORAGE_TYPE=memory` bırakılmış.

ÇÖZÜM:

- `volumes`: - `es-data:/usr/share/elasticsearch/data`
- `SPAN_STORAGE_TYPE=elasticsearch` + `ES_SERVER_URLS`
- Volume snapshot'lar production için yetmez; ES snapshot API gerekli

#80 — Compose'da Jaeger UI 'cluster_block_exception'

BELİRTİ: Trace ekleme çalışıyor ama UI sürekli error veriyor.

KÖK NEDEN: Elasticsearch read-only modda; disk %95+ veya cluster yellow/red.

ÇÖZÜM:

- `curl http://es:9200/_cluster/health` → status: green olmalı
- Eski index'leri sil + watermark'ları yükselt
- ES container memory limit'ini artır (en az 2Gi)

#81 — Container clock skew nedeniyle span timestamp'leri tutarsız

BELİRTİ: Trace'te child span parent'tan ÖNCE başlamış görünüyor.

KÖK NEDEN: Docker host saatler senkron değil; veya Mac/Windows Docker Desktop'ta saat sürüklenmesi.

ÇÖZÜM:

- Linux host: `chronyd` / `ntpd` kontrol
- Docker Desktop (macOS): Settings > Reset > 'Sync time on startup'
- Container içinde: `docker exec ... date` — host ile aynı olmalı

K. Kubernetes Özelinde Sorunlar

#82 — K8s'ta sidecar collector injection yapılmıyor

BELİRTİ: Pod annotation eklendi ama collector container yok.

KÖK NEDEN: OpenTelemetry Operator yüklü değil veya admission webhook çalışmıyor.

ÇÖZÜM:

- `kubectl get pods -n opentelemetry-operator-system`
- `kubectl get mutatingwebhookconfigurations | grep opentelemetry`
- cert-manager kurulu mu? Operator gerektirir
- Annotation namespace eşleşmesi: `instrumentation.opentelemetry.io/inject-java: 'namespace/inst-name'`

#83 — Pod restart sonrası span attribute'larında k8s.* yok

BELİRTİ: Eskiden `k8s.pod.name`, `k8s.namespace.name` vardı; şimdi yok.

KÖK NEDEN: `k8sattributes` processor yapılandırması bozuldu veya RBAC yetkisi alınmış.

ÇÖZÜM:

- `k8sattributes` processor: `pod_association` doğru mu? `from: resource_attribute, name: k8s.pod.ip`
- ServiceAccount'a Pod/Node read yetkisi var mı? ClusterRole + binding
- Hangi pod IP'sinden geldiği bilinmiyor olabilir; X-Forwarded-For chain

#84 — K8s service mesh (Istio) ile OTel span'ları çakışıyor

BELİRTİ: Istio envoy proxy span'ı ve app span'ı ikisi de var; trace ağacı karışık.

KÖK NEDEN: Istio default'ta kendi span'ını açar (`envoy.tracers.opencensus`).

ÇÖZÜM:

- Istio `meshConfig.defaultProviders.tracing`'i OTel'e yönlendir
- Veya app-level tracing'e bırakıp envoy tracing'i kapat
- B3 + W3C propagator ikisini de aktif et (Istio B3, OTel W3C)

#85 — K8s rolling update sırasında trace gap'i

BELİRTİ: Deployment image değişimi sırasında 30sn trace yok.

KÖK NEDEN: Pod terminate → BSP flush atlanıyor; veya yeni pod start'ta agent yüklenirken trace üretmiyor.

ÇÖZÜM:

- `preStop` lifecycle hook: `sleep 15 + curl actuator shutdown`
- `terminationGracePeriodSeconds`: 60
- Rolling update strategy: `maxUnavailable=0` (her zaman replica var)

#86 — OTel Collector pod'u node restart sonrası başlamıyor

BELİRTİ: ContainerCreating'de takılı.

KÖK NEDEN: ConfigMap mount sorunu; veya secret eksik.

ÇÖZÜM:

- kubectl describe pod ... → Events bölümünde clue
- ConfigMap exists mi? Doğru namespace?
- ServiceAccount + ImagePullSecret kontrolü

#87 — K8s ingress (nginx) traceparent header'ı modifiye ediyor

BELİRTİ: External request traceparent ile geldi; içeride değişmiş.

KÖK NEDEN: Nginx 'X-Forwarded-' chain ile header'ı manipüle ediyor; veya rate-limit middleware temizliyor.

ÇÖZÜM:

- nginx.ingress.kubernetes.io/enable-opentracing: 'true' (eski yöntem)
- Daha iyisi: nginx.ingress.kubernetes.io/configuration-snippet ile traceparent passthrough
- İdeali: ingress-nginx 1.10+ OTEL native

#88 — K8s HPA collector'ı scale ediyor ama trace drop

BELİRTİ: Yeni replica eklendi, kısa süre trace kaybı.

KÖK NEDEN: Service discovery DNS cache'i; client connection eski replica'lara pinned.

ÇÖZÜM:

- Client side connection refresh interval düşür (gRPC keepalive)
- Headless service + DNS round-robin değil, L7 LB (Envoy)
- OTEL SDK: OTEL_EXPORTER_OTLP_CHANNELZ_ENABLED debug

L. Grafana / Visualization

#89 — Grafana'da Tempo data source 'no data'

BELİRTİ: Trace ID girdim, 'no traces found'.

KÖK NEDEN: Tempo data source URL yanlış; veya tenant header eksik.

ÇÖZÜM:

- Tempo URL: http://tempo:3200 (HTTP) veya tempo:9095 (gRPC)
- Multi-tenant: HTTP header X-Scope-OrgID set edilmeli
- Time range backend retention dışı olabilir

#90 — Grafana exemplar tıklayınca trace açılmıyor

BELİRTİ: Metric grafikte exemplar noktaları var, ama tıklayınca 'no trace'.

KÖK NEDEN: Data source linkleri yanlış; veya exemplar trace_id'si trace'e ulaşmıyor (sample edilmedi).

ÇÖZÜM:

- Prometheus data source > Exemplars > Link to: Tempo
- Label name: trace_id (Prometheus exposition'da bu isim olmalı)
- Sampling: %100 olmayan sistemlerde exemplar'ın trace'i drop edilmiş olabilir

#91 — Grafana dashboard JSON import sonrası panel boş

BELİRTİ: Dashboard yüklendi, ama panel verileri yok.

KÖK NEDEN: Data source UID mismatch; veya metric isimleri farklı.

ÇÖZÜM:

- Dashboard JSON içinde datasource UID hard-coded; replace et
- Grafana provisioning: dashboards.yml ile uid override
- Metric isimlerini eski/yeni semantic conv'a göre kontrol et

#92 — Trace-to-logs linki Grafana'da çalışmıyor

BELİRTİ: Trace span'a tıkladığımda 'Logs for this span' butonu yok.

KÖK NEDEN: Tempo data source'da tracesToLogsV2 config'i yok; veya Loki data source UID yanlış.

ÇÖZÜM:

- Tempo data source > Trace to logs > Loki data source seç
- Tag mapping: service.name → service_name
- Time shift: -30s/+30s

M. Sampling / Coverage**#93 — Tail sampling decision_wait yetmiyor: hata span'ı geç geliyor**

BELİRTİ: Yavaş downstream nedeniyle hata span'ı 30sn'den sonra geliyor; tail sampling karar zaten verilmiş (drop).

KÖK NEDEN: decision_wait süresi gerçek trace süresinin altında.

ÇÖZÜM:

- decision_wait'i tipik trace süresinin 2-3 katı yap
- Bunu yapamıyorsan: rule policy ekle 'late_arriving_errors' → drop edilmiş trace'leri tekrar değerlendir (sınırlı çözüm)
- Producer/consumer ayrımıyla long-running consumer span'lar için ayrı pipeline

#94 — Adaptive sampling tahmin yapamıyor

BELİRTİ: RPS değişken, sampling rate her dakika dalgalı.

KÖK NEDEN: Jaeger adaptive sampler'ın hedef QPS'i gerçek workload'a uymuyor.

ÇÖZÜM:

- target_samples_per_second'ı gerçek p95'e göre ayarla
- Sampling decision cache TTL'i kısalt

- Veya tail sampling'e geç (daha deterministic)

#95 — Servis sampling kararını ignore ediyor

BELİRTİ: Sampler config %10 ama tüm trace'ler geliyor.

KÖK NEDEN: Spring Boot'taki management.tracing.sampling.probability 1.0 default; agent ile çalışıyor.

ÇÖZÜM:

- management.tracing.sampling.probability=0.1 yap
- Veya management.tracing.enabled=false (agent tek source)
- Logback debug ile gerçek sampler'ı kontrol: 'OTel Sampler: ...'

N. Security ve PII

#96 — Trace'te raw Authorization header görünüyor

BELİRTİ: Jaeger UI'da http.request.header.authorization=Bearer eyJ... görünüyor.

KÖK NEDEN: Default'ta agent bu header'ı capture etmez; ama otel.instrumentation.http.capture-headers.server.request='*' set edilmiş.

ÇÖZÜM:

- Bu config'i kaldır; specific header listesi ver
- Collector'da attributes/redact processor ile delete et
- PII redaction policy code review'da zorunlu olsun

#97 — db.statement içinde plain text password

BELİRTİ: Trace'te 'INSERT INTO users ... password='hello123\'

KÖK NEDEN: Hibernate JDBC instrumentation parametrelili sorgu kullanmıyor (prepared statement değil); veya ?% bind sırasında.

ÇÖZÜM:

- Tüm sorgular PreparedStatement olmalı (bind variables)
- Collector'da transform processor: replace_pattern(db.statement, "'[^']+'", "'?'")
- Otel SDK'da db.statement.sanitization parametresi (eğer destekleniyorsa)

#98 — GDPR audit'te user_id'lerin trace'te bulunduğu ortaya çıktı

BELİRTİ: Compliance ekip: 'user.id attribute'larını silin'.

KÖK NEDEN: Eski span'lar storage'ta; canlı sistemde de hala yazılıyor.

ÇÖZÜM:

- Live: collector'da delete_key(attributes, 'user.id')
- Geçmiş data: ES delete by query (yavaş ama mümkün)
- Future: user.id yerine hashed_user_id; orijinal mapping ayrı kontrol edilen DB'de

#99 — Cross-tenant trace contamination

BELİRTİ: Tenant A'nın trace'inde Tenant B'nin data'sı görünüyor.

KÖK NEDEN: Baggage cross-request leak; veya ThreadLocal context cleanup eksik (thread reuse).

ÇÖZÜM:

- Filter'da request bitiminde Context.clear() benzeri (Spring filter chain'in sonu)
- Baggage'a tenant_id koymak yerine span attribute olarak set et
- Thread pool'da threadLocal cleanup şart: PooledThreadFactory ile thread reset

O. Diğer Edge Case'ler

#100 — OTel agent + native image (GraalVM) çakışması

BELİRTİ: GraalVM native image build'te agent çalışmıyor.

KÖK NEDEN: Bytecode manipulation native image'a uygun değil.

ÇÖZÜM:

- GraalVM native image kullanıyorsan: agent yerine OTel Spring Boot Starter veya manual instrumentation
- Quarkus için ayrı OTel extension var
- Native image + tracing setup tamamen farklı paradigma

#101 — Agent'ta JFR çakışması

BELİRTİ: JFR recording başlatınca uygulama OOM veya pause.

KÖK NEDEN: Agent + JFR aynı bytecode'u monitor etmeye çalışıyor.

ÇÖZÜM:

- JFR settings dosyasında otel-related class'ları exclude et
- Agent debug kapalı tut: otel.javaagent.debug=false
- JFR continuous yerine periodic snapshot al

#102 — Multi-jar Spring Boot fat jar'da agent yüklenmiyor

BELİRTİ: java -javaagent:agent.jar -jar app.jar → 'OpenTelemetry agent not initialized'.

KÖK NEDEN: Spring Boot LauncherURL classloader sırası yanlış; agent classpath'i göremiyor.

ÇÖZÜM:

- Spring Boot 2.4+: layered jar yapısı agent dostu
- Spring Boot 3+: native compatibility iyileşti
- Alternatif: ExecutableJarLauncher tarafında özel classpath manipulation

#103 — Multiple agent jar'larından çakışma

BELİRTİ: JAVA_TOOL_OPTIONS'da hem OTel hem APM agent (Dynatrace, Datadog, NewRelic) var, ikisi çatışıyor.

KÖK NEDEN: Vendor agent + OTel agent aynı method'u transform etmeye çalışıyor.

ÇÖZÜM:

- Sadece bir agent kullan: OTel + collector ile vendor exporter
- Veya vendor'un OTel-compatible mode'u: Datadog Java SDK'sı OTLP exporter destekler
- İki agent zorunluysa: -javaagent sırası önemli; OTel önce

#104 — Long-running trace UI'da yarım gösteriliyor

BELİRTİ: 5 saatlik batch job trace'i; UI'da sadece ilk 100 span görünüyor.

KÖK NEDEN: Jaeger UI default render limit'i; veya trace tek index'i aşmış (gece yarısı geçti, span'lar iki günlük index'te).

ÇÖZÜM:

- Jaeger query.max-traces parametresi
- Long-running job'ları parent + child trace olarak ayır (her saat yeni trace)
- Batch işlerde span explosion'a dikkat

#105 — Trace'te 1000+ span var, UI donuyor

BELİRTİ: Span explosion: tek trace 5000+ span içeriyor.

KÖK NEDEN: Loop içinde span açma anti-pattern'i (Bölüm 22.5).

ÇÖZÜM:

- Loop dışı parent span + her iteration için event
- Eski trace'leri storage'tan sil: ES delete by query (trace_id)
- Filter processor ile excessive span'ları drop

#106 — Span attribute count limit aşıyor

BELİRTİ: Log'da 'Span attribute count limit exceeded'.

KÖK NEDEN: Default 128 attribute limit; biri 200 attribute set ediyor.

ÇÖZÜM:

- OTEL_ATTRIBUTE_COUNT_LIMIT=256 (artır)
- Veya ekstrayı azalt: gerçekten 128'den fazla attribute lazım mı?
- Bazı agent versiyonları header capture ile 50+ attribute ekler; kapatabilirsin

#107 — Span event sayısı çok yüksek, trace size patladı

BELİRTİ: Tek trace 50MB; ES indexlenirken timeout.

KÖK NEDEN: Event'ler retry/cache/iteration için açılıyor; binlerce.

ÇÖZÜM:

- OTEL_SPAN_EVENT_COUNT_LIMIT=128 (varsayılan)
- Loop içinde event açma; aggregate olarak tek event 'iterations=1000'
- Retry event'leri sadece error olduğunda

#108 — Yeni servis OTel ile prod'a çıktı, %100 fail

BELİRTİ: Deploy sonrası uygulama crash, NoClassDefFoundError.

KÖK NEDEN: Agent jar dosyası container'a eklenmedi veya path yanlış.

ÇÖZÜM:

- Dockerfile'da: COPY otel-agent.jar /app/otel-agent.jar
- Permission: chmod 644 /app/otel-agent.jar
- JAVA_TOOL_OPTIONS: -javaagent:/app/otel-agent.jar (boşluk olmasın)

#109 — OTLP gRPC TLS handshake fail

BELİRTİ: Log: 'transport: authentication handshake failed: x509: certificate signed by unknown authority'.

KÖK NEDEN: Client OS trust store'da CA yok.

ÇÖZÜM:

- OTEL_EXPORTER_OTLP_CERTIFICATE=/etc/certs/ca.crt ile CA bundle ver
- OS trust store'a CA ekle: update-ca-certificates (Debian)
- Self-signed: OTEL_EXPORTER_OTLP_INSECURE=true (sadece dev)

#110 — Cloud provider managed Prometheus (AMP, GMP) push fail

BELİRTİ: AWS AMP remote-write 'AccessDenied'.

KÖK NEDEN: IAM role yanlış; SigV4 signing eksik.

ÇÖZÜM:

- Collector sigv4auth extension kur
- Role: aps:RemoteWrite policy
- GCP: Workload Identity ile service account

#111 — Tempo backend 'too many requests'

BELİRTİ: Trace ingestion 429.

KÖK NEDEN: Rate limit aşıldı; default 50k spans/sec/tenant.

ÇÖZÜM:

- Tempo distributor limits.ingestion_rate_limit_bytes artır
- Per-tenant override config
- Backend HPA scale

#112 — Datadog'a OTLP ile veri yolluyoruz, attribute'lar Datadog UI'da görünmüyor

BELİRTİ: Datadog APM'de span var ama attribute'lar boş.

KÖK NEDEN: Datadog kendi semantic conv'unu bekliyor; OTel attribute'lar mapping gerekiyor.

ÇÖZÜM:

- Collector'da datadog exporter kullan (resmi DD pipeline)

- Veya transform processor: OTel attr → DD attr remap
- Datadog 'unified service tagging' için service.name, service.version, deployment.environment zorunlu

#113 — Lambda trace cold start'ta agent slow

BELİRTİ: Cold start +3sn agent yüzünden.

KÖK NEDEN: Java agent ile Lambda + cold start çift cezalı.

ÇÖZÜM:

- AWS Distro for OpenTelemetry (ADOT) Lambda Layer kullan; optimized
- Lambda Snap-Start: cold start 10x azaltır (Java 11+)
- Provisioned concurrency

#114 — ECS Fargate'te collector sidecar 'unhealthy'

BELİRTİ: Task definition collector container'ı bitti gösteriyor.

KÖK NEDEN: Healthcheck endpoint yanlış veya port mapping eksik.

ÇÖZÜM:

- Healthcheck: command [CMD, /healthcheck] (curl yerine custom binary, image içinde yok olabilir)
- Image: otel/opentelemetry-collector-contrib:0.108.0 — wget/curl yok
- Healthcheck extension :13133 endpoint'ini probe et, image kendi wget'ile

#115 — Collector self-metric'leri Prometheus'a gitmiyor

BELİRTİ: Otelcol_* metric'leri Prometheus'ta yok.

KÖK NEDEN: Self-telemetry port'u (:8888) Prometheus scrape config'inde yok.

ÇÖZÜM:

- service.telemetry.metrics.address: 0.0.0.0:8888
- Prometheus scrape: targets: [otel-collector:8888]
- Metric path: /metrics (default)

#116 — Hibernate ManyToMany ilişkide span explosion

BELİRTİ: Tek istek 10.000 SELECT span açıyor.

KÖK NEDEN: N+1 query problemi; her ilişki için ayrı sorgu.

ÇÖZÜM:

- Fetch strategy düzelt: JOIN FETCH, EntityGraph
- Hibernate statistic etkinleştir: hibernate.generate_statistics=true
- Collector'da filter: drop spans with db.system=postgresql AND name='SELECT' if count > 100

#117 — OTel SDK 'GlobalOpenTelemetry has already been set'

BELİRTİ: Uygulama crash; SDK initialize edilmeye çalışılıyor ama agent zaten initialize etmiş.

KÖK NEDEN: Hem agent hem manuel SDK init kodu var.

ÇÖZÜM:

- Manuel SDK init kodunu kaldır; agent zaten yapıyor
- `GlobalOpenTelemetry.get()` ile mevcut instance'ı al

#118 — Prometheus stale marker'lar trace exemplar'a karışıyor

BELİRTİ: Eski exemplar'lar yeni grafiğe yansıyor.

KÖK NEDEN: Prometheus exemplar storage TTL'i yüksek; stale data.

ÇÖZÜM:

- `--storage.tsdb.exemplars-max-exemplars=100000`
- Eski metric series cleanup: `__name__` pattern'i ile clean

#119 — ClickHouse exporter retry yapmıyor, veri kayıp

BELİRTİ: Toplu insert fail oldu, retry hiç olmadı.

KÖK NEDEN: ClickHouse exporter (community) `sending_queue` + `retry` config'i eksik veya farklı.

ÇÖZÜM:

- Community exporter docs'unu oku; resmi pattern'i izle
- Async insert kullan (CH 22.8+): `async_insert=1`, `wait_for_async_insert=0`
- Batch boyutu uygun: 100k row/batch ideal

#120 — Migration: Sleuth → OTEL'de attribute isimleri farklı

BELİRTİ: Eski Sleuth'tan gelen 'http.url' yeni servisten 'url.full' geliyor; dashboard kırılıyor.

KÖK NEDEN: OpenTelemetry semantic conv 1.21+ ile attribute isimleri değişti.

ÇÖZÜM:

- Collector'da transform processor ile attribute rename: `set(attributes['http.url'], attributes['url.full'])`
- Migration sürecinde her iki isim de mevcut; aliasing geçici çözüm
- Dashboard'ları yeni isimlere göre güncelle

#121 — Production rollout sonrası sistem yavaşladı

BELİRTİ: Tracing açıldıktan sonra p99 latency 20% arttı.

KÖK NEDEN: Sampling %100 prod'a kaldı; veya BSP queue size düşük.

ÇÖZÜM:

- Sampling'i %5-10'a düşür (head + tail sampling kombinasyonu)
- `BSP_MAX_QUEUE_SIZE=8192`, `MAX_EXPORT_BATCH_SIZE=2048`, `SCHEDULE_DELAY=2000`
- JVM args: `-XX:MaxGCPauseMillis=100 -XX:+UseG1GC`

#122 — Trace'te şüpheli 'export failed: deadline exceeded'

BELİRTİ: Collector log'unda exporter timeout her dakika.

KÖK NEDEN: Backend yavaş veya network latency yüksek.

ÇÖZÜM:

- exporter timeout artır: 30s → 60s
- Backend response time'ı izle (kendi metric'lerinde)
- Cross-region trafiği azalt; her region kendi backend'ine yazsın

#123 — Auto-instrumentation Java agent eski Java 8 ile çalışmıyor

BELİRTİ: Java 8'de agent yüklenmiyor.

KÖK NEDEN: OTel Java agent 1.32+ minimum Java 8; 2.0+ minimum Java 11.

ÇÖZÜM:

- Java 11+'a geç
- Java 8'de kalmak zorundaysan: agent 1.x serisi (en son 1.32.x)
- Yeni özellikler için Java upgrade şart

#124 — OTel SDK loglarında 'OpenCensusToOpenTelemetry: shim deprecated'

BELİRTİ: Legacy OpenCensus instrumentation hala var.

KÖK NEDEN: Eski kod OpenCensus API kullanıyor; shim ile geçici çalışıyor.

ÇÖZÜM:

- OpenCensus kullanan kütüphaneleri OTel'e migrate et
- Shim deprecated; gelecek versiyonlarda kaldırılacak
- Yeni proje sadece OTel

#125 — OTel Collector 'reload config' sonrası eski config'le çalışıyor

BELİRTİ: ConfigMap güncellendi; SIGHUP sonrası eski davranış.

KÖK NEDEN: Collector reload mekanizması bazı extension'ları reload etmiyor.

ÇÖZÜM:

- Pod restart en güvenli yol
- Kustomize/Helm ile config hash annotation'ı: değişince pod rollout tetiklenir
- Operator + ConfigMap watcher ile otomatik restart

#126 — Multi-environment'tan gelen trace'ler karışıyor

BELİRTİ: Dev cluster'dan gelen trace prod backend'de görünüyor.

KÖK NEDEN: Dev OTel SDK endpoint'i prod collector'a yanlış işaret ediyor.

ÇÖZÜM:

- deployment.environment attribute'unu ZORUNLU yap (resource detector)
- Backend tarafında filter: deployment.environment=prod olmayanları drop
- Network policy: dev → prod observability cross-call engellensin

#127 — Trace API'sini extension için kullanıyoruz, span'lar collector'a ulaşmıyor

BELİRTİ: OTEL SDK API'siyle programmatic olarak span üretiyoruz; ama backend'de yok.

KÖK NEDEN: GlobalOpenTelemetry hiç initialize edilmemiş; default no-op çalışıyor.

ÇÖZÜM:

- Java agent yüklü mü? -javaagent argument'i kontrol
- Yoksa autoconfigure SDK initialize et:
io.opentelemetry.sdk.autoconfigure.AutoConfiguredOpenTelemetrySdk.initialize()
- Test ortamında InMemorySpanExporter ile unit test

#128 — OTLP/HTTP'de büyük batch reddediliyor

BELİRTİ: HTTP 413 Payload Too Large.

KÖK NEDEN: Default request size limit; collector veya gateway/proxy.

ÇÖZÜM:

- Collector receivers.otlp.http.max_request_body_size_mib: 32
- Ingress/proxy: nginx client_max_body_size 32m
- Client BSP: max_export_batch_size azalt (4096 → 1024)

#129 — Cassandra storage'da Jaeger query çok yavaş

BELİRTİ: Tag'ler ile arama 30sn+.

KÖK NEDEN: Cassandra trace_id-only optimized; tag arama secondary index pahalı.

ÇÖZÜM:

- Cassandra yerine Elasticsearch'e geç (tag arama için)
- Sadece trace_id ile lookup ediliyorsa Cassandra OK
- Sık sorgulanan tag'leri ayrı materialized view'lara yaz

#130 — OTEL + service mesh + Kubernetes ingress = 4 katman span çakışması

BELİRTİ: Bir istek için ingress, gateway, sidecar, app olmak üzere 4 span açılıyor; trace çok karışık.

KÖK NEDEN: Her L7 katman kendi span'ını açıyor; trace 'noise' içeriyor.

ÇÖZÜM:

- Stratejik karar: kim span açacak? Çoğunlukla 'app + sidecar' yeterli, ingress'i kapatabilirsin
- Veya tersine 'sadece ingress + app' (mesh tracing kapalı)
- Trace listesini sadeleştirmek için Jaeger UI 'service map' kullan

33.2 — Troubleshooting Methodology

130+ vakayı kataloglamak iyi bir referans; ama mühendis olarak asıl ihtiyacınız

37. KATMAN AYRIMI: Sorun client SDK'da mı, collector'da mı, network'te mi, backend'de mi? Her katmanı izole et.

38. SELF-METRIC: Önce kendi self-metric'lerine bak (BSP dropped, collector refused/dropped). Eğer 0 ise sorun upstream'de.
39. DEBUG LOG: OTel SDK debug + collector debug log seviyesi geçici aç; production'da sadece patch period.
40. REPRODUCE: Sorun staging'de yeniden üretilebiliyor mu? Üretilemiyorsa prod-specific (sampling, yük, network).
41. MINIMAL CASE: Tek bir test istek ile sorun gözleniyorsa client SDK; 1000 istekte gözleniyorsa pipeline.
42. DIFF: Çalışan başka bir servis ile karşılaştır. Hangi yapılandırma farkı?

DEBUGGING

Debugging araç kutusu: collector zpages (:55679/debug/tracez), Jaeger UI search, Prometheus PromQL self-metric, JFR/async-profiler, jstack, tcpdump.

Collector zpages özellikle exporter retry queue'sunu canlı gösterir; çok değerli.

Async-profiler ile Java agent allocation hotspot'ları tespit edilebilir.

Bölüm 34 — Production Checklist

Bu bölüm bir servisi tracing destekli olarak prod'a almadan önce yapılması gereken kontrollerin checklist'idir. Her madde için 'evet/hayır' net olmalı; emin değilseniz prod'a çıkmayın.

34.1 SDK ve Instrumentation

- OTEL_SERVICE_NAME explicit set edilmiş (unknown_service:java değil)
- OTEL_RESOURCE_ATTRIBUTES'ta service.version, deployment.environment, service.namespace mevcut
- Sampling: parentbased_traceidratio + uygun rate (head sampling) veya tail sampling pipeline'ı
- BSP queue size yüke uygun (default 2048 düşük; en az 4096)
- Java agent versiyonu en az 2.0.0 (Spring Boot 3 için)
- Logback/Log4j MDC instrumentation aktif (trace_id log'lara basılıyor)
- Hot path metotlarda @WithSpan kullanılmıyor
- Loop içinde span açılmıyor (event tercih edilmiş)
- Manuel span'larda try-with-resources + setStatus(ERROR) + recordException pattern'i kullanılmış

34.2 Collector

- Collector ayrı pod/process olarak (sidecar/daemonset/gateway)
- memory_limiter processor pipeline'in başında
- batch processor pipeline'in sonunda
- retry_on_failure aktif (exporter)
- sending_queue size yeterli ve persistent (file_storage)
- Self-telemetry port'u (:8888) Prometheus tarafından scrape ediliyor
- PII redaction processor aktif (attributes/transform/redaction)
- Healthcheck endpoint :13133 K8s liveness/readiness probe ile bağlı
- Resource limits set edilmiş (memory limit > memory_limiter.limit_mib + spike + %20)
- HPA veya replica >= 3 (gateway için)

34.3 Backend (Jaeger/Tempo/ES)

- ES master + data node ayrımı yapılmış (single-node prod'da değil)
- Index Lifecycle Management (ILM) aktif: hot/warm/cold + delete policy
- Disk watermark alarm: %85 sonrası uyarı
- Snapshot policy: günlük S3'e snapshot
- Multi-AZ deployment (HA)
- Backup/restore drill ayda 1 yapılıyor
- Tag/attribute field count limit: 1000 (ES default) yeterli
- Retention politikası: 7-14 gün (compliance gereği ise audit log ayrı)

34.4 Network ve Güvenlik

- TLS aktif (cluster-internal bile)
- mTLS gateway-backend arası (zero-trust)
- Authentication: bearer token veya OIDC
- Cert rotation: cert-manager veya hot reload
- Network policy: collector sadece application namespace'lerinden erişilebilir
- Egress kontrolü: backend'e cross-region trafiği minimize edilmiş

34.5 SLO ve Alerting

- Servis için availability + latency SLO yazılmış
- Burn rate multi-window alerting kurulmuş
- Tracing platform için ayrı SLO (uptime, ingest rate)
- Alert routing: pager (critical), email (warning)
- Runbook: her alert için 'next steps' belgesi
- Postmortem template trace_id alanı içeriyor

34.6 Cost ve Capacity

- Aylık cost dashboard: per-service storage maliyeti
- Cardinality monitoring: prometheus_tsdb_head_series alert
- Tail sampling oranı %2-5'in altında değilse review
- Capacity headroom %30+
- Backup storage maliyeti hesaplanmış

Bölüm 35 — Sık Sorulan Sorular (FAQ)

Q1: 'OpenTelemetry öğrenmek ne kadar sürer?'

Junior bir Java developer için temel kullanım (agent + manuel span) 1-2 hafta. Collector pipeline, sampling, troubleshooting senior seviyesi ise 3-6 ay. Mimari kararlar (HA, multi-tenancy, governance) 1-2 yıllık deneyim. Bu doküman tüm seviyeler için referans olacak şekilde tasarlandı.

Q2: 'Hangi sampling rate doğru?'

'Doğru rate yoktur' diyemem; olgun ekipler %1-10 head sampling + tail sampling kombinasyonu kullanır. Argela boyutlu staj projesinde %100 sampling (head) demo için iyi; gerçek prod öncesi tail sampling pipeline'ı kurun.

Q3: 'Java agent mi manuel mi?'

İkisi birlikte. Agent altyapıyı (HTTP, JDBC, Kafka) otomatik; manuel iş kuralı için custom span. 'Tek başına manuel' = ekstra iş; 'tek başına agent' = iş katmanı görünmüyor.

Q4: 'Jaeger mi Tempo mu?'

Self-hosted + maliyet odaklıysa Tempo + S3. Olgun search/UI istiyorsanız Jaeger + ES. Grafana ekosistemi içindeyseniz Tempo doğal. Argela demo için Jaeger v2 + ES yeterli.

Q5: 'Trace verisi GDPR kapsamında mı?'

Evet, eğer IP, user_id, email gibi 'kişisel veri' içeriyorsa. PII redaction collector'da merkezi yapılmalı. Audit log ayrı sistemde tutulmalı.

Q6: 'OTel'i kullanıp Datadog'a göndermek mümkün mü?'

Evet. Collector'da Datadog exporter kullanılır; uygulama OTel API'sine yazar. Vendor değişimi tek satır config değişikliğine indirgenir.

Q7: 'Production'da hangi sampling oranı normal?'

Tek bir doğru sayı yok ama referans: 100-1000 RPS sistemde %10 baseline + tail sampling errors/slow. 10k+ RPS'te %1 baseline + tail. Çok düşük RPS (<10) sistemlerde %100 OK.

Q8: 'Collector kullanmadan doğrudan Jaeger'a push edemez miyim?'

Yapabilirsiniz ama tavsiye etmem. Vendor lock-in, retry, batch, redaction tüm sorumluluk SDK'da. Collector ara katmanı kritik.

Q9: 'Tracing latency'i ne kadar etkiler?'

İyi yapılandırılmış sistem: p99 +%2-5. Kötü yapılandırılmış (sync exporter, %100 sampling, küçük BSP queue): +%20-30.

Q10: 'Yeni servis tracing eklerken nereden başlamalı?'

1) OTel Java agent JAR'ı ekle, 2) OTEL_SERVICE_NAME + OTEL_EXPORTER_OTLP_ENDPOINT env'i set et, 3) Logback JSON encoder kur (trace_id log'a basılsın), 4) İş kuralında 2-3 manuel span ekle, 5) Sampling %10 ile başla. Bu sırayla bir gün sürer.

Q11: 'Production'da agent versiyonunu nasıl güncelliyorum?'

Container image'a agent gömülü ise: image build + rolling restart. Sidecar olarak yüklüyse: OTel Operator + Instrumentation CRD güncelleme. Her durumda staging'de bir hafta tut, breaking change'leri yakala.

Q12: 'Span ile log arasındaki farkı bir cümlede özetler misin?'

Log 'bu noktada şu oldu' der; span 'bu işlemi şu kadar süre yaptım, bu attribute'larla' der. Span süre + bağlam, log mesaj.

Q13: 'Tracing'i CI/CD'ye nasıl entegre ederim?'

CI build step'lerini de OTel span'ı olarak modelleyebilirsiniz (otel-cli, Jenkins OTel plugin). Bu sayede 'build neden yavaşladı' sorusu trace üzerinden cevaplanır.

Q14: 'Local development'ta tracing gerek var mı?'

Demo amaçlı evet, hata ayıklama için OK. Sürekli tracing ile çalışmak development'ı yavaşlatabilir; başka mock backend (LogExporter) ile sadece span yapısını görmek yeterli.

Q15: 'Mevcut Sleuth + Zipkin sistemden OTel'e nasıl geçerim?'

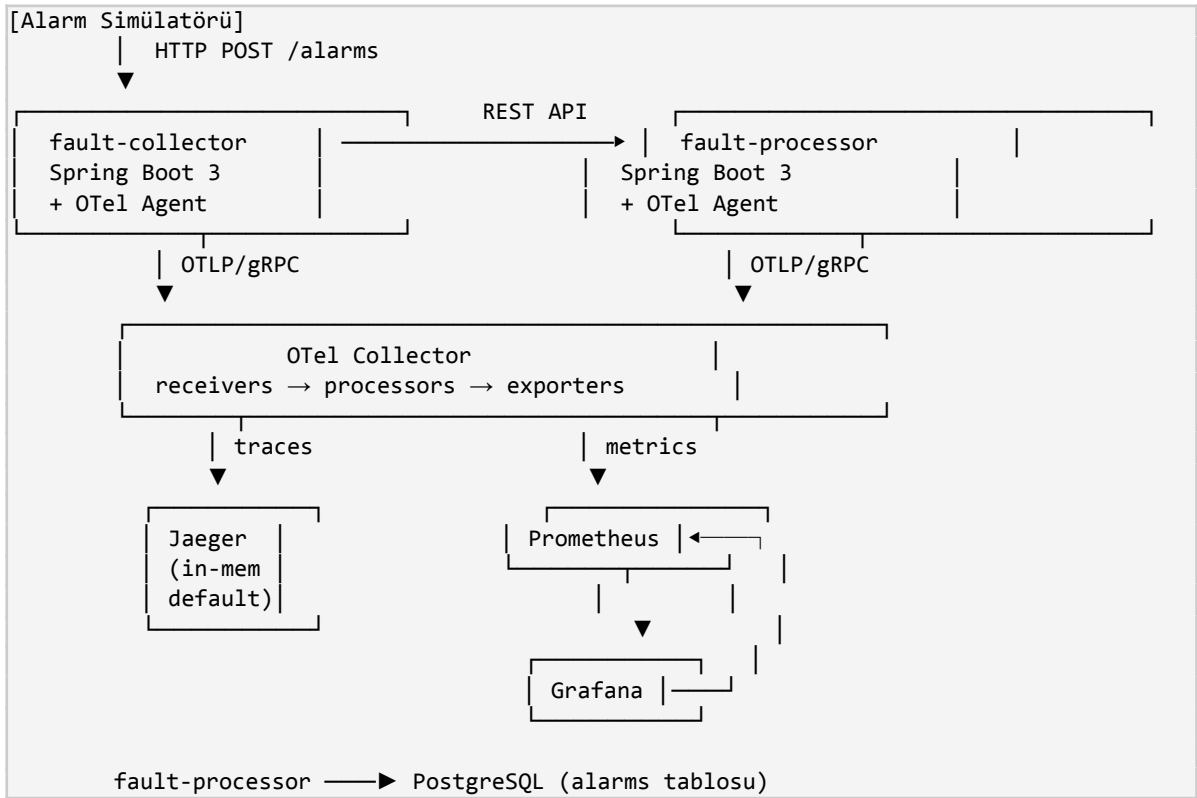
Migration adımları: 1) Mevcut Sleuth aktif tut, OTel agent'ı ekle (çift instrumentation kısa süreli OK), 2) Zipkin yerine collector + B3 propagator'a yönlendir, 3) Yeni servisleri sadece OTel ile aç, 4) Eski servislerin Sleuth bağımlılığını sırayla kaldır. 2-3 ay sürer.

Bölüm 36 — Argela Fault Management Case Study

Bu bölüm Argela Technologies'deki Fault Management staj projesinin OpenTelemetry kullanımını referans olarak doküman boyunca anlatılan kavramların somut bir uygulamasını sergiler. Mevcut yapı CLAUDE.md'de belgelenen mimariyi temel alır.

36.1 Mevcut Mimari Özeti

Argela Fault Management iki Spring Boot servisinden oluşan basit bir ağ alarmı işleme sistemidir: fault-collector alarm alır + iletir; fault-processor severity atar + PostgreSQL'e yazar. OTel auto-instrumentation Java agent (v2.10.0), manuel custom span'lar, custom metric'ler, structured logging ve Jaeger + Prometheus + Grafana ile zenginleştirilmiş.



36.2 Projede Uygulanan OpenTelemetry Pratikleri

36.2.1 Auto-Instrumentation (Phase 9)

OpenTelemetry Java Agent v2.10.0, infrastructure/otel-agent/ altında container image'a gömülü; Maven plugin <jvmArguments> ile -javaagent + otel.service.name set ediliyor. Bu yapılandırma şunları otomatik kapsıyor:

- Tomcat HTTP server (her gelen istek için SERVER span)
- WebClient (outbound HTTP, traceparent enjeksiyonu)
- Spring Data JPA + Hibernate
- JDBC driver (her SQL için CLIENT span)
- Logback MDC injection (trace_id JSON log'a otomatik)

36.2.2 Manual Custom Spans (Phase 10)

İş kuralı seviyesinde manuel span'lar:

- `alarm.validate` — fault-collector'da IP format + timestamp doğrulaması
- `alarm.process` — fault-processor'da işleme parent span'ı
- `severity.calculate` — Strategy Pattern içinde severity hesaplama child span'ı

Span attribute'ları: `alarm.id`, `alarm.type`, `alarm.source_ip`, `alarm.severity`, `alarm.age_minutes`. Hata durumlarında `setStatus(StatusCode.ERROR) + recordException(e)` pattern'i uygulanmış.

36.2.3 Custom Metrics (Phase 11)

Metric Adı	Tip	Label
<code>alarms.received.total</code>	Counter	<code>alarmType</code>
<code>alarms.processed.total</code>	Counter	<code>severityLevel</code>
<code>alarm.processing.duration</code>	Histogram	-
<code>alarms.pending.count</code>	Gauge	-

36.2.4 Severity Engine (Phase 8)

Strategy Pattern ile 5 SeverityStrategy implementasyonu (`LINK_DOWN`, `HIGH_LATENCY`, `PACKET_LOSS`, `NODE_UNREACHABLE`, `CONGESTION`). `simulate.sh` ile description varyantları üretiliyor; örnek `HIGH_LATENCY 500ms+ → HIGH severity`.

36.3 Argela Bağlamında Bu Dokümandaki Öğretilerin Uygulanması

36.3.1 İyileştirme Önerileri (Bölüm Referansları İle)

İyileştirme	İlgili Bölüm	Etki
Jaeger in-memory → Elasticsearch	18.2, CLAUDE Bilinen Kısıt #1	Trace kalıcı, geçmiş sorgulanabilir
BSP queue size 4096+	5.3.2, 33-#7	Yoğun saatte span kaybı azalır
Sampling: %100 → ParentBased %10	21.2, 24.2.1	Storage maliyeti 10x düşer
Tail sampling: errors + slow	21.4	Kritik trace'ler kaybolmaz
PII redaction processor	30.2.2	GDPR uyum
Spanmetrics processor	17.4	Manual metric'lere ihtiyaç azalır
Collector self-metric Prometheus scrape	13.11	Pipeline sağlığı izlenir
TLS aktif (cluster-internal)	29.1	Defense-in-depth
fault-collector healthcheck	CLAUDE Bilinen Kısıt #5 (çözüldü)	Hazırlık probu
SLO + burn rate alerting	32.3	Proaktif izleme

36.3.2 Production Rollout Yol Haritası

43. Phase 12 (logging) tamamlandıktan sonra: ELK/Loki entegrasyonu + Promtail
44. Jaeger storage → ES production-grade (en az 3 node)

45. OTEL Collector replica + HPA
46. Sampling %100 → %10 (head) + tail sampling
47. PII redaction policy code review zorunlu kılınsın
48. SLO tanımlama: fault-processor için %99.5 availability, p99 < 500ms
49. Kubernetes deployment + Helm chart
50. Multi-region disaster recovery

36.4 Argela İçin Önerilen Kod Snippetleri

36.4.1 Geliştirilmiş Collector Konfigi

```

receivers:
  otlp:
    protocols:
      grpc: { endpoint: 0.0.0.0:4317 }
      http: { endpoint: 0.0.0.0:4318 }

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 1024
    spike_limit_mib: 256

  resource:
    attributes:
      - key: service.namespace
        value: argela-fm
        action: insert
      - key: deployment.environment
        value: production
        action: upsert

  attributes/redact:
    actions:
      - key: http.request.header.authorization
        action: delete
      - key: http.request.header.cookie
        action: delete

# Yeni eklenmesi önerilen: spanmetrics
spanmetrics:
  metrics_exporter: prometheus
  latency_histogram_buckets: [10ms, 50ms, 100ms, 500ms, 1s]
  dimensions:
    - name: http.route
    - name: alarm.type

# Yeni eklenmesi önerilen: tail sampling
tail_sampling:
  decision_wait: 10s
  num_traces: 10000
  policies:
    - name: errors
      type: status_code
      status_code: { status_codes: [ERROR] }
    - name: slow
      type: latency
      latency: { threshold_ms: 1000 }

```

```

- name: critical_paths
  type: string_attribute
  string_attribute:
    key: http.route
    values: ["/api/v1/alarms", "/api/v1/process"]
- name: baseline
  type: probabilistic
  probabilistic: { sampling_percentage: 10 }

batch:
  timeout: 2s
  send_batch_size: 1024

exporters:
  otlp/jaeger:
    endpoint: jaeger:4317
    tls: { insecure: true }
  prometheus:
    endpoint: 0.0.0.0:8889
    resource_to_telemetry_conversion: { enabled: true }

extensions:
  health_check: { endpoint: 0.0.0.0:13133 }
  zpages: { endpoint: 0.0.0.0:55679 }
  file_storage:
    directory: /var/lib/otelcol/data

service:
  extensions: [health_check, zpages, file_storage]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource, attributes/redact, spanmetrics,
tail_sampling, batch]
      exporters: [otlp/jaeger]
    metrics:
      receivers: [otlp, spanmetrics]
      processors: [memory_limiter, resource, batch]
      exporters: [prometheus]
  telemetry:
    logs: { level: info }
    metrics:
      level: detailed
      address: 0.0.0.0:8888

```

36.4.2 Geliştirilmiş Spring Boot Konfigi

```

# application.yml
spring:
  application:
    name: fault-collector
  threads:
    virtual:
      enabled: true    # Java 21+ ile

management:
  tracing:
    enabled: false    # OTel agent tek kaynak
  metrics:
    enable:
      all: true

```



```

endpoints:
  web:
    exposure:
      include: health,info,prometheus
  endpoint:
    health:
      probes:
        enabled: true

# server.shutdown=graceful production'da kritik
server:
  shutdown: graceful

# OTEL env'leri (Dockerfile veya k8s deployment'ta set edilir)
# OTEL_SERVICE_NAME=fault-collector
#
OTEL_RESOURCE_ATTRIBUTES=service.version=1.2.0,service.namespace=argela-fm,deployment.environment=production
# OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
# OTEL_TRACES_SAMPLER=parentbased_traceidratio
# OTEL_TRACES_SAMPLER_ARG=1.0      # collector tail sampling karar veriyor
# OTEL_BSP_MAX_QUEUE_SIZE=8192
# OTEL_BSP_MAX_EXPORT_BATCH_SIZE=2048
# OTEL_BSP_SCHEDULE_DELAY=2000

```

36.4.3 Argela için Önerilen Custom Span Pattern'i

```

@Service
public class AlarmProcessorService {
    private final Tracer tracer =
GlobalOpenTelemetry.getTracer("argela.fault-processor");
    private final Meter meter = GlobalOpenTelemetry.getMeter("argela.fault-processor");

    private final LongCounter processedCounter = meter
        .counterBuilder("alarms.processed.total")
        .build();
    private final DoubleHistogram processingDuration = meter
        .histogramBuilder("alarm.processing.duration")
        .setUnit("ms")
        .build();

    public AlarmResponse process(AlarmRequest req) {
        Span parent = tracer.spanBuilder("alarm.process")
            .setSpanKind(SpanKind.INTERNAL)
            .setAttribute("alarm.id", req.getAlarmId())
            .setAttribute("alarm.type", req.getAlarmType().name())
            .setAttribute("alarm.source_ip", req.getSourceIp())
            .startSpan();

        long start = System.currentTimeMillis();
        try (Scope scope = parent.makeCurrent()) {

            SeverityLevel severity = calculateSeverity(req);
            parent.setAttribute("alarm.severity", severity.name());

            Alarm saved = persist(req, severity);
            parent.addEvent("alarm.persisted",
                Attributes.of(AttributeKey.longKey("alarm.db_id"), saved.getId()));

            processedCounter.add(1, Attributes.of(
                AttributeKey.stringKey("severityLevel"), severity.name(),

```

```

        AttributeKey.stringKey("alarmType"), req.getAlarmType().name()
    ));

    return AlarmResponse.from(saved);
} catch (Exception e) {
    parent.setStatus(StatusCode.ERROR, e.getMessage());
    parent.recordException(e);
    throw e;
} finally {
    long durationMs = System.currentTimeMillis() - start;
    processingDuration.record(durationMs);
    parent.end();
}
}

@WithSpan("severity.calculate")
private SeverityLevel calculateSeverity(@SpanAttribute("alarm.type") AlarmRequest
req) {
    return strategyMap.get(req.getAlarmType()).evaluate(req);
}
}

```

36.5 Argela için Önerilen Sonraki Adımlar

- Phase 12 (Structured Logging) tamamlanmış varsayımıyla Loki + Promtail entegrasyonu
- Phase 18 (önerilir): Jaeger storage Elasticsearch'e geçiş (Bilinen Kısıt #1)
- Phase 19 (önerilir): Sampling stratejisi (tail sampling + parentbased %10)
- Phase 20 (önerilir): SLO/SLI dashboard'u Grafana'da
- Phase 21 (önerilir): Kubernetes deployment + Helm chart + GitOps
- Phase 22 (önerilir): Security hardening — TLS, PII redaction, secret management

Bölüm 37 — Observability'nin Geleceği: eBPF, Profiles, AI

Bu kapanış bölümü gelecek 2-3 yılda observability alanında olan trendleri özetler. Argela ve benzeri ekiplerin orta vadeli yatırım planı için yön verir.

37.1 eBPF Tabanlı Observability

eBPF (extended Berkeley Packet Filter), Linux kernel'inde kod çalıştırarak kullanıcı uygulamasını instrument etmeyi sağlar. Avantaj: uygulama hiç değiştirilmez, dil-bağımsız, agent jar yok. Pixie, Cilium Hubble, Parca, Polar Signals gibi araçlar bu yaklaşımı kullanır.

eBPF'in OTel'i tamamen ikame edip etmeyeceği soruluyor. Şimdilik cevap: hayır — eBPF düşük seviye (syscall, network) instrumentation'da güçlü, ama 'iş kuralı' span'ları için yine OTel API gerek. İki yaklaşım birbirini tamamlar.

37.2 Continuous Profiling

Span ve metric istek-bazlı veridir. Profile ise işlemcide ne çalıştığını, hangi method'un kaç ns harcadığını gösterir. Async Profiler, JFR, Pyroscope, Polar Signals Parca gibi araçlar 'continuous profiling' yapar.

OpenTelemetry topluluğu 2024 sonu itibarıyla 'profiles' sinyalini spec'e ekleme çalışmasını sürdürüyor. Önümüzdeki 1-2 yıl içinde OTLP üzerinden profile verisinin de iletildiği 4-sinyal pipeline'ları yaygınlaşacak.

37.3 LLM Tabanlı Anomaly Detection ve Root Cause

Datadog Watchdog, Dynatrace Davis AI gibi vendor'lar trace + metric + log verisi üzerinde otomatik anomaly detection sunuyor. Açık kaynak tarafında Keplers, Prometheus Anomaly Detection deneyleri var. LLM tabanlı root cause analysis (örn. 'Bu trace neden yavaş?' diye sorulduğunda LLM trace içeriğini analiz edip cevap üretmesi) ilgi çekici bir alan.

Kısa vadede LLM'ler 'incident retrieval' (postmortem'lerden benzer vaka bulma) ve 'playbook generation' (alert için runbook üretme) gibi alanlarda fayda sağlıyor.

37.4 OTel'in Olgunlaşması

OpenTelemetry'nin önümüzdeki 2-3 yılına dair beklenen gelişmeler:

- Logs API stable olacak (şu an Beta)
- Profiles 4. sinyal olarak resmî
- OTel Operator daha olgun (zero-touch instrumentation)
- Collector için 'WASM processor' (özelleştirilebilir transform'lar runtime'da)
- OTLP/Arrow (yüksek throughput için yeni encoding)
- Real User Monitoring (RUM) için OTel spec'i

37.5 Maturity Beklentisi

Bir staj projesinin (Argela Fault Management) bugünkü L2-L3 seviyesinden, 1-2 yıl içinde L3-L4 seviyesine çıkması bekleniyor. Bu sıçramayı yapacak araçlar zaten OTel ekosisteminde var; mesele kültürel ve disiplin. Bu doküman tam bu sıçramaya rehber olmak için yazıldı.

37.6 Kapanış

OpenTelemetry, dağıtık sistemler dünyasının 'görme' duyusudur. Bir mühendislik ekibi observability'ye ne kadar yatırım yaparsa, o kadar hızlı sorun tespit eder, o kadar hızlı öğrenir, o kadar dayanıklı sistemler üretir. Bu doküman 130+ vaka, 37 bölüm, 100+ kod örneği ile yola çıktığınızda yanınızda taşımanız için tasarlandı.

Argela Fault Management staj projeniz, bu yolculuğun başlangıcıdır. Mentorunuzun "CTRL+F yapıp çözümü bulayım" beklentisini bu doküman karşılıyor olmalı. Sorularınız oldukça bu doküman güncellenebilir, genişletilebilir.

İyi observability!